Week8_Gueorgui_Poklitar

August 1, 2026

****Week 8 - K-Nearest Neighbors****

[1]:
```
%pip install -q pandas numpy matplotlib scikit-learn kagglehub
```

```
[notice] A new release of pip is
available: 26.0.1 -> 26.2
[notice] To update, run:
pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

[2]:
```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
import kagglehub

from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    f1_score,
    precision_recall_curve,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve
)
```

```
from sklearn.model_selection import (
    GridSearchCV,
    StratifiedKFold,
    cross_val_score,
    train_test_split
)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.utils.class_weight import compute_sample_weight

pd.set_option("display.max_columns", 100)
pd.set_option("display.width", 150)

print(f"scikit-learn version: {sklearn.__version__}")
print("All imports successful.")
```

```
scikit-learn version: 1.9.0
All imports successful.
```

Data Loading and Cleaning

[3]:
```
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")

df = df.drop_duplicates().copy()

df["person_emp_length"] = df["person_emp_length"].fillna(
    df["person_emp_length"].median()
)

df["loan_int_rate"] = df["loan_int_rate"].fillna(
    df.groupby("loan_grade")["loan_int_rate"].transform("median")
)

df = df[df["person_emp_length"] <= df["person_age"]]
df = df[df["person_age"] <= 100]
df = df[df["person_emp_length"] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate: {df['loan_status'].mean() * 100:.2f}%")
print("\nMissing values:")
print(df.isna().sum())
```

```
Raw shape: (32581, 12)
Clean shape: (32409, 12)
Default rate: 21.87%
```

```
Missing values:
person_age                    0
person_income                 0
person_home_ownership         0
person_emp_length             0
loan_intent                   0
loan_grade                    0
loan_amnt                     0
loan_int_rate                 0
loan_status                   0
loan_percent_income           0
cb_person_default_on_file     0
cb_person_cred_hist_length    0
dtype: int64
```

Classification Target and Feature Preparation

[4]:
```
numeric_features = [
    "person_income",
    "person_age",
    "person_emp_length",
    "loan_amnt",
    "loan_percent_income",
    "loan_int_rate"
]

categorical_features = [
    "loan_intent",
    "loan_grade",
    "person_home_ownership",
    "cb_person_default_on_file"
]

X = df[numeric_features + categorical_features].copy()
y = df["loan_status"].copy()

complete_mask = X.notna().all(axis=1)
X = X.loc[complete_mask]
y = y.loc[complete_mask]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

```
scaled_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ),
            categorical_features
        )
    ]
)

unscaled_preprocessor = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numeric_features),
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ),
            categorical_features
        )
    ]
)

numeric_only_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features)
    ]
)

X_train_enc = unscaled_preprocessor.fit_transform(X_train)
X_test_enc = unscaled_preprocessor.transform(X_test)

categorical_names = (
    unscaled_preprocessor.named_transformers_["cat"]
    .get_feature_names_out(categorical_features)
    .tolist()
)
feature_names = numeric_features + categorical_names

X_train_enc = np.asarray(X_train_enc, dtype=np.float32)
X_test_enc = np.asarray(X_test_enc, dtype=np.float32)
```

```
train_weights = compute_sample_weight(
    class_weight="balanced",
    y=y_train
)

print(f"Encoded features: {len(feature_names)}")
print(f"Training observations: {len(y_train):,}")
print(f"Testing observations: {len(y_test):,}")
print(f"Training default rate: {y_train.mean() * 100:.2f}%")
print(f"Testing default rate: {y_test.mean() * 100:.2f}%")
```

```
Encoded features: 25
Training observations: 25,927
Testing observations: 6,482
Training default rate: 21.87%
Testing default rate: 21.88%
```

The scale problem

[5]:
```
scale_summary = pd.DataFrame({
    "Feature": numeric_features,
    "Minimum": [X_train[column].min() for column in numeric_features],
    "Maximum": [X_train[column].max() for column in numeric_features],
    "Standard Deviation": [X_train[column].std() for column in numeric_features]
})

scale_summary["Range"] = (
    scale_summary["Maximum"]
    - scale_summary["Minimum"]
)

scale_summary = scale_summary.sort_values(
    "Standard Deviation",
    ascending=False
)

print("Raw Numeric Feature Scales")
print(scale_summary.round(3).to_string(index=False))

largest_std = scale_summary["Standard Deviation"].max()
smallest_std = scale_summary["Standard Deviation"].min()

print(
    "\nRatio of largest to smallest standard deviation:",
    f"{largest_std / smallest_std:,.0f} to 1"
)
print(
```

```
    "Without standardisation, one feature would dominate every "
    "distance computation in this notebook."
)
```

```
Raw Numeric Feature Scales
            Feature  Minimum    Maximum  Standard Deviation      Range
      person_income  4080.00 2039784.00           52539.326 2035704.00
          loan_amnt   500.00   35000.00            6293.701   34500.00
         person_age    20.00      94.00               6.164      74.00
  person_emp_length     0.00      38.00               3.969      38.00
      loan_int_rate     5.42      23.22               3.212      17.80
loan_percent_income     0.00       0.83               0.107       0.83

Ratio of largest to smallest standard deviation: 492,295 to 1
Without standardisation, one feature would dominate every distance computation
in this notebook.
```

Reusable Evaluation Function

[19]:
```
def evaluate_classifier(
    model_name,
    model,
    X_eval,
    y_eval,
    threshold=0.50
):
    probabilities = model.predict_proba(X_eval)[:, 1]
    predictions = (probabilities >= threshold).astype(int)

    results = {
        "Model": model_name,
        "Threshold": threshold,
        "Accuracy": accuracy_score(y_eval, predictions),
        "Balanced Accuracy": balanced_accuracy_score(y_eval, predictions),
        "ROC AUC": roc_auc_score(y_eval, probabilities),
        "Average Precision": average_precision_score(y_eval, probabilities),
        "Default Precision": precision_score(
            y_eval,
            predictions,
            zero_division=0
        ),
        "Default Recall": recall_score(
            y_eval,
            predictions,
            zero_division=0
        ),
        "Default F1": f1_score(
            y_eval,
```

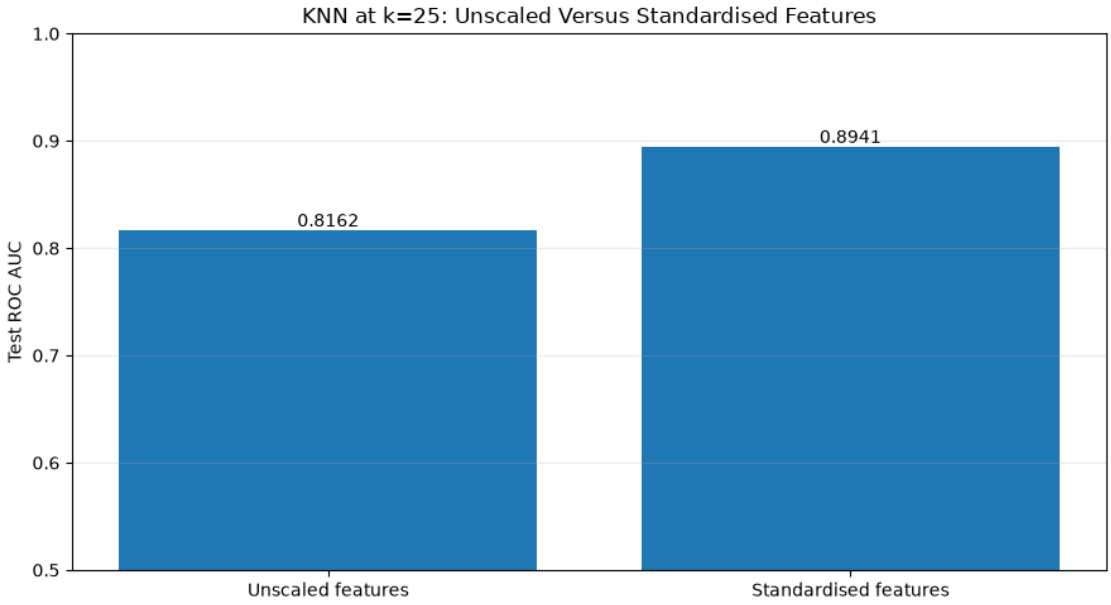
```
            predictions,
            zero_division=0
        )
    }

    return results, probabilities, predictions


def build_knn_pipeline(preprocessor, **knn_settings):
    return Pipeline(
        steps=[
            ("prep", preprocessor),
            (
                "knn",
                KNeighborsClassifier(
                    n_jobs=-1,
                    **knn_settings
                )
            )
        ]
    )


def cross_validated_scores(
    pipeline,
    X_data,
    y_data,
    n_splits=4
):
    cv = StratifiedKFold(
        n_splits=n_splits,
        shuffle=True,
        random_state=42
    )

    auc_scores = cross_val_score(
        pipeline,
        X_data,
        y_data,
        cv=cv,
        scoring="roc_auc",
        n_jobs=-1
    )

    return auc_scores.mean(), auc_scores.std()
```

Why a Distance-Based Model Requires Scaling

[20]:
```
scaling_comparison_results = []

scaling_configurations = [
    ("Unscaled features", unscaled_preprocessor),
    ("Standardised features", scaled_preprocessor)
]

for label, preprocessor in scaling_configurations:
    pipeline = build_knn_pipeline(
        preprocessor=preprocessor,
        n_neighbors=25,
        metric="minkowski",
        p=2,
        weights="uniform"
    )

    mean_auc, std_auc = cross_validated_scores(
        pipeline=pipeline,
        X_data=X_train,
        y_data=y_train,
        n_splits=4
    )

    pipeline.fit(X_train, y_train)

    test_results, _, _ = evaluate_classifier(
        model_name=f"KNN (k=25) - {label}",
        model=pipeline,
        X_eval=X_test,
        y_eval=y_test
    )

    scaling_comparison_results.append({
        "Preprocessing": label,
        "CV AUC Mean": mean_auc,
        "CV AUC Std": std_auc,
        "Test AUC": test_results["ROC AUC"],
        "Test Average Precision": test_results["Average Precision"],
        "Default Recall": test_results["Default Recall"]
    })

scaling_comparison_df = pd.DataFrame(scaling_comparison_results)

print("Effect of Feature Scaling on KNN")
print(scaling_comparison_df.round(4).to_string(index=False))

scaling_gain = (
```

```
    scaling_comparison_df.loc[1, "Test AUC"]
    - scaling_comparison_df.loc[0, "Test AUC"]
)

print(
    f"\nStandardisation changes test AUC by {scaling_gain:+.4f}."
)

plt.figure(figsize=(9, 5))
plt.bar(
    scaling_comparison_df["Preprocessing"],
    scaling_comparison_df["Test AUC"]
)
for index, value in enumerate(scaling_comparison_df["Test AUC"]):
    plt.text(
        index,
        value + 0.004,
        f"{value:.4f}",
        ha="center"
    )
plt.ylabel("Test ROC AUC")
plt.ylim(0.5, 1.0)
plt.title("KNN at k=25: Unscaled Versus Standardised Features")
plt.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Effect of Feature Scaling on KNN
        Preprocessing  CV AUC Mean  CV AUC Std  Test AUC  Test Average Precision
Default Recall
    Unscaled features       0.8199      0.0037    0.8162                  0.6235
0.4492
Standardised features       0.8893      0.0018    0.8941                  0.8029
0.5959

Standardisation changes test AUC by +0.0780.
```



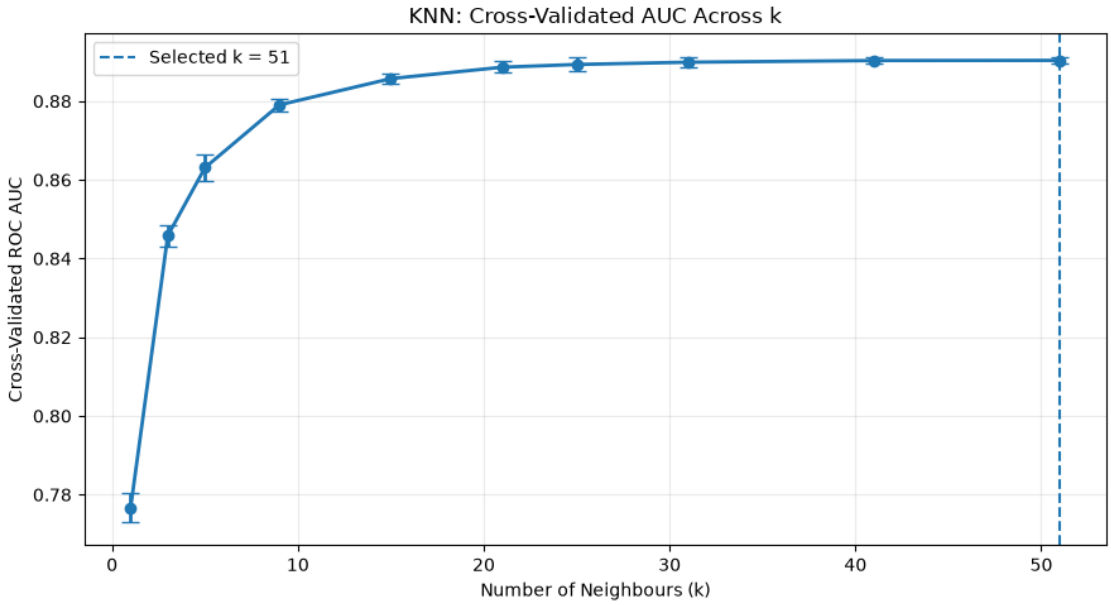
Choosing k with Cross-Validation

[8]:
```
neighbour_values = [1, 3, 5, 9, 15, 21, 25, 31, 41, 51]

neighbour_results = []

for k in neighbour_values:
    pipeline = build_knn_pipeline(
        preprocessor=scaled_preprocessor,
        n_neighbors=k,
        metric="minkowski",
        p=2,
        weights="uniform"
    )

    mean_auc, std_auc = cross_validated_scores(
        pipeline=pipeline,
        X_data=X_train,
        y_data=y_train,
        n_splits=4
    )

    neighbour_results.append({
        "Neighbours": k,
        "CV AUC Mean": mean_auc,
        "CV AUC Std": std_auc
    })
```

```python
neighbour_df = pd.DataFrame(neighbour_results)

print("Cross-Validated AUC by Number of Neighbours")
print(neighbour_df.round(4).to_string(index=False))

best_neighbour_row = neighbour_df.loc[
    neighbour_df["CV AUC Mean"].idxmax()
]

best_k = int(best_neighbour_row["Neighbours"])

print(
    f"\nBest k by cross-validated AUC: {best_k} "
    f"(CV AUC = {best_neighbour_row['CV AUC Mean']:.4f})"
)

plt.figure(figsize=(9, 5))
plt.errorbar(
    neighbour_df["Neighbours"],
    neighbour_df["CV AUC Mean"],
    yerr=neighbour_df["CV AUC Std"],
    marker="o",
    capsize=5,
    linewidth=2
)
plt.axvline(
    best_k,
    linestyle="--",
    label=f"Selected k = {best_k}"
)
plt.xlabel("Number of Neighbours (k)")
plt.ylabel("Cross-Validated ROC AUC")
plt.title("KNN: Cross-Validated AUC Across k")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Cross-Validated AUC by Number of Neighbours
 Neighbours  CV AUC Mean  CV AUC Std
          1       0.7767      0.0037
          3       0.8458      0.0028
          5       0.8631      0.0033
          9       0.8790      0.0016
         15       0.8857      0.0014
         21       0.8886      0.0014
         25       0.8893      0.0018
```

```
31        0.8899      0.0012
41        0.8903      0.0008
51        0.8903      0.0009

Best k by cross-validated AUC: 51 (CV AUC = 0.8903)
```



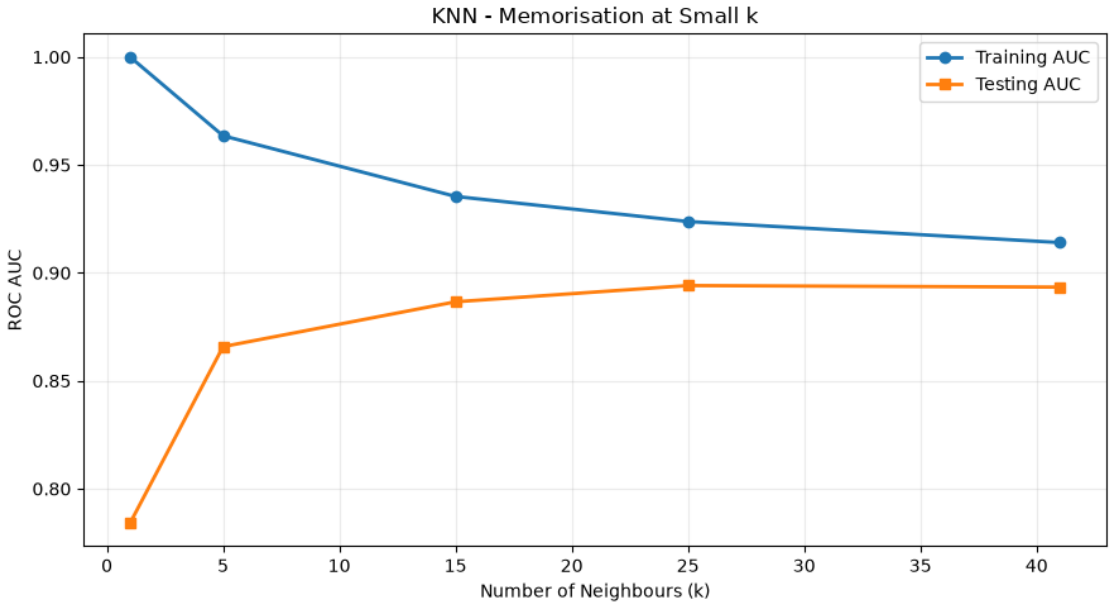
The Overfitting Signature of Small k

[9]:
```python
diagnostic_neighbour_values = [1, 5, 15, 25, 41]

diagnostic_results = []

for k in diagnostic_neighbour_values:
    pipeline = build_knn_pipeline(
        preprocessor=scaled_preprocessor,
        n_neighbors=k,
        metric="minkowski",
        p=2,
        weights="uniform"
    )

    pipeline.fit(X_train, y_train)

    train_probabilities = pipeline.predict_proba(X_train)[:, 1]
    test_probabilities = pipeline.predict_proba(X_test)[:, 1]

    diagnostic_results.append({
```

```
        "Neighbours": k,
        "Train AUC": roc_auc_score(y_train, train_probabilities),
        "Test AUC": roc_auc_score(y_test, test_probabilities)
    })

diagnostic_df = pd.DataFrame(diagnostic_results)
diagnostic_df["Train Minus Test"] = (
    diagnostic_df["Train AUC"]
    - diagnostic_df["Test AUC"]
)

print("Training Versus Test AUC Across k")
print(diagnostic_df.round(4).to_string(index=False))

print(
    "\nThe train-test gap at k=1:",
    f"{diagnostic_df.loc[0, 'Train Minus Test']:.4f}"
)
print(
    "The train-test gap at the widest neighbourhood tested:",
    f"{diagnostic_df.iloc[-1]['Train Minus Test']:.4f}"
)

plt.figure(figsize=(9, 5))
plt.plot(
    diagnostic_df["Neighbours"],
    diagnostic_df["Train AUC"],
    marker="o",
    linewidth=2,
    label="Training AUC"
)
plt.plot(
    diagnostic_df["Neighbours"],
    diagnostic_df["Test AUC"],
    marker="s",
    linewidth=2,
    label="Testing AUC"
)
plt.xlabel("Number of Neighbours (k)")
plt.ylabel("ROC AUC")
plt.title("KNN - Memorisation at Small k")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Training Versus Test AUC Across k
 Neighbours  Train AUC  Test AUC  Train Minus Test
```

```
  1     1.0000     0.7841           0.2159
  5     0.9635     0.8658           0.0977
 15     0.9354     0.8866           0.0488
 25     0.9238     0.8941           0.0297
 41     0.9140     0.8934           0.0206

The train-test gap at k=1: 0.2159
The train-test gap at the widest neighbourhood tested: 0.0206
```



Distance Metric Comparison

- **Euclidean (Minkowski p=2)** takes the straight-line distance and, because differences are squared, is dominated by the feature on which two borrowers differ most.
- **Manhattan (Minkowski p=1)** sums absolute differences, treating a large gap on one feature as equivalent to several modest gaps spread across many. It is generally the more robust choice when the data contain outliers, which the income distribution here certainly does.
- **Minkowski p=3** pushes further in the euclidean direction, weighting the single largest disagreement even more heavily.
- **Chebyshev** takes this to its limit and considers *only* the largest single-feature difference, discarding all other information.
- **Cosine** ignores magnitude entirely and compares direction. Two borrowers with the same profile shape at different absolute scales are treated as neighbours.

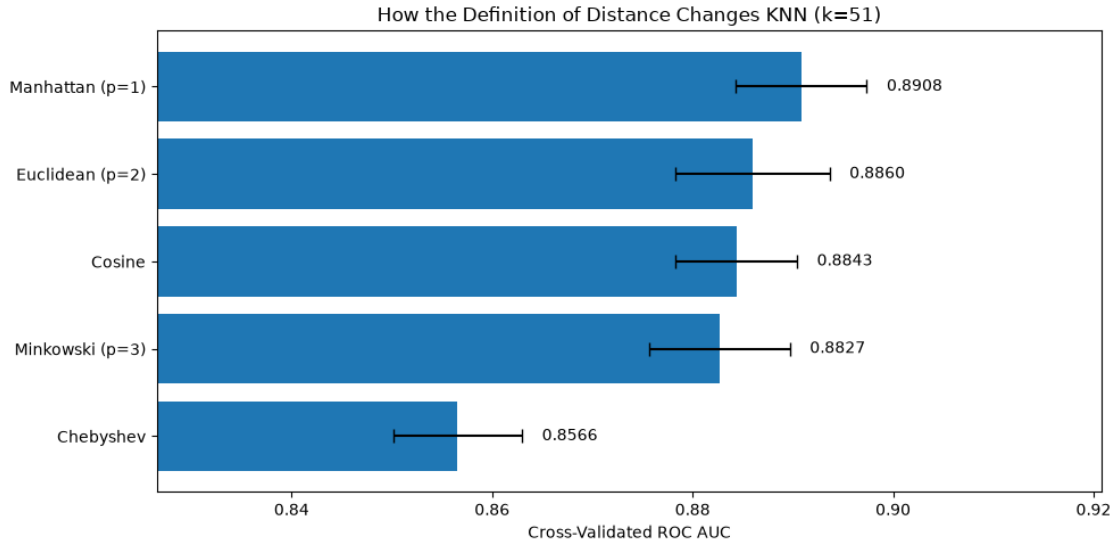
[15]:
```
metric_subsample_size = 12000

subsample_indices, _ = train_test_split(
    np.arange(len(y_train)),
```

```
    train_size=metric_subsample_size,
    random_state=42,
    stratify=np.asarray(y_train)
)

X_metric = X_train.iloc[subsample_indices]
y_metric = y_train.iloc[subsample_indices]

print(f"Metric comparison subsample: {len(y_metric):,} borrowers")
print(f"Subsample default rate: {y_metric.mean() * 100:.2f}%")

distance_metric_settings = [
    {
        "Label": "Manhattan (p=1)",
        "Settings": {"metric": "minkowski", "p": 1}
    },
    {
        "Label": "Euclidean (p=2)",
        "Settings": {"metric": "minkowski", "p": 2}
    },
    {
        "Label": "Minkowski (p=3)",
        "Settings": {"metric": "minkowski", "p": 3}
    },
    {
        "Label": "Chebyshev",
        "Settings": {"metric": "chebyshev"}
    },
    {
        "Label": "Cosine",
        "Settings": {"metric": "cosine", "algorithm": "brute"}
    }
]

metric_results = []

for entry in distance_metric_settings:
    pipeline = build_knn_pipeline(
        preprocessor=scaled_preprocessor,
        n_neighbors=best_k,
        weights="uniform",
        **entry["Settings"]
    )

    mean_auc, std_auc = cross_validated_scores(
        pipeline=pipeline,
        X_data=X_metric,
```

```
        y_data=y_metric,
        n_splits=4
    )

    metric_results.append({
        "Distance Metric": entry["Label"],
        "CV AUC Mean": mean_auc,
        "CV AUC Std": std_auc
    })

metric_df = pd.DataFrame(metric_results).sort_values(
    "CV AUC Mean",
    ascending=False
).reset_index(drop=True)

print(f"\nDistance Metric Comparison at k={best_k}")
print(metric_df.round(4).to_string(index=False))

best_metric_label = metric_df.loc[0, "Distance Metric"]
metric_spread = (
    metric_df["CV AUC Mean"].max()
    - metric_df["CV AUC Mean"].min()
)

print(f"\nStrongest metric: {best_metric_label}")
print(f"Spread between best and worst metric: {metric_spread:.4f} AUC")

metric_order = metric_df.sort_values("CV AUC Mean")

plt.figure(figsize=(10, 5))
plt.barh(
    metric_order["Distance Metric"],
    metric_order["CV AUC Mean"],
    xerr=metric_order["CV AUC Std"],
    capsize=4
)


for index, (value, std) in enumerate(zip(metric_order["CV AUC Mean"],␣
 ↪metric_order["CV AUC Std"])):
    plt.text(
        value + std + 0.002,
        index,
        f"{value:.4f}",
        va="center"
    )
```

```
plt.xlabel("Cross-Validated ROC AUC")
plt.title(f"How the Definition of Distance Changes KNN (k={best_k})")
plt.xlim(
    metric_order["CV AUC Mean"].min() - 0.03,
    metric_order["CV AUC Mean"].max() + 0.03
)
plt.tight_layout()
plt.show()
```

```
Metric comparison subsample: 12,000 borrowers
Subsample default rate: 21.87%


Distance Metric Comparison at k=51
Distance Metric  CV AUC Mean  CV AUC Std
Manhattan (p=1)       0.8908      0.0065
Euclidean (p=2)       0.8860      0.0077
         Cosine       0.8843      0.0061
Minkowski (p=3)       0.8827      0.0070
      Chebyshev       0.8566      0.0064

Strongest metric: Manhattan (p=1)
Spread between best and worst metric: 0.0343 AUC
```



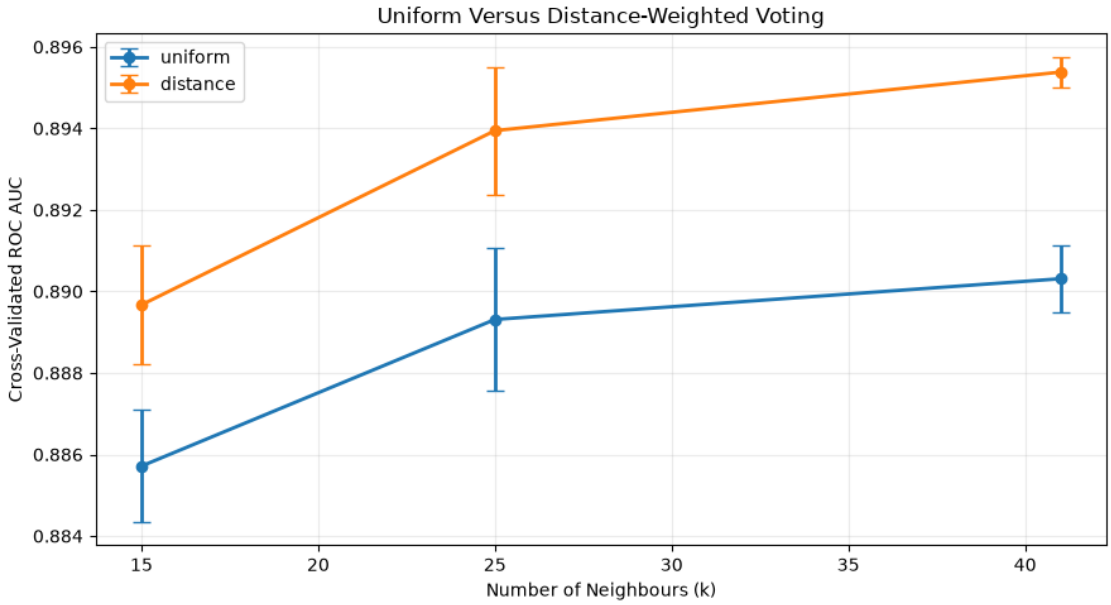
Uniform Versus Distance Weighting

[11]:
```
weighting_neighbour_values = [15, 25, 41]

weighting_results = []
```

```
for k in weighting_neighbour_values:
    for weighting in ["uniform", "distance"]:
        pipeline = build_knn_pipeline(
            preprocessor=scaled_preprocessor,
            n_neighbors=k,
            metric="minkowski",
            p=2,
            weights=weighting
        )

        mean_auc, std_auc = cross_validated_scores(
            pipeline=pipeline,
            X_data=X_train,
            y_data=y_train,
            n_splits=4
        )

        weighting_results.append({
            "Neighbours": k,
            "Weighting": weighting,
            "CV AUC Mean": mean_auc,
            "CV AUC Std": std_auc
        })

weighting_df = pd.DataFrame(weighting_results)

print("Neighbour Weighting Comparison")
print(weighting_df.round(4).to_string(index=False))

weighting_pivot = weighting_df.pivot(
    index="Neighbours",
    columns="Weighting",
    values="CV AUC Mean"
)

weighting_pivot["Distance Advantage"] = (
    weighting_pivot["distance"]
    - weighting_pivot["uniform"]
)

print("\nDistance Weighting Advantage by k")
print(weighting_pivot.round(4).to_string())

plt.figure(figsize=(9, 5))
for weighting in ["uniform", "distance"]:
    subset = weighting_df[weighting_df["Weighting"] == weighting]
    plt.errorbar(
```

```
        subset["Neighbours"],
        subset["CV AUC Mean"],
        yerr=subset["CV AUC Std"],
        marker="o",
        capsize=5,
        linewidth=2,
        label=weighting
    )
plt.xlabel("Number of Neighbours (k)")
plt.ylabel("Cross-Validated ROC AUC")
plt.title("Uniform Versus Distance-Weighted Voting")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Neighbour Weighting Comparison
 Neighbours Weighting  CV AUC Mean  CV AUC Std
         15   uniform       0.8857      0.0014
         15  distance       0.8897      0.0015
         25   uniform       0.8893      0.0018
         25  distance       0.8939      0.0016
         41   uniform       0.8903      0.0008
         41  distance       0.8954      0.0004

Distance Weighting Advantage by k
Weighting   distance  uniform  Distance Advantage
Neighbours
15            0.8897   0.8857              0.0040
25            0.8939   0.8893              0.0046
41            0.8954   0.8903              0.0051
```



Dimensionality and the Encoded Feature Space

[31]:
```
feature_space_configurations = [
    (
        "Numeric only (6 features)",
        numeric_only_preprocessor
    ),
    (
        f"Numeric plus one-hot ({len(feature_names)} features)",
        scaled_preprocessor
    )
]
feature_space_results = []
for label, preprocessor in feature_space_configurations:
    pipeline = build_knn_pipeline(
        preprocessor=preprocessor,
        n_neighbors=best_k,
        metric="minkowski",
        p=2,
        weights="distance"
    )
    mean_auc, std_auc = cross_validated_scores(
        pipeline=pipeline,
        X_data=X_train,
        y_data=y_train,
        n_splits=4
    )
```

```
    pipeline.fit(X_train, y_train)
    test_results, _, _ = evaluate_classifier(
        model_name=f"KNN - {label}",
        model=pipeline,
        X_eval=X_test,
        y_eval=y_test
    )
    feature_space_results.append({
        "Feature Space": label,
        "CV AUC Mean": mean_auc,
        "CV AUC Std": std_auc,
        "Test AUC": test_results["ROC AUC"],
        "Test Average Precision": test_results["Average Precision"]
    })
feature_space_df = pd.DataFrame(feature_space_results)
print("Effect of the Encoded Feature Space on KNN")
print(feature_space_df.round(4).to_string(index=False))
feature_space_difference = (
    feature_space_df.loc[1, "CV AUC Mean"]
    - feature_space_df.loc[0, "CV AUC Mean"]
)
print(
    "\nAdding the one-hot categoricals changes CV AUC by",
    f"{feature_space_difference:+.4f}."
)
if feature_space_difference > 0.002:
    selected_preprocessor = scaled_preprocessor
    selected_space_label = "numeric plus one-hot"
    print(
        " The categorical indicators contribute usable signal "
        "to the distance metric and are retained."
    )
elif feature_space_difference < -0.002:
    selected_preprocessor = numeric_only_preprocessor
    selected_space_label = "numeric only"
    print(
        "The categorical indicators dilute the distance metric. "
        "The numeric-only space is the better representation for KNN."
    )
else:
    selected_preprocessor = scaled_preprocessor
    selected_space_label = "numeric plus one-hot"
    print(
        "The two feature spaces are effectively tied, so the "
        "fuller representation is retained for comparability with the "
        "tree-based models of Weeks 6 and 9."
    )
```

```
print(f"Feature space carried into tuning: {selected_space_label}")
```

```
Effect of the Encoded Feature Space on KNN
                     Feature Space  CV AUC Mean  CV AUC Std  Test AUC  Test
Average Precision
         Numeric only (6 features)       0.8817      0.0012    0.8884
0.7588
Numeric plus one-hot (25 features)       0.8955      0.0003    0.9003
0.8199

Adding the one-hot categoricals changes CV AUC by +0.0138.
 The categorical indicators contribute usable signal to the distance metric and
are retained.
Feature space carried into tuning: numeric plus one-hot
```

[33]:
```
feature_space_configurations = [
    (
        "Numeric only (6 features)",
        numeric_only_preprocessor
    ),
    (
        f"Numeric plus one-hot ({len(feature_names)} features)",
        scaled_preprocessor
    )
]

feature_space_results = []

for label, preprocessor in feature_space_configurations:
    pipeline = build_knn_pipeline(
        preprocessor=preprocessor,
        n_neighbors=best_k,
        metric="minkowski",
        p=2,
        weights="distance"
    )

    mean_auc, std_auc = cross_validated_scores(
        pipeline=pipeline,
        X_data=X_train,
        y_data=y_train,
        n_splits=4
    )

    pipeline.fit(X_train, y_train)

    test_results, _, _ = evaluate_classifier(
```

```
        model_name=f"KNN - {label}",
        model=pipeline,
        X_eval=X_test,
        y_eval=y_test
    )

    feature_space_results.append({
        "Feature Space": label,
        "CV AUC Mean": mean_auc,
        "CV AUC Std": std_auc,
        "Test AUC": test_results["ROC AUC"],
        "Test Average Precision": test_results["Average Precision"]
    })

feature_space_df = pd.DataFrame(feature_space_results)

print("Effect of the Encoded Feature Space on KNN")
print(feature_space_df.round(4).to_string(index=False))

feature_space_difference = (
    feature_space_df.loc[1, "CV AUC Mean"]
    - feature_space_df.loc[0, "CV AUC Mean"]
)

print(
    "\nAdding the one-hot categoricals changes CV AUC by",
    f"{feature_space_difference:+.4f}."
)

if feature_space_difference > 0.002:
    selected_preprocessor = scaled_preprocessor
    selected_space_label = "numeric plus one-hot"
    print(
        "The categorical indicators contribute usable signal "
        "to the distance metric and are retained."
    )
elif feature_space_difference < -0.002:
    selected_preprocessor = numeric_only_preprocessor
    selected_space_label = "numeric only"
    print(
        "The categorical indicators dilute the distance metric. "
        "The numeric-only space is the better representation for KNN."
    )
else:
    selected_preprocessor = scaled_preprocessor
    selected_space_label = "numeric plus one-hot"
    print(
```

```
        "Possibly the two feature spaces are effectively tied, so the "
        "fuller representation is retained for comparability with the "
        "tree-based models of Weeks 6 and 9."
    )

print(f"Feature space carried into tuning: {selected_space_label}")
```

```
Effect of the Encoded Feature Space on KNN
                     Feature Space  CV AUC Mean  CV AUC Std  Test AUC  Test
Average Precision
         Numeric only (6 features)       0.8817      0.0012    0.8884
0.7588
Numeric plus one-hot (25 features)       0.8955      0.0003    0.9003
0.8199

Adding the one-hot categoricals changes CV AUC by +0.0138.
The categorical indicators contribute usable signal to the distance metric and
are retained.
Feature space carried into tuning: numeric plus one-hot
```

Hyperparameter Tuning with Stratified Cross-Validation

[14]:
```
parameter_grid = [
    {
        "knn__n_neighbors": [15, 25, 35],
        "knn__weights": ["uniform", "distance"],
        "knn__metric": ["minkowski"],
        "knn__p": [1, 2]
    },
    {
        "knn__n_neighbors": [15, 25, 35],
        "knn__weights": ["uniform", "distance"],
        "knn__metric": ["cosine"],
        "knn__algorithm": ["brute"]
    }
]

base_search_pipeline = build_knn_pipeline(
    preprocessor=selected_preprocessor
)

stratified_cv = StratifiedKFold(
    n_splits=4,
    shuffle=True,
    random_state=42
)

knn_search = GridSearchCV(
```

```
    estimator=base_search_pipeline,
    param_grid=parameter_grid,
    scoring={
        "roc_auc": "roc_auc",
        "average_precision": "average_precision"
    },
    refit="roc_auc",
    cv=stratified_cv,
    n_jobs=1, # CHANGED: Set to 1 to prevent Out of Memory (OOM) crashes
    verbose=1,
    return_train_score=True
)

# If memory still crashes even with n_jobs=1, replace X_train/y_train with␣
 ↪X_metric/y_metric below
knn_search.fit(X_train, y_train)

print("\nBest Parameters")
print(knn_search.best_params_)
print(f"\nBest Cross-Validated AUC: {knn_search.best_score_:.4f}")

search_results = pd.DataFrame(knn_search.cv_results_)

search_summary = search_results[
    [
        "rank_test_roc_auc",
        "mean_test_roc_auc",
        "std_test_roc_auc",
        "mean_test_average_precision",
        "mean_train_roc_auc",
        "params"
    ]
].sort_values("rank_test_roc_auc").head(10)

print("\nTop Hyperparameter Combinations")
print(search_summary.round(4).to_string(index=False))

best_row = search_results.loc[
    search_results["rank_test_roc_auc"].idxmin()
]

print(
    "\nTrain minus validation AUC for the selected model:",
    f"{best_row['mean_train_roc_auc'] - best_row['mean_test_roc_auc']:+.4f}"
)
```

```
Fitting 4 folds for each of 18 candidates, totalling 72 fits
```

```
Best Parameters
{'knn__metric': 'minkowski', 'knn__n_neighbors': 35, 'knn__p': 1,
'knn__weights': 'distance'}

Best Cross-Validated AUC: 0.9017

Top Hyperparameter Combinations
 rank_test_roc_auc  mean_test_roc_auc  std_test_roc_auc
mean_test_average_precision  mean_train_roc_auc
params
                 1             0.9017            0.0014
0.8258              1.0000            {'knn__metric': 'minkowski',
'knn__n_neighbors': 35, 'knn__p': 1, 'knn__weights': 'distance'}
                 2             0.9002            0.0013
0.8271              1.0000            {'knn__metric': 'minkowski',
'knn__n_neighbors': 25, 'knn__p': 1, 'knn__weights': 'distance'}
                 3             0.8968            0.0018
0.8065              0.9199             {'knn__metric': 'minkowski',
'knn__n_neighbors': 35, 'knn__p': 1, 'knn__weights': 'uniform'}
                 4             0.8955            0.0014
0.8063              0.9260             {'knn__metric': 'minkowski',
'knn__n_neighbors': 25, 'knn__p': 1, 'knn__weights': 'uniform'}
                 5             0.8951            0.0010
0.8133              1.0000            {'knn__metric': 'minkowski',
'knn__n_neighbors': 35, 'knn__p': 2, 'knn__weights': 'distance'}
                 6             0.8950            0.0018
0.8142              1.0000 {'knn__algorithm': 'brute', 'knn__metric': 'cosine',
'knn__n_neighbors': 35, 'knn__weights': 'distance'}
                 7             0.8947            0.0019
0.8239              1.0000            {'knn__metric': 'minkowski',
'knn__n_neighbors': 15, 'knn__p': 1, 'knn__weights': 'distance'}
                 8             0.8944            0.0024
0.8147              1.0000 {'knn__algorithm': 'brute', 'knn__metric': 'cosine',
'knn__n_neighbors': 25, 'knn__weights': 'distance'}
                 9             0.8939            0.0016
0.8149              1.0000            {'knn__metric': 'minkowski',
'knn__n_neighbors': 25, 'knn__p': 2, 'knn__weights': 'distance'}
                10             0.8903            0.0026
0.7994              0.9371             {'knn__metric': 'minkowski',
'knn__n_neighbors': 15, 'knn__p': 1, 'knn__weights': 'uniform'}

Train minus validation AUC for the selected model: +0.0983
```

Evaluate the Tuned KNN Model

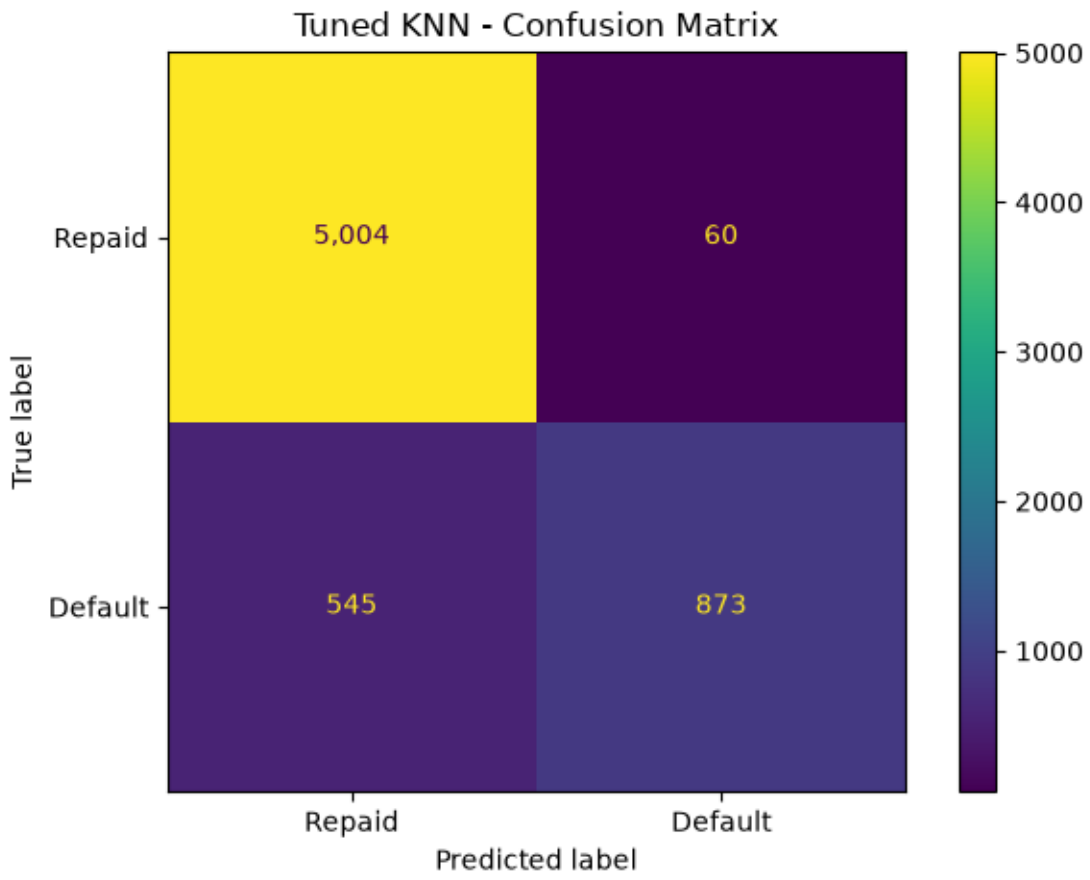
[21]:
```
best_knn = knn_search.best_estimator_

tuned_results, tuned_probabilities, tuned_predictions = evaluate_classifier(
```

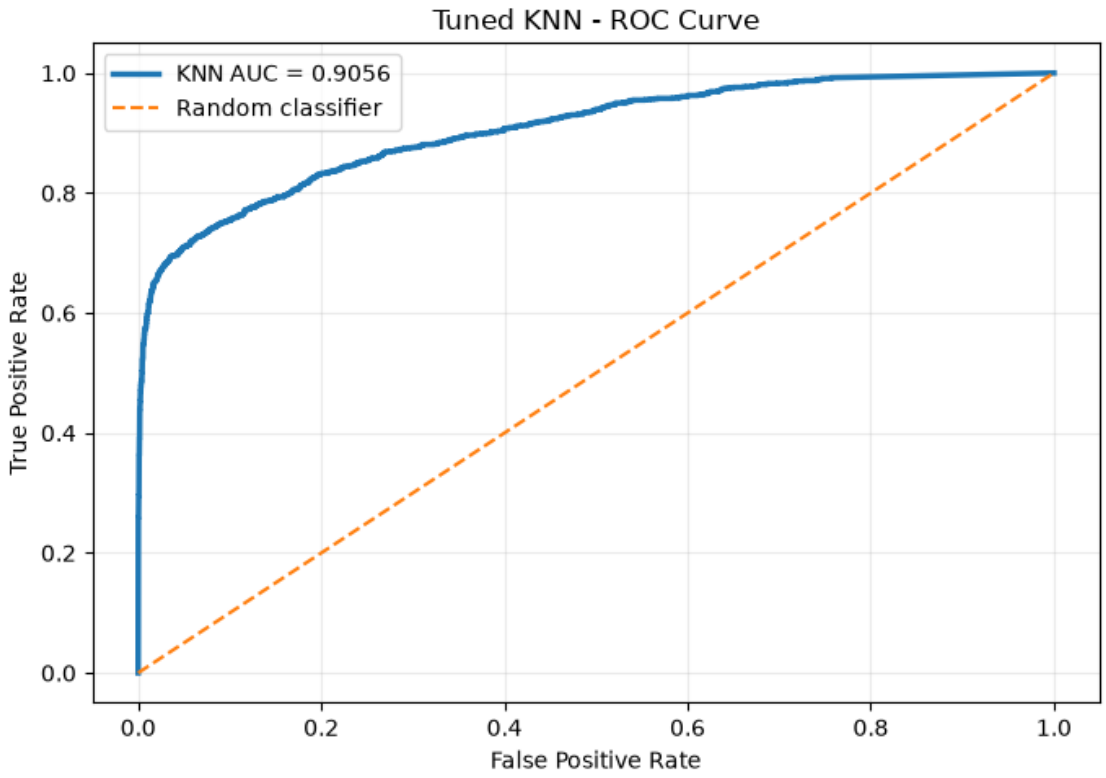
```
    model_name="Tuned KNN",
    model=best_knn,
    X_eval=X_test,
    y_eval=y_test,
    threshold=0.50
)

print("Tuned KNN")
print("-" * 40)

for metric, value in tuned_results.items():
    if metric not in ["Model", "Threshold"]:
        print(f"{metric:<22}: {value:.4f}")

print("\nClassification Report")
print(
    classification_report(
        y_test,
        tuned_predictions,
        target_names=["Repaid", "Default"]
    )
)

ConfusionMatrixDisplay.from_predictions(
    y_test,
    tuned_predictions,
    display_labels=["Repaid", "Default"],
    values_format=",d"
)
plt.title("Tuned KNN - Confusion Matrix")
plt.tight_layout()
plt.show()

false_positive_rate, true_positive_rate, _ = roc_curve(
    y_test,
    tuned_probabilities
)

plt.figure(figsize=(7, 5))
plt.plot(
    false_positive_rate,
    true_positive_rate,
    linewidth=2.5,
    label=f"KNN AUC = {tuned_results['ROC AUC']:.4f}"
)
plt.plot(
    [0, 1],
```

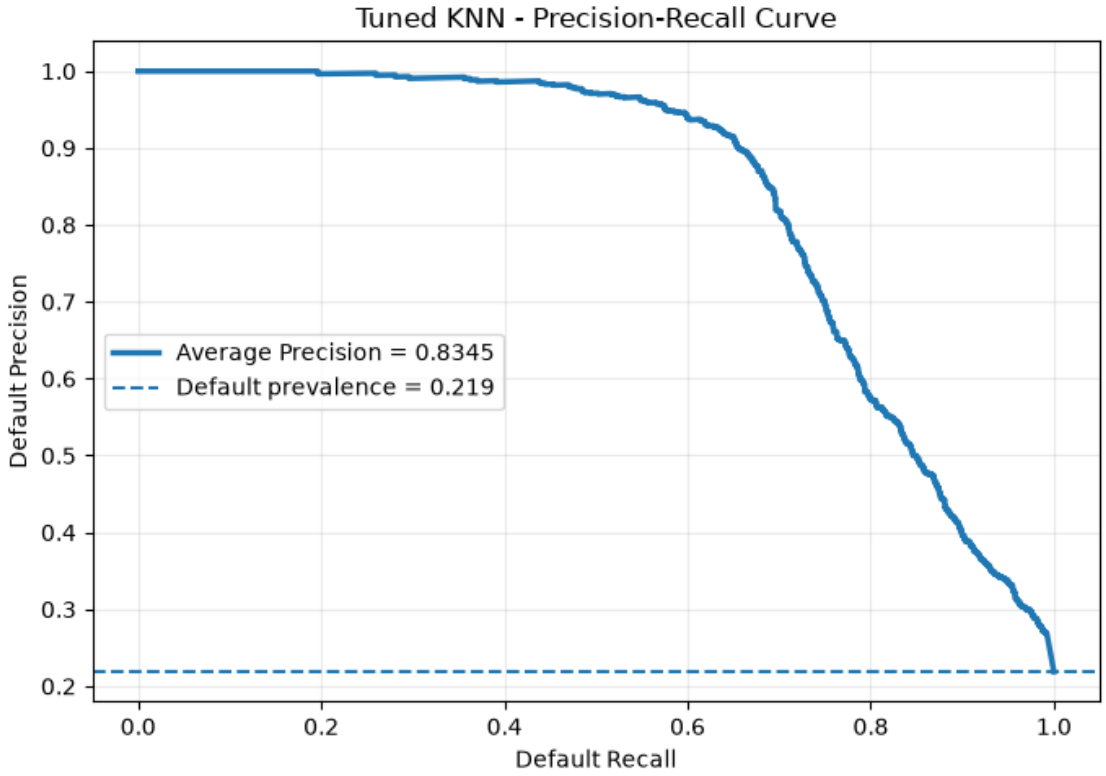
```
    [0, 1],
    linestyle="--",
    label="Random classifier"
)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Tuned KNN - ROC Curve")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()

test_precision_curve, test_recall_curve, _ = precision_recall_curve(
    y_test,
    tuned_probabilities
)

plt.figure(figsize=(7, 5))
plt.plot(
    test_recall_curve,
    test_precision_curve,
    linewidth=2.5,
    label=(
        "Average Precision = "
        f"{tuned_results['Average Precision']:.4f}"
    )
)
plt.axhline(
    y=y_test.mean(),
    linestyle="--",
    label=f"Default prevalence = {y_test.mean():.3f}"
)
plt.xlabel("Default Recall")
plt.ylabel("Default Precision")
plt.title("Tuned KNN - Precision-Recall Curve")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Tuned KNN
----------------------------------------
Accuracy              : 0.9067
Balanced Accuracy     : 0.8019
ROC AUC               : 0.9056
Average Precision     : 0.8345
Default Precision     : 0.9357
Default Recall        : 0.6157
```

```
Default F1            : 0.7427

Classification Report
              precision    recall  f1-score   support

      Repaid       0.90      0.99      0.94      5064
     Default       0.94      0.62      0.74      1418

    accuracy                           0.91      6482
   macro avg       0.92      0.80      0.84      6482
weighted avg       0.91      0.91      0.90      6482
```




Tuned KNN - ROC Curve
KNN AUC = 0.9056
Random classifier
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0


Tuned KNN - Precision-Recall Curve
Average Precision = 0.8345
Default prevalence = 0.219
Default Precision
Default Recall
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0

Selecting a Classification Threshold on Validation Data

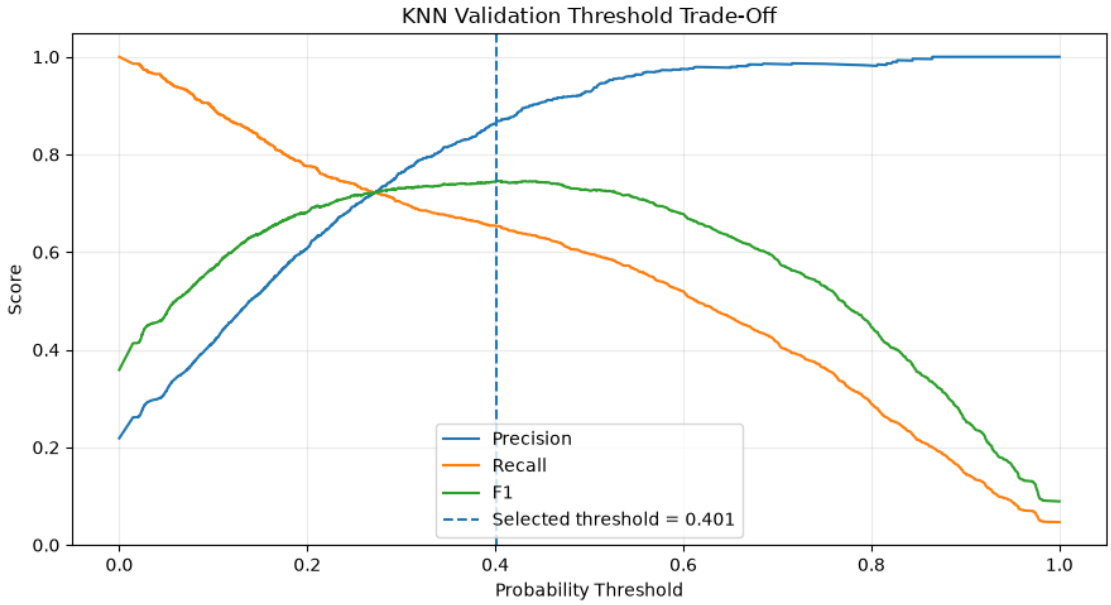
[22]:
```
y_train_array = np.asarray(y_train)
all_training_indices = np.arange(len(y_train_array))

threshold_fit_indices, threshold_validation_indices = train_test_split(
    all_training_indices,
    test_size=0.20,
    random_state=42,
    stratify=y_train_array
)

threshold_model = clone(best_knn)

threshold_model.fit(
    X_train.iloc[threshold_fit_indices],
    y_train_array[threshold_fit_indices]
)

validation_probabilities = threshold_model.predict_proba(
    X_train.iloc[threshold_validation_indices]
)[:, 1]

print(
    "Distinct probability values available from this neighbourhood vote:",
    f"{len(np.unique(validation_probabilities))}"
)

validation_precision, validation_recall, validation_thresholds = (
    precision_recall_curve(
        y_train_array[threshold_validation_indices],
        validation_probabilities
    )
)

validation_f1 = (
    2
    * validation_precision[:-1]
    * validation_recall[:-1]
    / (
        validation_precision[:-1]
        + validation_recall[:-1]
        + 1e-12
    )
)
```

```
best_threshold_index = int(np.nanargmax(validation_f1))
selected_threshold = float(
    validation_thresholds[best_threshold_index]
)

print(f"\nValidation-selected threshold: {selected_threshold:.4f}")
print(
    "Validation F1 at selected threshold:",
    f"{validation_f1[best_threshold_index]:.4f}"
)

threshold_results, _, threshold_predictions = evaluate_classifier(
    model_name="Tuned KNN - Validation Threshold",
    model=best_knn,
    X_eval=X_test,
    y_eval=y_test,
    threshold=selected_threshold
)

threshold_comparison = pd.DataFrame(
    [tuned_results, threshold_results]
)

print("\nThreshold Comparison")
print(
    threshold_comparison[
        [
            "Model",
            "Threshold",
            "Accuracy",
            "Balanced Accuracy",
            "Default Precision",
            "Default Recall",
            "Default F1"
        ]
    ].round(4).to_string(index=False)
)

print("\nClassification Report at Validation-Selected Threshold")
print(
    classification_report(
        y_test,
        threshold_predictions,
        target_names=["Repaid", "Default"]
    )
)
```

```
plt.figure(figsize=(9, 5))
plt.plot(
    validation_thresholds,
    validation_precision[:-1],
    label="Precision"
)
plt.plot(
    validation_thresholds,
    validation_recall[:-1],
    label="Recall"
)
plt.plot(
    validation_thresholds,
    validation_f1,
    label="F1"
)
plt.axvline(
    selected_threshold,
    linestyle="--",
    label=f"Selected threshold = {selected_threshold:.3f}"
)
plt.xlabel("Probability Threshold")
plt.ylabel("Score")
plt.title("KNN Validation Threshold Trade-Off")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Distinct probability values available from this neighbourhood vote: 4217

Validation-selected threshold: 0.4015
Validation F1 at selected threshold: 0.7458

Threshold Comparison
                           Model  Threshold  Accuracy  Balanced Accuracy
Default Precision  Default Recall  Default F1
                       Tuned KNN     0.5000    0.9067             0.8019
0.9357          0.6157      0.7427
Tuned KNN - Validation Threshold     0.4015    0.9073             0.8244
0.8704          0.6770      0.7616

Classification Report at Validation-Selected Threshold
              precision    recall  f1-score   support

      Repaid       0.91      0.97      0.94      5064
     Default       0.87      0.68      0.76      1418
```

```
    accuracy                           0.91      6482
   macro avg       0.89      0.82      0.85      6482
weighted avg       0.91      0.91      0.90      6482
```

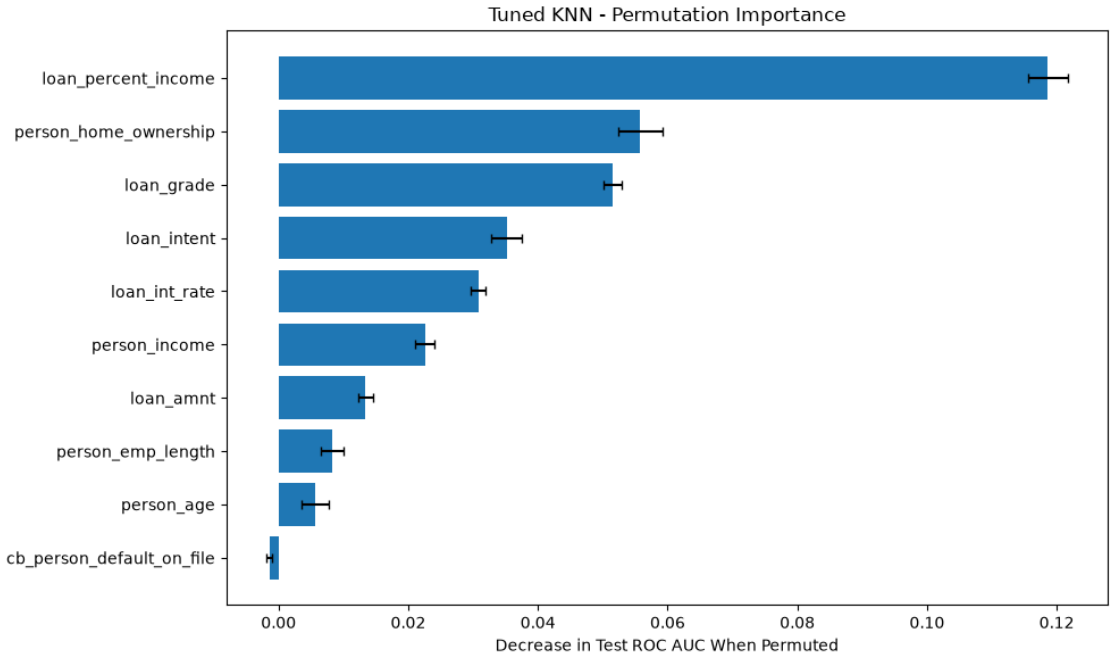


Permutation Importance

[23]:
```
permutation_results = permutation_importance(
    estimator=best_knn,
    X=X_test,
    y=y_test,
    scoring="roc_auc",
    n_repeats=5,
    random_state=42,
    n_jobs=-1
)

permutation_importance_df = pd.DataFrame({
    "Feature": X_test.columns,
    "Permutation Importance": permutation_results.importances_mean,
    "Importance Std": permutation_results.importances_std
}).sort_values(
    "Permutation Importance",
    ascending=False
)
```

```
print("KNN Permutation Importance")
print(permutation_importance_df.round(5).to_string(index=False))

top_permutation = permutation_importance_df.sort_values(
    "Permutation Importance"
)

plt.figure(figsize=(10, 6))
plt.barh(
    top_permutation["Feature"],
    top_permutation["Permutation Importance"],
    xerr=top_permutation["Importance Std"],
    capsize=3
)
plt.xlabel("Decrease in Test ROC AUC When Permuted")
plt.title("Tuned KNN - Permutation Importance")
plt.tight_layout()
plt.show()
```

```
KNN Permutation Importance
                 Feature  Permutation Importance  Importance Std
      loan_percent_income                 0.11865         0.00302
    person_home_ownership                 0.05573         0.00341
               loan_grade                 0.05153         0.00133
              loan_intent                 0.03518         0.00232
            loan_int_rate                 0.03083         0.00114
            person_income                 0.02255         0.00142
                loan_amnt                 0.01340         0.00110
        person_emp_length                 0.00822         0.00178
               person_age                 0.00561         0.00209
cb_person_default_on_file                -0.00141         0.00047
```



Benchmark Against Weeks 6 and 9

[34]:
```
random_forest = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_leaf=5,
    class_weight="balanced",
    n_jobs=-1,
    random_state=42
)

random_forest.fit(
    X_train_enc,
    y_train
)

rf_results, rf_probabilities, rf_predictions = evaluate_classifier(
    model_name="Random Forest - Week 6 Specification",
    model=random_forest,
    X_eval=X_test_enc,
    y_eval=y_test,
    threshold=0.50
)

gradient_boosting = GradientBoostingClassifier(
    subsample=0.85,
```

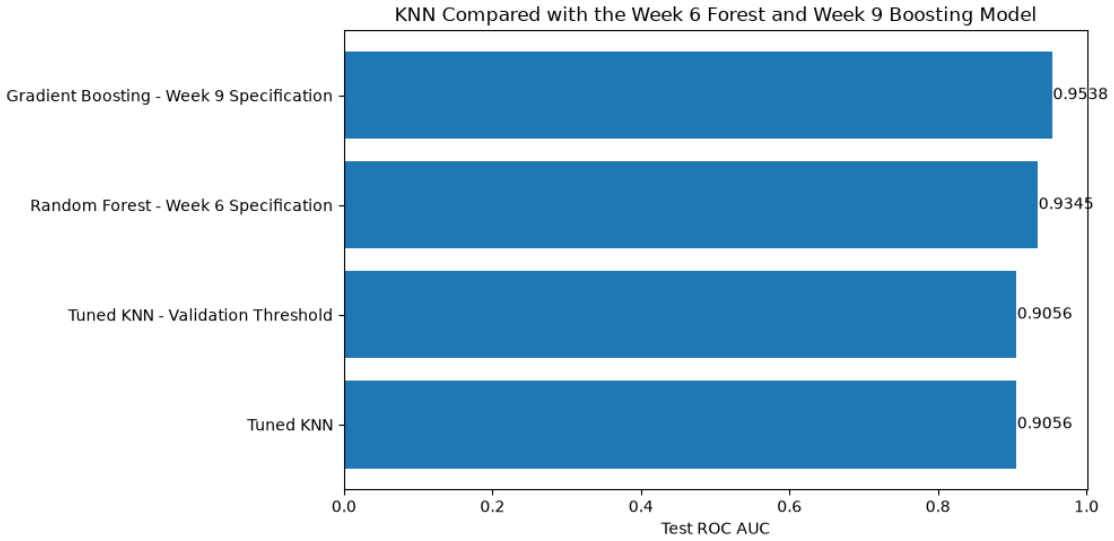
```
    n_estimators=300,
    min_samples_leaf=3,
    max_features=None,
    max_depth=4,
    learning_rate=0.10,
    random_state=42
)

gradient_boosting.fit(
    X_train_enc,
    y_train,
    sample_weight=train_weights
)

gb_results, gb_probabilities, gb_predictions = evaluate_classifier(
    model_name="Gradient Boosting - Week 9 Specification",
    model=gradient_boosting,
    X_eval=X_test_enc,
    y_eval=y_test,
    threshold=0.50
)

model_comparison = pd.DataFrame([
    tuned_results,
    threshold_results,
    rf_results,
    gb_results
]).sort_values(
    "ROC AUC",
    ascending=False
).reset_index(drop=True)

comparison_columns = [
    "Model",
    "ROC AUC",
    "Average Precision",
    "Accuracy",
    "Balanced Accuracy",
    "Default Precision",
    "Default Recall",
    "Default F1"
]

print("Week 8 Model Leaderboard")
print(
    model_comparison[comparison_columns]
    .round(4)
```

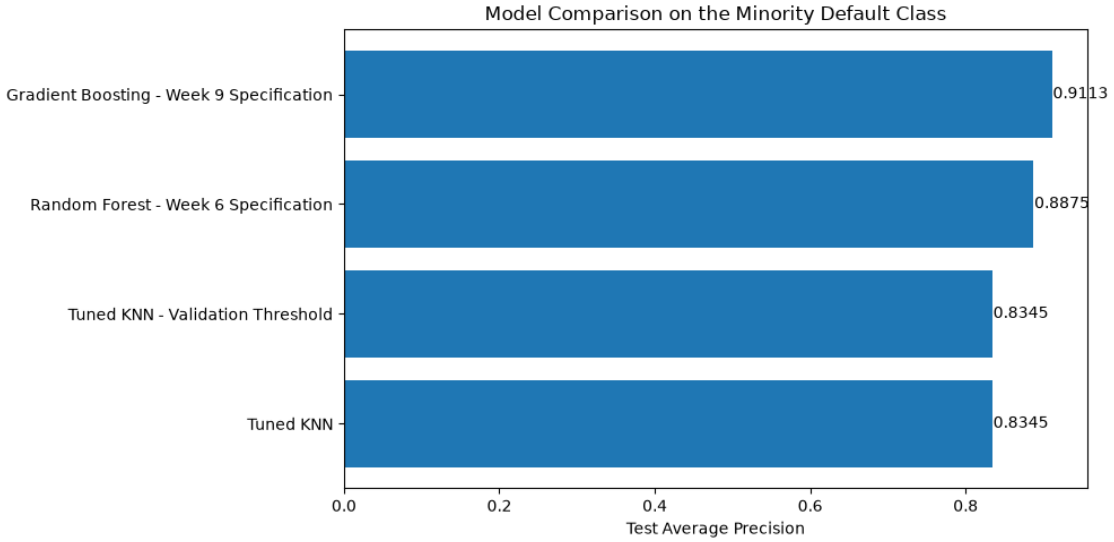
```
    .to_string(index=False)
)

auc_order = model_comparison.sort_values("ROC AUC")

plt.figure(figsize=(10, 5))
plt.barh(
    auc_order["Model"],
    auc_order["ROC AUC"]
)
for index, value in enumerate(auc_order["ROC AUC"]):
    plt.text(
        value + 0.001,
        index,
        f"{value:.4f}",
        va="center"
    )
plt.xlabel("Test ROC AUC")
plt.title("KNN Compared with the Week 6 Forest and Week 9 Boosting Model")
plt.tight_layout()
plt.show()

ap_order = model_comparison.sort_values("Average Precision")

plt.figure(figsize=(10, 5))
plt.barh(
    ap_order["Model"],
    ap_order["Average Precision"]
)
for index, value in enumerate(ap_order["Average Precision"]):
    plt.text(
        value + 0.001,
        index,
        f"{value:.4f}",
        va="center"
    )
plt.xlabel("Test Average Precision")
plt.title("Model Comparison on the Minority Default Class")
plt.tight_layout()
plt.show()

knn_gb_difference = (
    tuned_results["ROC AUC"]
    - gb_results["ROC AUC"]
)

knn_rf_difference = (
```

```
    tuned_results["ROC AUC"]
    - rf_results["ROC AUC"]
)

print(
    f"\nTuned KNN minus Week 9 Gradient Boosting AUC: {knn_gb_difference:+.4f}"
)
print(
    f"Tuned KNN minus Week 6 Random Forest AUC: {knn_rf_difference:+.4f}"
)

if knn_gb_difference > 0.002:
    print(
        "A purely local, memory-based method outranks the "
        "boosted ensemble on this test set, which would be a notable result "
        "for a model that fits no global structure whatsoever."
    )
elif knn_gb_difference < -0.002:
    print(
        "Gradient boosting retains the stronger risk ranking. "
        "KNN recovers much of the available signal from local similarity "
        "alone, but the sequential ensemble captures feature interactions "
        "that neighbourhood voting cannot express."
    )
else:
    print(
        "KNN and gradient boosting rank borrower risk almost "
        "identically, so prediction speed, interpretability, and the cost "
        "of storing the training set become the deciding criteria."
    )

print(
    "\n KNN carries no trained parameters, so the entire training "
    "set must be retained and searched at prediction time. For a lender "
    "scoring applications in real time, that operational cost is a genuine "
    "consideration alongside AUC."
)
```

```
Week 8 Model Leaderboard
                                    Model  ROC AUC  Average Precision  Accuracy
Balanced Accuracy  Default Precision  Default Recall  Default F1
Gradient Boosting - Week 9 Specification   0.9538             0.9113    0.9148
0.8813             0.7958          0.8216      0.8085
    Random Forest - Week 6 Specification   0.9345             0.8875    0.9185
0.8659             0.8423          0.7722      0.8057
        Tuned KNN - Validation Threshold   0.9056             0.8345    0.9073
0.8244             0.8704          0.6770      0.7616
                                Tuned KNN   0.9056             0.8345    0.9067
```

0.8019 0.9357 0.6157 0.7427





Tuned KNN minus Week 9 Gradient Boosting AUC: -0.0481
Tuned KNN minus Week 6 Random Forest AUC: -0.0289
Gradient boosting retains the stronger risk ranking. KNN recovers much of the available signal from local similarity alone, but the sequential ensemble captures feature interactions that neighbourhood voting cannot express.

KNN carries no trained parameters, so the entire training set must be retained and searched at prediction time. For a lender scoring applications in real time,

that operational cost is a genuine consideration alongside AUC.

In [20]:
```
%pip install -q pandas numpy matplotlib scikit-learn kagglehub
```

```
[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

In [21]:
```
# Week 9 - Gradient Boosting

import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
import kagglehub

from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassif
from sklearn.inspection import permutation_importance, PartialDependenceDisp
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    f1_score,
    precision_recall_curve,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve
)
from sklearn.model_selection import (
    RandomizedSearchCV,
    StratifiedKFold,
    train_test_split
)
from sklearn.preprocessing import OneHotEncoder
from sklearn.utils.class_weight import compute_sample_weight

pd.set_option("display.max_columns", 100)
pd.set_option("display.width", 150)

print(f"scikit-learn version: {sklearn.__version__}")
print("All imports successful.")
```

```
scikit-learn version: 1.9.0
All imports successful.
```

In [22]:
```
# 1. Data Loading and Cleaning

path = kagglehub.dataset_download("laotse/credit-risk-dataset")
```

```
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")

# Remove duplicate records
df = df.drop_duplicates().copy()

# Median imputation for employment length
df["person_emp_length"] = df["person_emp_length"].fillna(
    df["person_emp_length"].median()
)

# Grade-matched median imputation for interest rate
df["loan_int_rate"] = df["loan_int_rate"].fillna(
    df.groupby("loan_grade")["loan_int_rate"].transform("median")
)

# Remove logically impossible observations
df = df[df["person_emp_length"] <= df["person_age"]]
df = df[df["person_age"] <= 100]
df = df[df["person_emp_length"] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate: {df['loan_status'].mean() * 100:.2f}%")
print("\nMissing values:")
print(df.isna().sum())
```

```
Raw shape: (32581, 12)
Clean shape: (32409, 12)
Default rate: 21.87%

Missing values:
person_age                    0
person_income                 0
person_home_ownership         0
person_emp_length             0
loan_intent                   0
loan_grade                    0
loan_amnt                     0
loan_int_rate                 0
loan_status                   0
loan_percent_income           0
cb_person_default_on_file     0
cb_person_cred_hist_length    0
dtype: int64
```

In [23]:
```
# 2. Classification Target and Feature Preparation

numeric_features = [
    "person_income",
    "person_age",
    "person_emp_length",
    "loan_amnt",
    "loan_percent_income",
    "loan_int_rate"
]
```

```
categorical_features = [
    "loan_intent",
    "loan_grade",
    "person_home_ownership",
    "cb_person_default_on_file"
]

X = df[numeric_features + categorical_features].copy()
y = df["loan_status"].copy()

# Keep complete modeling rows
complete_mask = X.notna().all(axis=1)
X = X.loc[complete_mask]
y = y.loc[complete_mask]

# Preserve the same stratified split used in the previous classification not
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# Gradient-boosted trees do not require scaling.
# Categorical variables are one-hot encoded into numeric indicators.
encoder = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numeric_features),
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ),
            categorical_features
        )
    ]
)

X_train_enc = encoder.fit_transform(X_train)
X_test_enc = encoder.transform(X_test)

categorical_names = (
    encoder.named_transformers_["cat"]
    .get_feature_names_out(categorical_features)
    .tolist()
)

feature_names = numeric_features + categorical_names

# Use float32 to reduce memory consumption
X_train_enc = np.asarray(X_train_enc, dtype=np.float32)
X_test_enc = np.asarray(X_test_enc, dtype=np.float32)
```

```
# GradientBoostingClassifier does not use a class_weight argument.
# Balanced sample weights give greater influence to the minority default cla
train_weights = compute_sample_weight(
    class_weight="balanced",
    y=y_train
)

print(f"Encoded features: {len(feature_names)}")
print(f"Training observations: {len(y_train):,}")
print(f"Testing observations: {len(y_test):,}")
print(f"Training default rate: {y_train.mean() * 100:.2f}%")
print(f"Testing default rate: {y_test.mean() * 100:.2f}%")
```

```
Encoded features: 25
Training observations: 25,927
Testing observations: 6,482
Training default rate: 21.87%
Testing default rate: 21.88%
```

In [24]:

```
# 3. Reusable Evaluation Function

def evaluate_classifier(
    model_name,
    model,
    X_eval,
    y_eval,
    threshold=0.50
):
    probabilities = model.predict_proba(X_eval)[:, 1]
    predictions = (probabilities >= threshold).astype(int)

    results = {
        "Model": model_name,
        "Threshold": threshold,
        "Accuracy": accuracy_score(y_eval, predictions),
        "Balanced Accuracy": balanced_accuracy_score(y_eval, predictions),
        "ROC AUC": roc_auc_score(y_eval, probabilities),
        "Average Precision": average_precision_score(y_eval, probabilities),
        "Default Precision": precision_score(
            y_eval,
            predictions,
            zero_division=0
        ),
        "Default Recall": recall_score(
            y_eval,
            predictions,
            zero_division=0
        ),
        "Default F1": f1_score(
            y_eval,
            predictions,
            zero_division=0
        )
    }

    return results, probabilities, predictions
```

In [25]:

```python
# 4. Baseline Gradient Boosting Classifier

baseline_gb = GradientBoostingClassifier(
    random_state=42
)

baseline_gb.fit(
    X_train_enc,
    y_train,
    sample_weight=train_weights
)

baseline_results, baseline_probabilities, baseline_predictions = (
    evaluate_classifier(
        model_name="Baseline Gradient Boosting",
        model=baseline_gb,
        X_eval=X_test_enc,
        y_eval=y_test
    )
)

print("Baseline Gradient Boosting")
print("-" * 40)

for metric, value in baseline_results.items():
    if metric not in ["Model", "Threshold"]:
        print(f"{metric:<22}: {value:.4f}")

print("\nClassification Report")
print(
    classification_report(
        y_test,
        baseline_predictions,
        target_names=["Repaid", "Default"]
    )
)
```

```
Baseline Gradient Boosting
----------------------------------------
Accuracy              : 0.8854
Balanced Accuracy     : 0.8530
ROC AUC               : 0.9323
Average Precision     : 0.8779
Default Precision     : 0.7135
Default Recall        : 0.7955
Default F1            : 0.7523

Classification Report
              precision    recall  f1-score   support

      Repaid       0.94      0.91      0.93      5064
     Default       0.71      0.80      0.75      1418

    accuracy                           0.89      6482
   macro avg       0.83      0.85      0.84      6482
weighted avg       0.89      0.89      0.89      6482
```

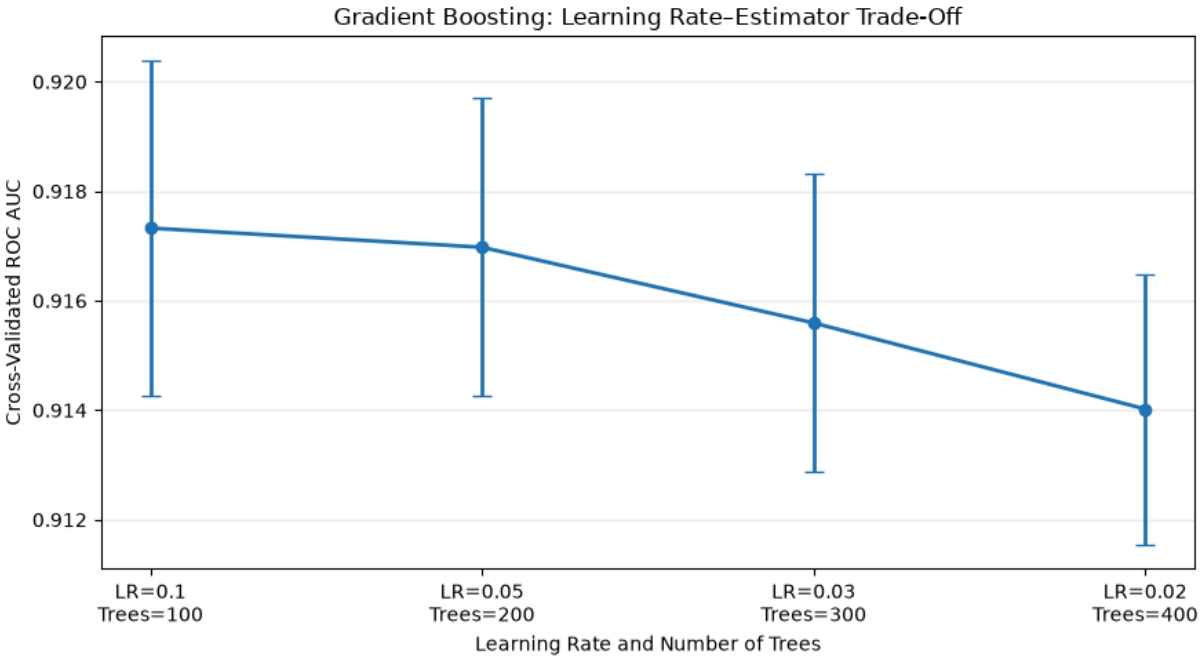
In [26]:

```python
# 5. Learning Rate and Number-of-Trees Experiment

def weighted_cross_validated_auc(
    model,
    X_data,
    y_data,
    n_splits=4
):
    y_array = np.asarray(y_data)

    cv = StratifiedKFold(
        n_splits=n_splits,
        shuffle=True,
        random_state=42
    )

    fold_scores = []

    for fit_indices, validation_indices in cv.split(X_data, y_array):
        fold_model = clone(model)

        fold_weights = compute_sample_weight(
            class_weight="balanced",
            y=y_array[fit_indices]
        )

        fold_model.fit(
            X_data[fit_indices],
            y_array[fit_indices],
            sample_weight=fold_weights
        )

        validation_probabilities = fold_model.predict_proba(
            X_data[validation_indices]
        )[:, 1]
```

```
        fold_auc = roc_auc_score(
            y_array[validation_indices],
            validation_probabilities
        )

        fold_scores.append(fold_auc)

    return np.mean(fold_scores), np.std(fold_scores)


learning_rate_settings = [
    {"learning_rate": 0.10, "n_estimators": 100},
    {"learning_rate": 0.05, "n_estimators": 200},
    {"learning_rate": 0.03, "n_estimators": 300},
    {"learning_rate": 0.02, "n_estimators": 400}
]

learning_rate_results = []

for setting in learning_rate_settings:
    model = GradientBoostingClassifier(
        learning_rate=setting["learning_rate"],
        n_estimators=setting["n_estimators"],
        max_depth=2,
        min_samples_leaf=5,
        subsample=0.85,
        random_state=42
    )

    mean_auc, std_auc = weighted_cross_validated_auc(
        model=model,
        X_data=X_train_enc,
        y_data=y_train,
        n_splits=4
    )

    learning_rate_results.append({
        "Learning Rate": setting["learning_rate"],
        "Number of Trees": setting["n_estimators"],
        "CV AUC Mean": mean_auc,
        "CV AUC Std": std_auc
    })

learning_rate_df = pd.DataFrame(learning_rate_results)

print(learning_rate_df.round(4).to_string(index=False))

labels = [
    f"LR={row['Learning Rate']}\nTrees={int(row['Number of Trees'])}"
    for _, row in learning_rate_df.iterrows()
]

plt.figure(figsize=(9, 5))
plt.errorbar(
    labels,
```

```
    learning_rate_df["CV AUC Mean"],
    yerr=learning_rate_df["CV AUC Std"],
    marker="o",
    capsize=5,
    linewidth=2
)
plt.xlabel("Learning Rate and Number of Trees")
plt.ylabel("Cross-Validated ROC AUC")
plt.title("Gradient Boosting: Learning Rate–Estimator Trade-Off")
plt.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.show()

best_experiment = learning_rate_df.loc[
    learning_rate_df["CV AUC Mean"].idxmax()
]

print(
    "\nBest learning-rate experiment:",
    f"learning_rate={best_experiment['Learning Rate']},",
    f"n_estimators={int(best_experiment['Number of Trees'])},",
    f"CV AUC={best_experiment['CV AUC Mean']:.4f}"
)
```

Learning Rate	Number of Trees	CV AUC Mean	CV AUC Std
0.10	100	0.9173	0.0031
0.05	200	0.9170	0.0027
0.03	300	0.9156	0.0027
0.02	400	0.9140	0.0025




```
Best learning-rate experiment: learning_rate=0.1, n_estimators=100, CV AUC=
0.9173
```

In [27]:
```
# 6. Hyperparameter Tuning with Stratified Cross-Validation

parameter_distributions = {
    "n_estimators": [100, 150, 200, 250, 300, 400],
```

```
    "learning_rate": [0.02, 0.03, 0.05, 0.075, 0.10],
    "max_depth": [1, 2, 3, 4],
    "min_samples_leaf": [3, 5, 10, 20],
    "subsample": [0.70, 0.85, 1.00],
    "max_features": [None, "sqrt", 0.70]
}

base_search_model = GradientBoostingClassifier(
    random_state=42
)

stratified_cv = StratifiedKFold(
    n_splits=4,
    shuffle=True,
    random_state=42
)

gradient_search = RandomizedSearchCV(
    estimator=base_search_model,
    param_distributions=parameter_distributions,
    n_iter=18,
    scoring={
        "roc_auc": "roc_auc",
        "average_precision": "average_precision"
    },
    refit="roc_auc",
    cv=stratified_cv,
    n_jobs=-1,
    random_state=42,
    verbose=1,
    return_train_score=True
)

gradient_search.fit(
    X_train_enc,
    y_train,
    sample_weight=train_weights
)

print("\nBest Parameters")
print(gradient_search.best_params_)

print(f"\nBest Cross-Validated AUC: {gradient_search.best_score_:.4f}")

search_results = pd.DataFrame(gradient_search.cv_results_)

search_summary = search_results[
    [
        "rank_test_roc_auc",
        "mean_test_roc_auc",
        "std_test_roc_auc",
        "mean_test_average_precision",
        "mean_train_roc_auc",
        "params"
    ]
].sort_values("rank_test_roc_auc").head(10)
```

```
print("\nTop Hyperparameter Combinations")
print(search_summary.round(4).to_string(index=False))
```

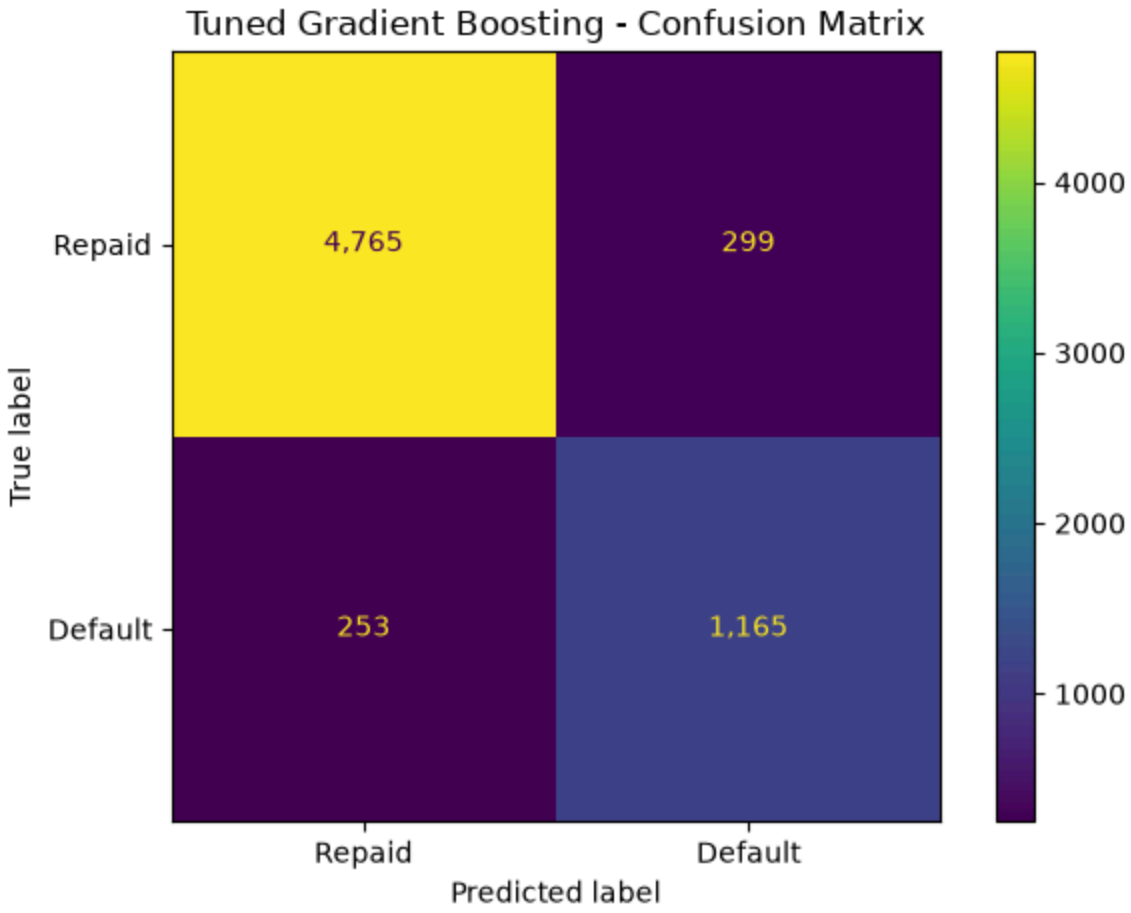
```
Fitting 4 folds for each of 18 candidates, totalling 72 fits

Best Parameters
{'subsample': 0.85, 'n_estimators': 300, 'min_samples_leaf': 3, 'max_feature
s': None, 'max_depth': 4, 'learning_rate': 0.1}

Best Cross-Validated AUC: 0.9480

Top Hyperparameter Combinations
 rank_test_roc_auc  mean_test_roc_auc  std_test_roc_auc  mean_test_average_p
recision  mean_train_roc_auc
params
                1             0.9480            0.0013
0.9577             0.9805   {'subsample': 0.85, 'n_estimators': 300, 'min_s
amples_leaf': 3, 'max_features': None, 'max_depth': 4, 'learning_rate': 0.1}
                2             0.9457            0.0021
0.9560             0.9718   {'subsample': 0.7, 'n_estimators': 400, 'min_sa
mples_leaf': 3, 'max_features': None, 'max_depth': 4, 'learning_rate': 0.05}
                3             0.9409            0.0011
0.9523             0.9602  {'subsample': 0.7, 'n_estimators': 200, 'min_sam
ples_leaf': 20, 'max_features': 0.7, 'max_depth': 4, 'learning_rate': 0.075}
                4             0.9392            0.0023
0.9508             0.9520 {'subsample': 1.0, 'n_estimators': 300, 'min_samp
les_leaf': 20, 'max_features': None, 'max_depth': 3, 'learning_rate': 0.075}
                5             0.9349            0.0026
0.9476             0.9450  {'subsample': 0.7, 'n_estimators': 150, 'min_sam
ples_leaf': 20, 'max_features': None, 'max_depth': 4, 'learning_rate': 0.05}
                6             0.9329            0.0029
0.9462             0.9420    {'subsample': 1.0, 'n_estimators': 250, 'min_s
amples_leaf': 5, 'max_features': 0.7, 'max_depth': 4, 'learning_rate': 0.03}
                7             0.9327            0.0026
0.9461             0.9425  {'subsample': 0.85, 'n_estimators': 100, 'min_sa
mples_leaf': 3, 'max_features': 0.7, 'max_depth': 4, 'learning_rate': 0.075}
                8             0.9302            0.0030
0.9443             0.9373   {'subsample': 1.0, 'n_estimators': 300, 'min_sa
mples_leaf': 20, 'max_features': 0.7, 'max_depth': 4, 'learning_rate': 0.02}
                9             0.9299            0.0018
0.9437             0.9370   {'subsample': 0.7, 'n_estimators': 150, 'min_sa
mples_leaf': 3, 'max_features': 0.7, 'max_depth': 3, 'learning_rate': 0.075}
               10             0.9263            0.0025
0.9409             0.9317  {'subsample': 0.85, 'n_estimators': 400, 'min_sa
mples_leaf': 5, 'max_features': None, 'max_depth': 3, 'learning_rate': 0.02}
```

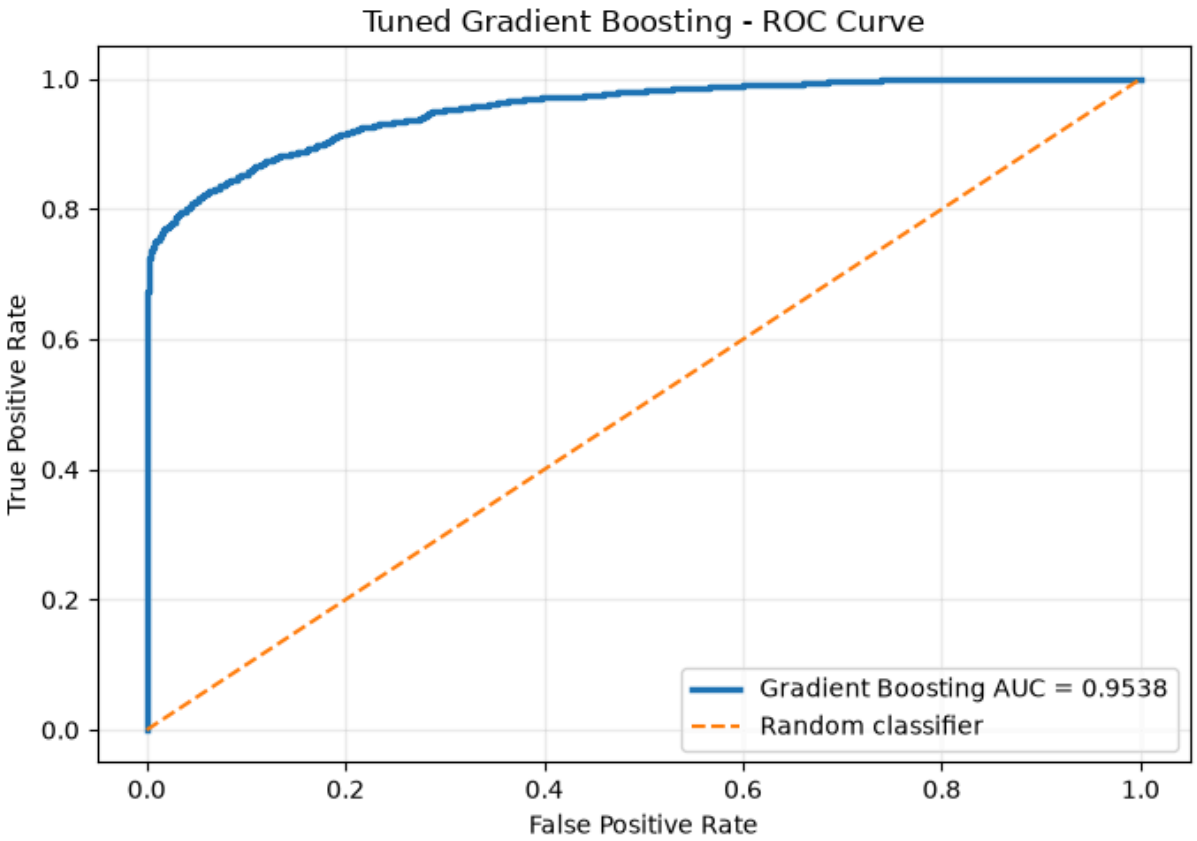
In [28]:
```
# 7. Evaluate the Tuned Gradient Boosting Model

best_gb = gradient_search.best_estimator_

tuned_results, tuned_probabilities, tuned_predictions = evaluate_classifier(
    model_name="Tuned Gradient Boosting",
    model=best_gb,
    X_eval=X_test_enc,
    y_eval=y_test,
```

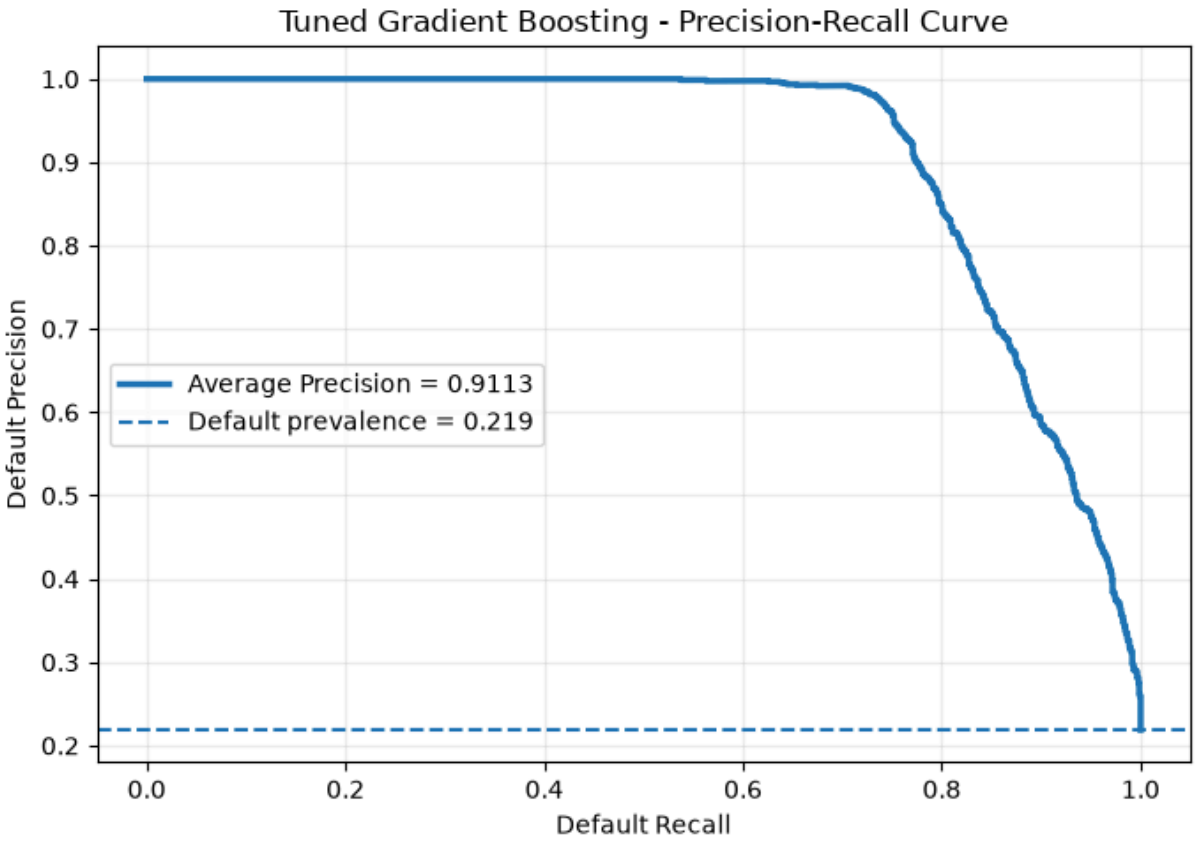
```
    threshold=0.50
)

print("Tuned Gradient Boosting")
print("-" * 40)

for metric, value in tuned_results.items():
    if metric not in ["Model", "Threshold"]:
        print(f"{metric:<22}: {value:.4f}")

print("\nClassification Report")
print(
    classification_report(
        y_test,
        tuned_predictions,
        target_names=["Repaid", "Default"]
    )
)

ConfusionMatrixDisplay.from_predictions(
    y_test,
    tuned_predictions,
    display_labels=["Repaid", "Default"],
    values_format=",d"
)
plt.title("Tuned Gradient Boosting - Confusion Matrix")
plt.tight_layout()
plt.show()

false_positive_rate, true_positive_rate, _ = roc_curve(
    y_test,
    tuned_probabilities
)

plt.figure(figsize=(7, 5))
plt.plot(
    false_positive_rate,
    true_positive_rate,
    linewidth=2.5,
    label=f"Gradient Boosting AUC = {tuned_results['ROC AUC']:.4f}"
)
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random classifier"
)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Tuned Gradient Boosting - ROC Curve")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()

test_precision_curve, test_recall_curve, _ = precision_recall_curve(
```

```
    y_test,
    tuned_probabilities
)

plt.figure(figsize=(7, 5))
plt.plot(
    test_recall_curve,
    test_precision_curve,
    linewidth=2.5,
    label=(
        "Average Precision = "
        f"{tuned_results['Average Precision']:.4f}"
    )
)
plt.axhline(
    y=y_test.mean(),
    linestyle="--",
    label=f"Default prevalence = {y_test.mean():.3f}"
)
plt.xlabel("Default Recall")
plt.ylabel("Default Precision")
plt.title("Tuned Gradient Boosting - Precision-Recall Curve")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Tuned Gradient Boosting
----------------------------------------
Accuracy              : 0.9148
Balanced Accuracy     : 0.8813
ROC AUC               : 0.9538
Average Precision     : 0.9113
Default Precision     : 0.7958
Default Recall        : 0.8216
Default F1            : 0.8085

Classification Report
              precision    recall  f1-score   support

      Repaid       0.95      0.94      0.95      5064
     Default       0.80      0.82      0.81      1418

    accuracy                           0.91      6482
   macro avg       0.87      0.88      0.88      6482
weighted avg       0.92      0.91      0.92      6482
```

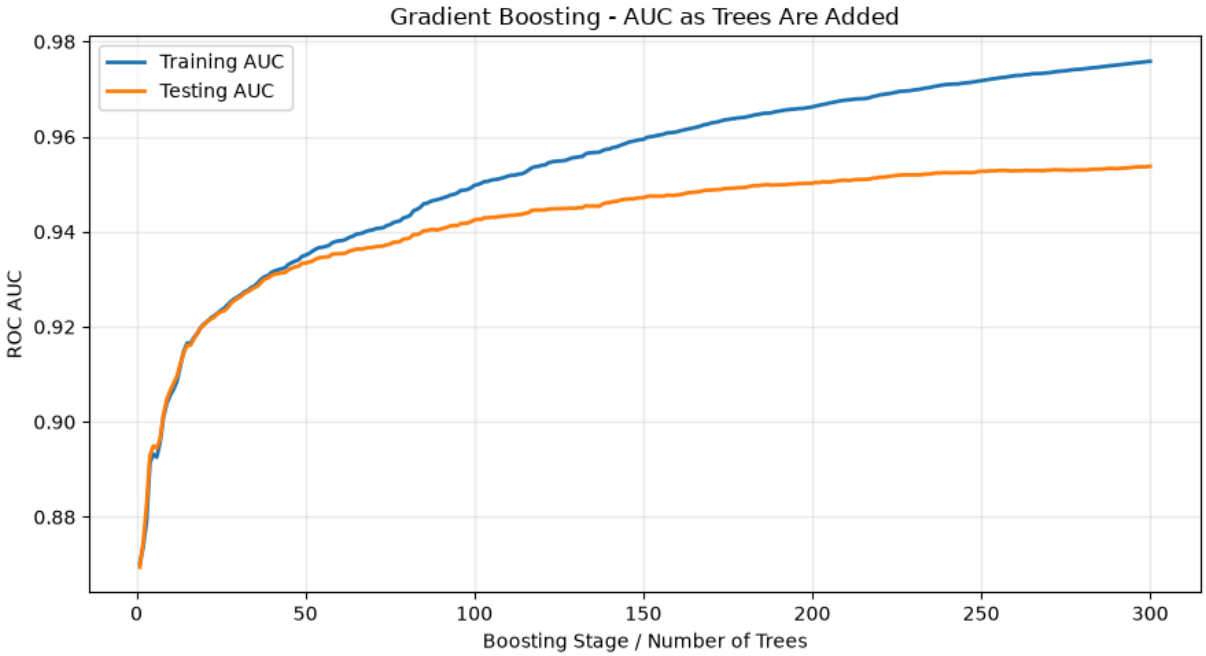

Tuned Gradient Boosting - Confusion Matrix
Repaid
Default
True label
4,765
299
253
1,165
Repaid
Default
Predicted label
4000
3000
2000
1000


Tuned Gradient Boosting - ROC Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
Gradient Boosting AUC = 0.9538
Random classifier



In [29]:
```python
# 8. Examine Performance After Each Boosting Stage

train_auc_by_stage = []
test_auc_by_stage = []

for train_stage_probabilities, test_stage_probabilities in zip(
    best_gb.staged_predict_proba(X_train_enc),
    best_gb.staged_predict_proba(X_test_enc)
):
    train_auc_by_stage.append(
        roc_auc_score(y_train, train_stage_probabilities[:, 1])
    )

    test_auc_by_stage.append(
        roc_auc_score(y_test, test_stage_probabilities[:, 1])
    )

stages = np.arange(1, len(train_auc_by_stage) + 1)

plt.figure(figsize=(9, 5))
plt.plot(
    stages,
    train_auc_by_stage,
    linewidth=2,
    label="Training AUC"
)
plt.plot(
    stages,
    test_auc_by_stage,
```

```
    linewidth=2,
    label="Testing AUC"
)
plt.xlabel("Boosting Stage / Number of Trees")
plt.ylabel("ROC AUC")
plt.title("Gradient Boosting - AUC as Trees Are Added")
plt.legend()
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()

highest_diagnostic_stage = int(np.argmax(test_auc_by_stage) + 1)
highest_diagnostic_auc = max(test_auc_by_stage)

print(f"Model estimators: {best_gb.n_estimators_}")
print(
    "Highest test AUC on the diagnostic curve:",
    f"{highest_diagnostic_auc:.4f} at stage {highest_diagnostic_stage}"
)
print(
    "The test curve is displayed only as an overfitting diagnostic. "
    "The model was selected using cross-validation, not this test curve."
)
```




```
Model estimators: 300
Highest test AUC on the diagnostic curve: 0.9538 at stage 300
The test curve is displayed only as an overfitting diagnostic. The model was
selected using cross-validation, not this test curve.
```

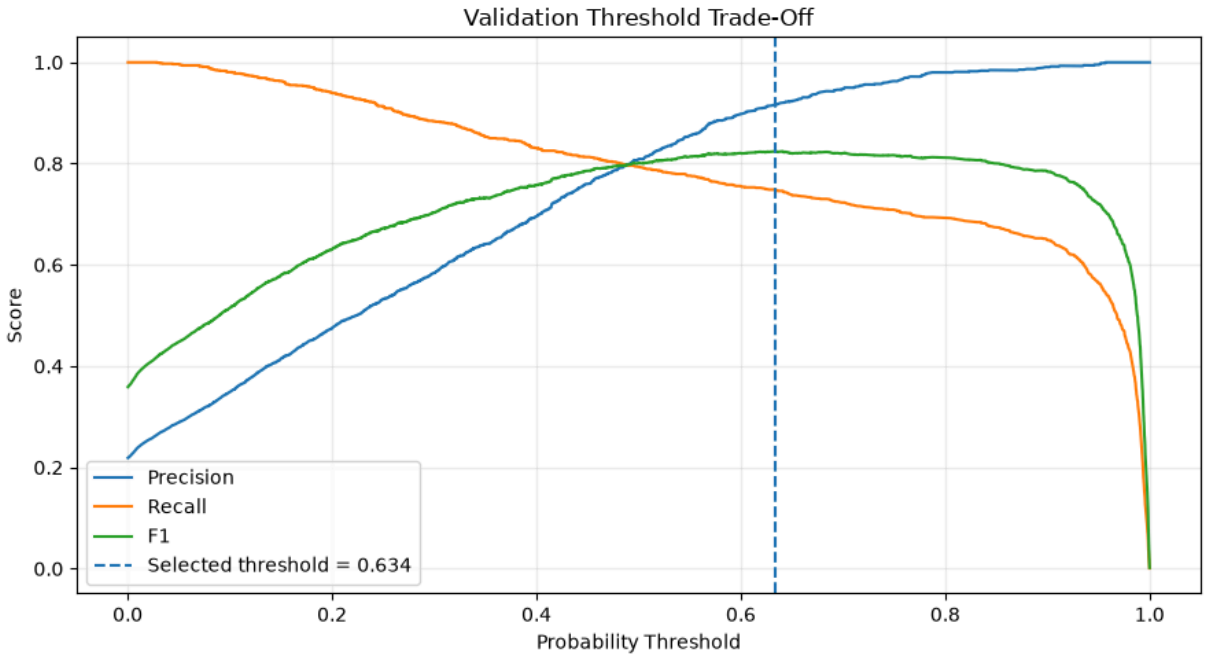
In [30]:
```
# 9. Select a Classification Threshold Using Training Validation Data

y_train_array = np.asarray(y_train)
all_training_indices = np.arange(len(y_train_array))

threshold_fit_indices, threshold_validation_indices = train_test_split(
    all_training_indices,
```

```
    test_size=0.20,
    random_state=42,
    stratify=y_train_array
)

threshold_model = clone(best_gb)

threshold_fit_weights = compute_sample_weight(
    class_weight="balanced",
    y=y_train_array[threshold_fit_indices]
)

threshold_model.fit(
    X_train_enc[threshold_fit_indices],
    y_train_array[threshold_fit_indices],
    sample_weight=threshold_fit_weights
)

validation_probabilities = threshold_model.predict_proba(
    X_train_enc[threshold_validation_indices]
)[:, 1]

validation_precision, validation_recall, validation_thresholds = (
    precision_recall_curve(
        y_train_array[threshold_validation_indices],
        validation_probabilities
    )
)

validation_f1 = (
    2
    * validation_precision[:-1]
    * validation_recall[:-1]
    / (
        validation_precision[:-1]
        + validation_recall[:-1]
        + 1e-12
    )
)

best_threshold_index = int(np.nanargmax(validation_f1))
selected_threshold = float(
    validation_thresholds[best_threshold_index]
)

print(f"Validation-selected threshold: {selected_threshold:.4f}")
print(
    "Validation F1 at selected threshold:",
    f"{validation_f1[best_threshold_index]:.4f}"
)

threshold_results, _, threshold_predictions = evaluate_classifier(
    model_name="Tuned Gradient Boosting - Validation Threshold",
    model=best_gb,
    X_eval=X_test_enc,
    y_eval=y_test,
```

```
    threshold=selected_threshold
)

threshold_comparison = pd.DataFrame(
    [tuned_results, threshold_results]
)

print("\nThreshold Comparison")
print(
    threshold_comparison[
        [
            "Model",
            "Threshold",
            "Accuracy",
            "Balanced Accuracy",
            "Default Precision",
            "Default Recall",
            "Default F1"
        ]
    ].round(4).to_string(index=False)
)

print("\nClassification Report at Validation-Selected Threshold")
print(
    classification_report(
        y_test,
        threshold_predictions,
        target_names=["Repaid", "Default"]
    )
)

plt.figure(figsize=(9, 5))
plt.plot(
    validation_thresholds,
    validation_precision[:-1],
    label="Precision"
)
plt.plot(
    validation_thresholds,
    validation_recall[:-1],
    label="Recall"
)
plt.plot(
    validation_thresholds,
    validation_f1,
    label="F1"
)
plt.axvline(
    selected_threshold,
    linestyle="--",
    label=f"Selected threshold = {selected_threshold:.3f}"
)
plt.xlabel("Probability Threshold")
plt.ylabel("Score")
plt.title("Validation Threshold Trade-Off")
plt.legend()
```

```
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

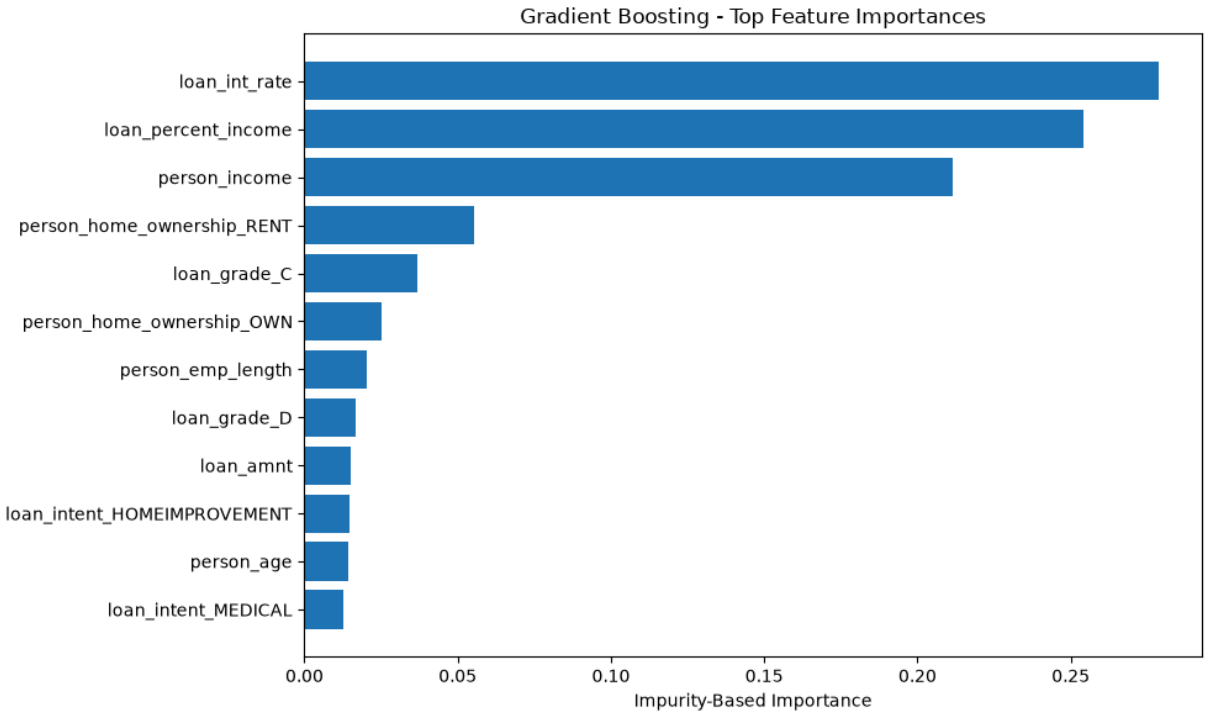
```
Validation-selected threshold: 0.6336
Validation F1 at selected threshold: 0.8243

Threshold Comparison
                                          Model  Threshold  Accuracy  Balance
d Accuracy  Default Precision  Default Recall  Default F1
                      Tuned Gradient Boosting     0.5000    0.9148
0.8813             0.7958          0.8216      0.8085
Tuned Gradient Boosting - Validation Threshold     0.6336    0.9320
0.8747             0.9021          0.7729      0.8325

Classification Report at Validation-Selected Threshold
              precision    recall  f1-score   support

      Repaid       0.94      0.98      0.96      5064
     Default       0.90      0.77      0.83      1418

    accuracy                           0.93      6482
   macro avg       0.92      0.87      0.89      6482
weighted avg       0.93      0.93      0.93      6482
```



In [31]:
```
# 10. Impurity-Based Feature Importance

impurity_importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Impurity Importance": best_gb.feature_importances_
}).sort_values(
    "Impurity Importance",
    ascending=False
)
```

```
print("Top Impurity-Based Features")
print(
    impurity_importance_df.head(15)
    .round(5)
    .to_string(index=False)
)

top_impurity = (
    impurity_importance_df
    .head(12)
    .sort_values("Impurity Importance")
)

plt.figure(figsize=(10, 6))
plt.barh(
    top_impurity["Feature"],
    top_impurity["Impurity Importance"]
)
plt.xlabel("Impurity-Based Importance")
plt.title("Gradient Boosting - Top Feature Importances")
plt.tight_layout()
plt.show()
```

```
Top Impurity-Based Features
                       Feature  Impurity Importance
                 loan_int_rate              0.27884
           loan_percent_income              0.25434
                 person_income              0.21134
   person_home_ownership_RENT               0.05526
                loan_grade_C              0.03700
    person_home_ownership_OWN              0.02505
            person_emp_length              0.02044
                 loan_grade_D              0.01675
                    loan_amnt              0.01493
  loan_intent_HOMEIMPROVEMENT              0.01474
                   person_age              0.01447
          loan_intent_MEDICAL              0.01261
          loan_intent_VENTURE              0.01255
 loan_intent_DEBTCONSOLIDATION             0.01110
person_home_ownership_MORTGAGE             0.00546
```

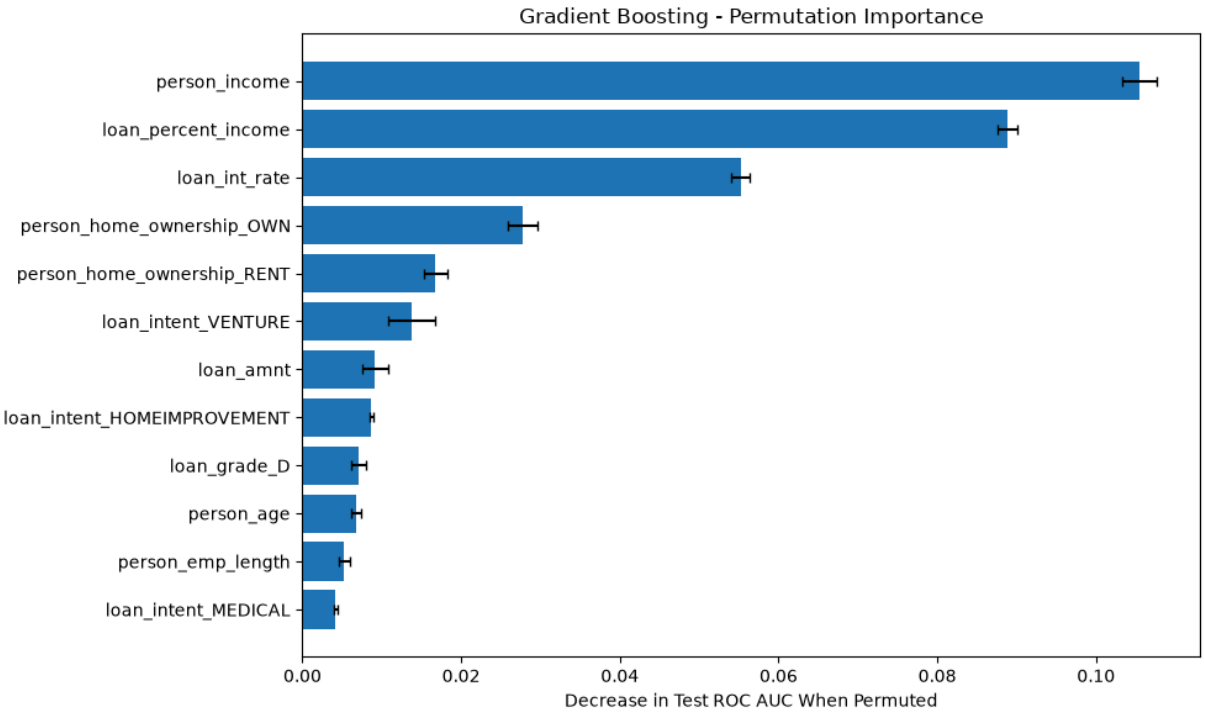


In [32]:

```
# 11. Permutation Importance on the Held-Out Test Set

permutation_results = permutation_importance(
    estimator=best_gb,
    X=X_test_enc,
    y=y_test,
    scoring="roc_auc",
    n_repeats=5,
    random_state=42,
    n_jobs=-1
)

permutation_importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Permutation Importance": permutation_results.importances_mean,
    "Importance Std": permutation_results.importances_std
}).sort_values(
    "Permutation Importance",
    ascending=False
)

print("Top Permutation-Importance Features")
print(
    permutation_importance_df.head(15)
    .round(5)
    .to_string(index=False)
)

top_permutation = (
    permutation_importance_df
    .head(12)
    .sort_values("Permutation Importance")
)
```

```
plt.figure(figsize=(10, 6))
plt.barh(
    top_permutation["Feature"],
    top_permutation["Permutation Importance"],
    xerr=top_permutation["Importance Std"],
    capsize=3
)
plt.xlabel("Decrease in Test ROC AUC When Permuted")
plt.title("Gradient Boosting - Permutation Importance")
plt.tight_layout()
plt.show()

importance_comparison = (
    impurity_importance_df
    .merge(
        permutation_importance_df,
        on="Feature",
        how="inner"
    )
    .sort_values(
        "Permutation Importance",
        ascending=False
    )
)

print("\nImportance Comparison")
print(
    importance_comparison.head(15)
    .round(5)
    .to_string(index=False)
)
```

```
Top Permutation-Importance Features
                      Feature  Permutation Importance  Importance Std
                person_income                 0.10548         0.00222
           loan_percent_income                 0.08885         0.00123
                 loan_int_rate                 0.05520         0.00121
    person_home_ownership_OWN                  0.02775         0.00186
   person_home_ownership_RENT                  0.01675         0.00146
          loan_intent_VENTURE                  0.01376         0.00296
                    loan_amnt                  0.00916         0.00163
  loan_intent_HOMEIMPROVEMENT                  0.00870         0.00022
                loan_grade_D                   0.00712         0.00092
                  person_age                   0.00675         0.00062
           person_emp_length                   0.00528         0.00076
          loan_intent_MEDICAL                  0.00419         0.00025
loan_intent_DEBTCONSOLIDATION                  0.00357         0.00059
                loan_grade_C                   0.00352         0.00023
person_home_ownership_MORTGAGE                 0.00154         0.00033
```

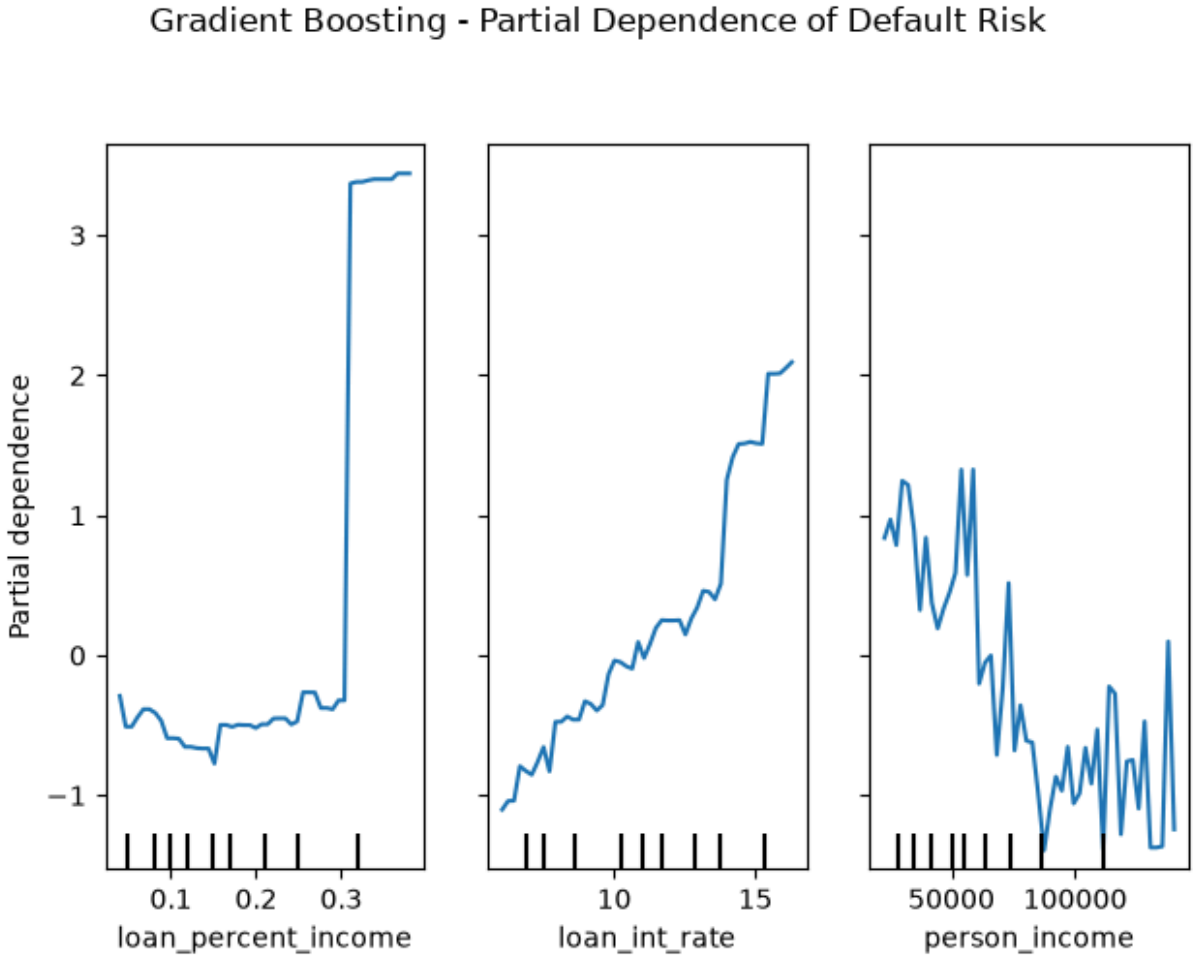


Importance Comparison

Feature	Impurity Importance	Permutation Importance	Importance Std
person_income	0.21134	0.10548	0.00222
loan_percent_income	0.25434	0.08885	0.00123
loan_int_rate	0.27884	0.05520	0.00121
person_home_ownership_OWN	0.02505	0.02775	0.00186
person_home_ownership_RENT	0.05526	0.01675	0.00146
loan_intent_VENTURE	0.01255	0.01376	0.00296
loan_amnt	0.01493	0.00916	0.00163
loan_intent_HOMEIMPROVEMENT	0.01474	0.00870	0.00022
loan_grade_D	0.01675	0.00712	0.00092
person_age	0.01447	0.00675	0.00062
person_emp_length	0.02044	0.00528	0.00076
loan_intent_MEDICAL	0.01261	0.00419	0.00025
loan_intent_DEBTCONSOLIDATION	0.01110	0.00357	0.00059
loan_grade_C	0.03700	0.00352	0.00023
person_home_ownership_MORTGAGE	0.00546	0.00154	0.00033

In [33]:

```
# 12. Partial Dependence for Important Continuous Variables

partial_dependence_features = [
    "loan_percent_income",
    "loan_int_rate",
    "person_income"
]

partial_dependence_indices = [
    feature_names.index(feature)
    for feature in partial_dependence_features
]

# A sample reduces plotting time without changing the fitted model
random_generator = np.random.default_rng(42)

partial_dependence_sample_indices = random_generator.choice(
    len(X_test_enc),
    size=min(3000, len(X_test_enc)),
    replace=False
)

X_partial_dependence = X_test_enc[
    partial_dependence_sample_indices
]

PartialDependenceDisplay.from_estimator(
    estimator=best_gb,
    X=X_partial_dependence,
    features=partial_dependence_indices,
    feature_names=feature_names,
    kind="average",
    grid_resolution=50
)

plt.suptitle(
    "Gradient Boosting - Partial Dependence of Default Risk",
    y=1.05
)
plt.tight_layout()
plt.show()
```



In [ ]:

```python
#13.Rebuild the Week 6 Random Forest Benchmark

random_forest = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_leaf=5,
    class_weight="balanced",
    n_jobs=-1,
    random_state=42
)

random_forest.fit(
    X_train_enc,
    y_train
)

rf_results, rf_probabilities, rf_predictions = evaluate_classifier(
    model_name="Random Forest - Week 6 Specification",
    model=random_forest,
    X_eval=X_test_enc,
    y_eval=y_test,
    threshold=0.50
)

print("Random Forest Benchmark")
print("-" * 40)
```

```
for metric, value in rf_results.items():
    if metric not in ["Model", "Threshold"]:
        print(f"{metric:<22}: {value:.4f}")

print("\nClassification Report")
print(
    classification_report(
        y_test,
        rf_predictions,
        target_names=["Repaid", "Default"]
    )
)
```

```
Random Forest Benchmark
----------------------------------------
Accuracy              : 0.9185
Balanced Accuracy     : 0.8659
ROC AUC               : 0.9345
Average Precision     : 0.8875
Default Precision     : 0.8423
Default Recall        : 0.7722
Default F1            : 0.8057

Classification Report
              precision    recall  f1-score   support

      Repaid       0.94      0.96      0.95      5064
     Default       0.84      0.77      0.81      1418

    accuracy                           0.92      6482
   macro avg       0.89      0.87      0.88      6482
weighted avg       0.92      0.92      0.92      6482
```

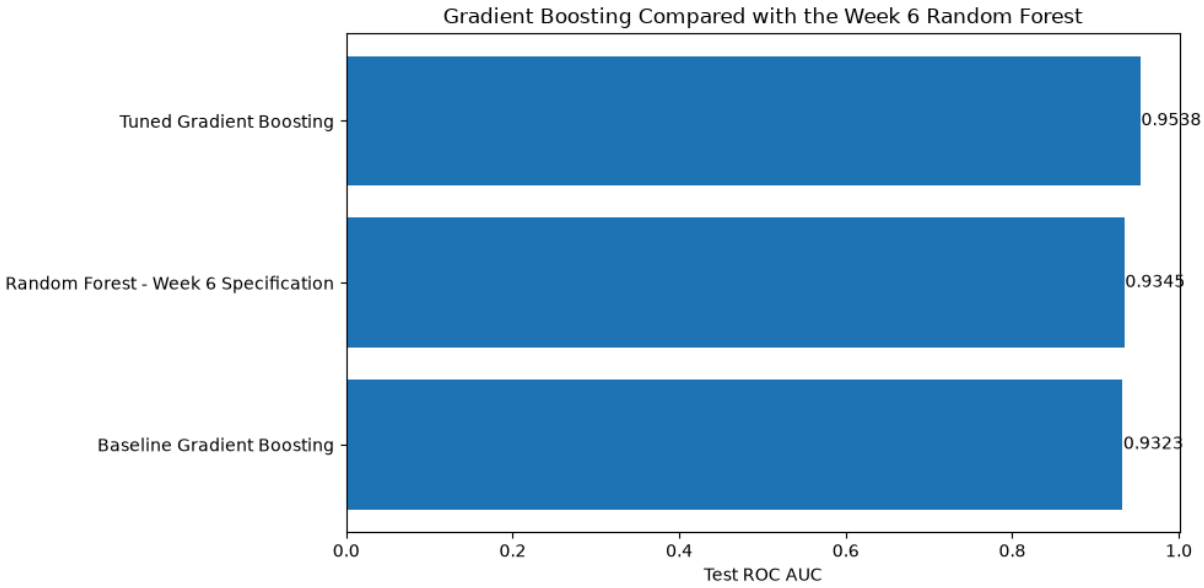
In [35]:

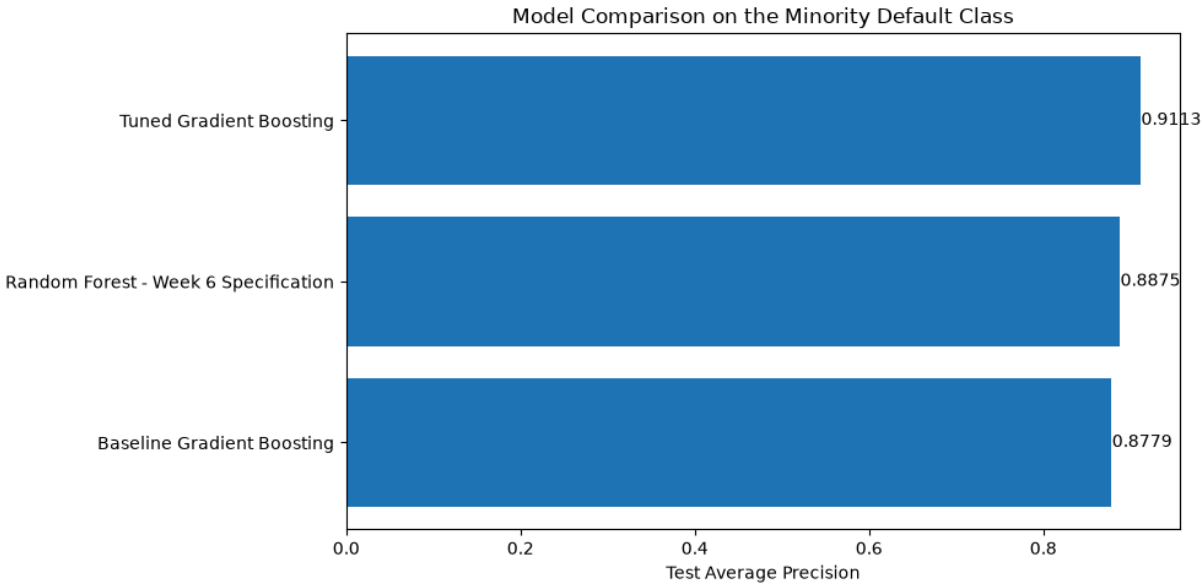
```
# 14. Final Week 9 Model Comparison

model_comparison = pd.DataFrame([
    baseline_results,
    tuned_results,
    rf_results
]).sort_values(
    "ROC AUC",
    ascending=False
).reset_index(drop=True)

comparison_columns = [
    "Model",
    "ROC AUC",
    "Average Precision",
    "Accuracy",
    "Balanced Accuracy",
    "Default Precision",
    "Default Recall",
    "Default F1"
]
```

```
print("Week 9 Model Leaderboard")
print(
    model_comparison[comparison_columns]
    .round(4)
    .to_string(index=False)
)

auc_order = model_comparison.sort_values("ROC AUC")

plt.figure(figsize=(10, 5))
plt.barh(
    auc_order["Model"],
    auc_order["ROC AUC"]
)

for index, value in enumerate(auc_order["ROC AUC"]):
    plt.text(
        value + 0.001,
        index,
        f"{value:.4f}",
        va="center"
    )

plt.xlabel("Test ROC AUC")
plt.title("Gradient Boosting Compared with the Week 6 Random Forest")
plt.tight_layout()
plt.show()

ap_order = model_comparison.sort_values("Average Precision")

plt.figure(figsize=(10, 5))
plt.barh(
    ap_order["Model"],
    ap_order["Average Precision"]
)

for index, value in enumerate(ap_order["Average Precision"]):
    plt.text(
        value + 0.001,
        index,
        f"{value:.4f}",
        va="center"
    )

plt.xlabel("Test Average Precision")
plt.title("Model Comparison on the Minority Default Class")
plt.tight_layout()
plt.show()

best_auc_model = model_comparison.iloc[0]

gb_rf_auc_difference = (
    tuned_results["ROC AUC"]
    - rf_results["ROC AUC"]
)
```

```
print(
    f"\nHighest test AUC: {best_auc_model['Model']} "
    f"({best_auc_model['ROC AUC']:.4f})"
)

print(
    "Tuned Gradient Boosting minus Random Forest AUC:",
    f"{gb_rf_auc_difference:+.4f}"
)

if gb_rf_auc_difference > 0.002:
    print(
        "[Inference] Gradient boosting provides a meaningful improvement "
        "in borrower-risk ranking on this test set."
    )
elif gb_rf_auc_difference < -0.002:
    print(
        "[Inference] The random forest remains the stronger risk-ranking "
        "model on this test set."
    )
else:
    print(
        "[Inference] Gradient boosting and random forest perform nearly "
        "identically, so runtime, stability, and interpretability become "
        "more important selection criteria."
    )

print(
    "\n[Inference] The threshold comparison shows how the same probability "
    "model can produce different lending outcomes depending on the desired "
    "balance between catching defaults and incorrectly flagging borrowers "
    "who would repay."
)
```

```
Week 9 Model Leaderboard
                              Model  ROC AUC  Average Precision  Accuracy
Balanced Accuracy  Default Precision  Default Recall  Default F1
             Tuned Gradient Boosting   0.9538             0.9113    0.9148
0.8813             0.7958          0.8216      0.8085
Random Forest - Week 6 Specification   0.9345             0.8875    0.9185
0.8659             0.8423          0.7722      0.8057
          Baseline Gradient Boosting   0.9323             0.8779    0.8854
0.8530             0.7135          0.7955      0.7523
```






```
Highest test AUC: Tuned Gradient Boosting (0.9538)
Tuned Gradient Boosting minus Random Forest AUC: +0.0193
[Inference] Gradient boosting provides a meaningful improvement in borrower-
risk ranking on this test set.

[Inference] The threshold comparison shows how the same probability model ca
n produce different lending outcomes depending on the desired balance betwee
n catching defaults and incorrectly flagging borrowers who would repay.
```

Week10_Gueorgui_Poklitar

August 1, 2026

******Week 10 - K-Means Clustering******

[1]:
```
%pip install -q pandas numpy matplotlib scikit-learn kagglehub
```

```
[notice] A new release of pip is
available: 26.0.1 -> 26.2
[notice] To update, run:
pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

[2]:
```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import sklearn
import kagglehub

from sklearn.cluster import KMeans
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import (
    adjusted_rand_score,
    average_precision_score,
    calinski_harabasz_score,
    davies_bouldin_score,
    roc_auc_score,
    silhouette_samples,
    silhouette_score
)
from sklearn.metrics.pairwise import pairwise_distances
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import (
    MinMaxScaler,
```

```
    OneHotEncoder,
    RobustScaler,
    StandardScaler,
    normalize
)
from sklearn.utils.class_weight import compute_sample_weight

pd.set_option("display.max_columns", 100)
pd.set_option("display.width", 180)

print(f"scikit-learn version: {sklearn.__version__}")
print("All imports successful.")
```

```
scikit-learn version: 1.9.0
All imports successful.
```

Data Loading and Cleaning

[3]:
```
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")

df = df.drop_duplicates().copy()

df["person_emp_length"] = df["person_emp_length"].fillna(
    df["person_emp_length"].median()
)

df["loan_int_rate"] = df["loan_int_rate"].fillna(
    df.groupby("loan_grade")["loan_int_rate"].transform("median")
)

df = df[df["person_emp_length"] <= df["person_age"]]
df = df[df["person_age"] <= 100]
df = df[df["person_emp_length"] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate: {df['loan_status'].mean() * 100:.2f}%")
print("\nMissing values:")
print(df.isna().sum())
```

```
Raw shape: (32581, 12)
Clean shape: (32409, 12)
Default rate: 21.87%

Missing values:
person_age                    0
person_income                 0
```

```
person_home_ownership         0
person_emp_length             0
loan_intent                   0
loan_grade                    0
loan_amnt                     0
loan_int_rate                 0
loan_status                   0
loan_percent_income           0
cb_person_default_on_file     0
cb_person_cred_hist_length    0
dtype: int64
```

Unsupervised Feature Preparation

[4]:
```
numeric_features = [
    "person_income",
    "person_age",
    "person_emp_length",
    "loan_amnt",
    "loan_percent_income",
    "loan_int_rate"
]

categorical_features = [
    "loan_intent",
    "loan_grade",
    "person_home_ownership",
    "cb_person_default_on_file"
]

X = df[numeric_features + categorical_features].copy()
y_holdout = df["loan_status"].copy()

complete_mask = X.notna().all(axis=1)
X = X.loc[complete_mask]
y_holdout = y_holdout.loc[complete_mask]
df_model = df.loc[complete_mask].copy()

scaled_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ),
            categorical_features
```

```
        )
    ]
)

unscaled_preprocessor = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numeric_features),
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ),
            categorical_features
        )
    ]
)

numeric_only_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features)
    ]
)

X_scaled_full = scaled_preprocessor.fit_transform(X)
X_unscaled_full = unscaled_preprocessor.fit_transform(X)
X_numeric_only = numeric_only_preprocessor.fit_transform(X)

categorical_names = (
    scaled_preprocessor.named_transformers_["cat"]
    .get_feature_names_out(categorical_features)
    .tolist()
)
full_feature_names = numeric_features + categorical_names

X_scaled_full = np.asarray(X_scaled_full, dtype=np.float64)
X_unscaled_full = np.asarray(X_unscaled_full, dtype=np.float64)
X_numeric_only = np.asarray(X_numeric_only, dtype=np.float64)

print(f"Borrowers available for clustering: {X_scaled_full.shape[0]:,}")
print(f"Full representation dimensions: {X_scaled_full.shape[1]}")
print(f"Numeric-only representation dimensions: {X_numeric_only.shape[1]}")
print(f"Withheld default rate: {y_holdout.mean() * 100:.2f}%")
print(
    "\nThe label above is reported for reference only and is not used "
    "in any fit call in this notebook."
)
```

```
Borrowers available for clustering: 32,409
Full representation dimensions: 25
Numeric-only representation dimensions: 6
Withheld default rate: 21.87%

The label above is reported for reference only and is not used in any fit call
in this notebook.
```

Feature Scaling: Why K-Means Cannot Be Run on Raw Columns

[5]:
```python
scale_summary = pd.DataFrame({
    "Feature": numeric_features,
    "Standard Deviation": [X[column].std() for column in numeric_features]
}).sort_values("Standard Deviation", ascending=False)

print("Raw Numeric Feature Scales")
print(scale_summary.round(4).to_string(index=False))

variance_ratio = (
    scale_summary["Standard Deviation"].max() ** 2
    / scale_summary["Standard Deviation"].min() ** 2
)

print(
    "\nRatio of largest to smallest *squared* standard deviation:",
    f"{variance_ratio:,.0f} to 1"
)

scaling_test_k = 4

scaling_comparison = []

for label, matrix in [
    ("Unscaled", X_unscaled_full),
    ("Standardised", X_scaled_full)
]:
    model = KMeans(
        n_clusters=scaling_test_k,
        n_init=10,
        random_state=42
    )
    labels = model.fit_predict(matrix)

    silhouette = silhouette_score(
        matrix,
        labels,
        sample_size=5000,
        random_state=42
```

```
    )

    income_by_cluster = pd.Series(
        df_model["person_income"].values
    ).groupby(labels).mean()

    income_spread = (
        income_by_cluster.max()
        - income_by_cluster.min()
    )

    percent_income_by_cluster = pd.Series(
        df_model["loan_percent_income"].values
    ).groupby(labels).mean()

    percent_income_spread = (
        percent_income_by_cluster.max()
        - percent_income_by_cluster.min()
    )

    scaling_comparison.append({
        "Preprocessing": label,
        "Silhouette": silhouette,
        "Calinski-Harabasz": calinski_harabasz_score(matrix, labels),
        "Income Spread Across Clusters": income_spread,
        "Debt Burden Spread Across Clusters": percent_income_spread,
        "Smallest Cluster Share": (
            pd.Series(labels).value_counts(normalize=True).min()
        )
    })

scaling_comparison_df = pd.DataFrame(scaling_comparison)

print(f"\nK-Means at k={scaling_test_k}: Unscaled Versus Standardised")
print(scaling_comparison_df.round(4).to_string(index=False))

print(
    "\n A high silhouette on unscaled data is not evidence of a "
    "good segmentation. It reflects that one dominant column produces "
    "cleanly separated bands. The debt-burden spread column shows whether "
    "the partition distinguishes borrowers on anything beyond income."
)
```

```
Raw Numeric Feature Scales
            Feature  Standard Deviation
      person_income          52517.8700
          loan_amnt           6320.8851
         person_age              6.2104
```

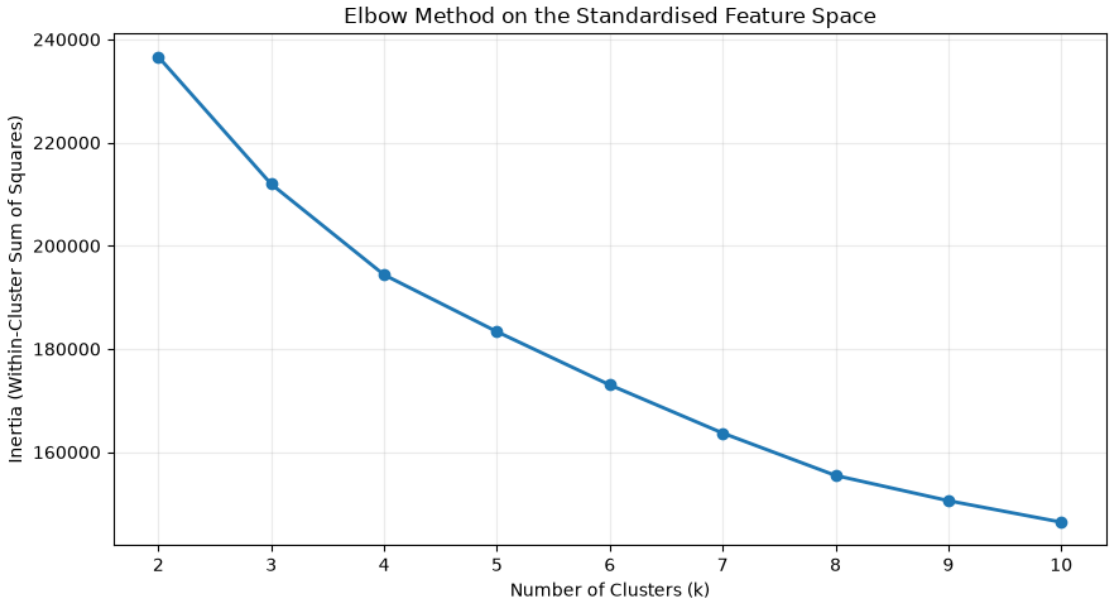
```
  person_emp_length              3.9838
      loan_int_rate              3.2135
loan_percent_income              0.1068

Ratio of largest to smallest *squared* standard deviation: 241,876,641,369 to 1


K-Means at k=4: Unscaled Versus Standardised
Preprocessing  Silhouette  Calinski-Harabasz  Income Spread Across Clusters
Debt Burden Spread Across Clusters  Smallest Cluster Share
     Unscaled      0.5535         37730.1454                    805719.4592
0.1747                  0.0015
 Standardised      0.1351          4370.0550                     60927.0771
0.1812                  0.1207

 A high silhouette on unscaled data is not evidence of a good segmentation. It
reflects that one dominant column produces cleanly separated bands. The debt-
burden spread column shows whether the partition distinguishes borrowers on
anything beyond income.
```

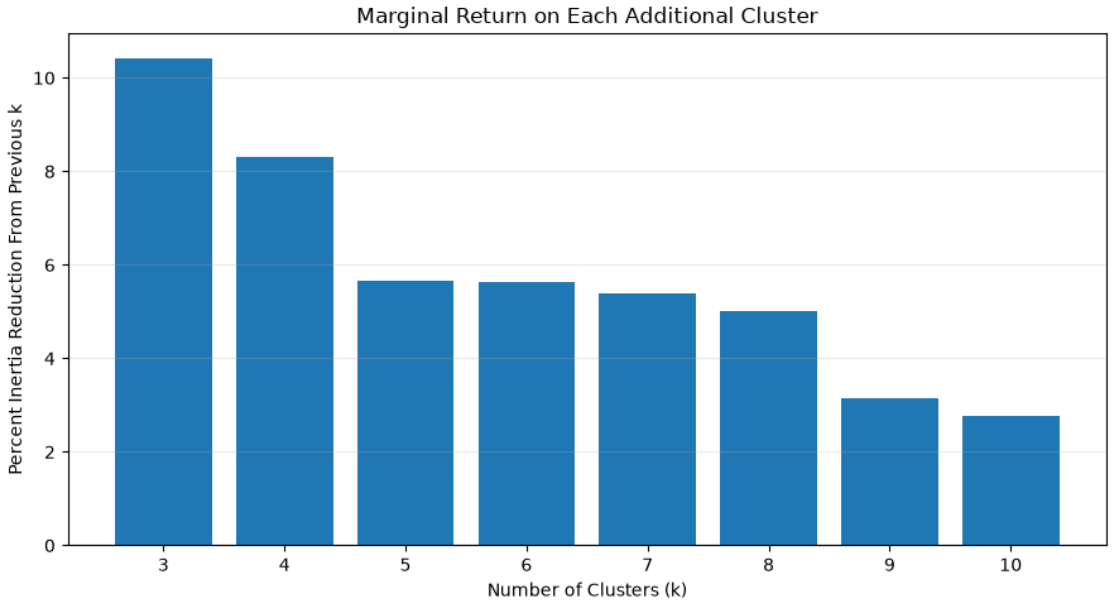
The Elbow Method

[6]:
```
candidate_k_values = list(range(2, 11))

elbow_results = []

for k in candidate_k_values:
    model = KMeans(
        n_clusters=k,
        n_init=10,
        random_state=42
    )
    model.fit(X_scaled_full)

    elbow_results.append({
        "Clusters": k,
        "Inertia": model.inertia_
    })

elbow_df = pd.DataFrame(elbow_results)

elbow_df["Inertia Reduction"] = (
    elbow_df["Inertia"].shift(1)
    - elbow_df["Inertia"]
)

elbow_df["Percent Reduction"] = (
    100
```

```
    * elbow_df["Inertia Reduction"]
    / elbow_df["Inertia"].shift(1)
)

print("Elbow Analysis")
print(elbow_df.round(3).to_string(index=False))

plt.figure(figsize=(9, 5))
plt.plot(
    elbow_df["Clusters"],
    elbow_df["Inertia"],
    marker="o",
    linewidth=2
)
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia (Within-Cluster Sum of Squares)")
plt.title("Elbow Method on the Standardised Feature Space")
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()

plt.figure(figsize=(9, 5))
plt.bar(
    elbow_df["Clusters"],
    elbow_df["Percent Reduction"]
)
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Percent Inertia Reduction From Previous k")
plt.title("Marginal Return on Each Additional Cluster")
plt.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Elbow Analysis
 Clusters    Inertia  Inertia Reduction  Percent Reduction
        2 236620.770                NaN                NaN
        3 211951.589          24669.181             10.426
        4 194366.277          17585.312              8.297
        5 183343.423          11022.854              5.671
        6 173018.552          10324.871              5.631
        7 163687.848           9330.704              5.393
        8 155471.634           8216.214              5.019
        9 150575.355           4896.279              3.149
       10 146418.241           4157.114              2.761
```





Silhouette Analysis

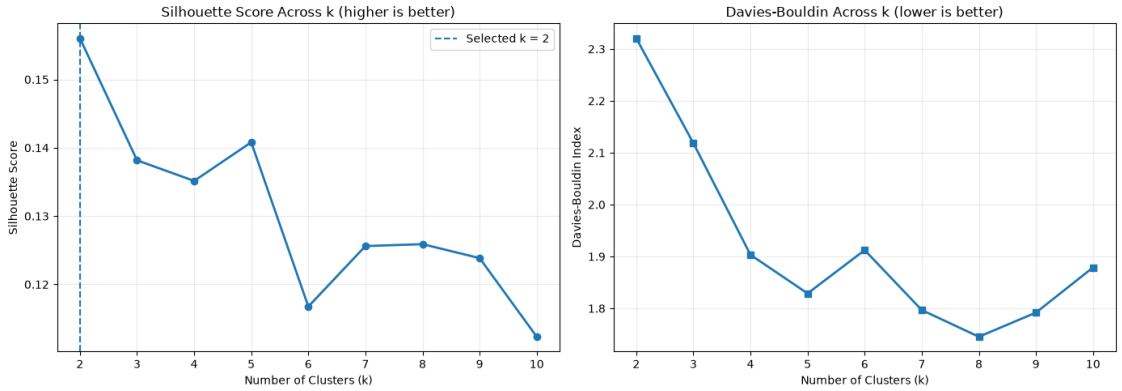
[7]:
```
silhouette_results = []

cluster_labels_by_k = {}

for k in candidate_k_values:
```

```
    model = KMeans(
        n_clusters=k,
        n_init=10,
        random_state=42
    )
    labels = model.fit_predict(X_scaled_full)
    cluster_labels_by_k[k] = labels

    silhouette_results.append({
        "Clusters": k,
        "Silhouette": silhouette_score(
            X_scaled_full,
            labels,
            sample_size=5000,
            random_state=42
        ),
        "Davies-Bouldin": davies_bouldin_score(X_scaled_full, labels),
        "Calinski-Harabasz": calinski_harabasz_score(X_scaled_full, labels),
        "Smallest Cluster Share": (
            pd.Series(labels).value_counts(normalize=True).min()
        )
    })

silhouette_df = pd.DataFrame(silhouette_results)

print("Internal Validation Criteria Across k")
print(silhouette_df.round(4).to_string(index=False))

best_silhouette_row = silhouette_df.loc[
    silhouette_df["Silhouette"].idxmax()
]
best_davies_row = silhouette_df.loc[
    silhouette_df["Davies-Bouldin"].idxmin()
]

selected_k = int(best_silhouette_row["Clusters"])

print(
    f"\nHighest silhouette at k={selected_k} "
    f"({best_silhouette_row['Silhouette']:.4f})"
)
print(
    "Lowest Davies-Bouldin at k="
    f"{int(best_davies_row['Clusters'])} "
    f"({best_davies_row['Davies-Bouldin']:.4f})"
)
```

```
if int(best_davies_row["Clusters"]) == selected_k:
    print(
        "Silhouette and Davies-Bouldin agree on the same k, "
        "which is reassuring given that they are computed differently."
    )
else:
    print(
        "The two criteria disagree, which is common on "
        "continuous data where cluster boundaries are gradual rather than "
        "sharp. Section 8 profiles the candidates so the choice can rest on "
        "business interpretability as well as internal geometry."
    )

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(
    silhouette_df["Clusters"],
    silhouette_df["Silhouette"],
    marker="o",
    linewidth=2
)
axes[0].axvline(
    selected_k,
    linestyle="--",
    label=f"Selected k = {selected_k}"
)
axes[0].set_xlabel("Number of Clusters (k)")
axes[0].set_ylabel("Silhouette Score")
axes[0].set_title("Silhouette Score Across k (higher is better)")
axes[0].legend()
axes[0].grid(alpha=0.25)

axes[1].plot(
    silhouette_df["Clusters"],
    silhouette_df["Davies-Bouldin"],
    marker="s",
    linewidth=2
)
axes[1].set_xlabel("Number of Clusters (k)")
axes[1].set_ylabel("Davies-Bouldin Index")
axes[1].set_title("Davies-Bouldin Across k (lower is better)")
axes[1].grid(alpha=0.25)

plt.tight_layout()
plt.show()
```

```
Internal Validation Criteria Across k
 Clusters  Silhouette  Davies-Bouldin  Calinski-Harabasz  Smallest Cluster Share
```

```
 2      0.1560          2.3205          4982.6078          0.3415
 3      0.1382          2.1184          4667.0599          0.2492
 4      0.1351          1.9033          4370.0550          0.1207
 5      0.1408          1.8288          3961.5275          0.0562
 6      0.1168          1.9123          3744.9664          0.0443
 7      0.1256          1.7968          3606.4344          0.0324
 8      0.1259          1.7454          3499.1069          0.0027
 9      0.1238          1.7919          3292.8839          0.0020
10      0.1123          1.8790          3112.2179          0.0020

Highest silhouette at k=2 (0.1560)
Lowest Davies-Bouldin at k=8 (1.7454)
The two criteria disagree, which is common on continuous data where cluster
boundaries are gradual rather than sharp. Section 8 profiles the candidates so
the choice can rest on business interpretability as well as internal geometry.
```

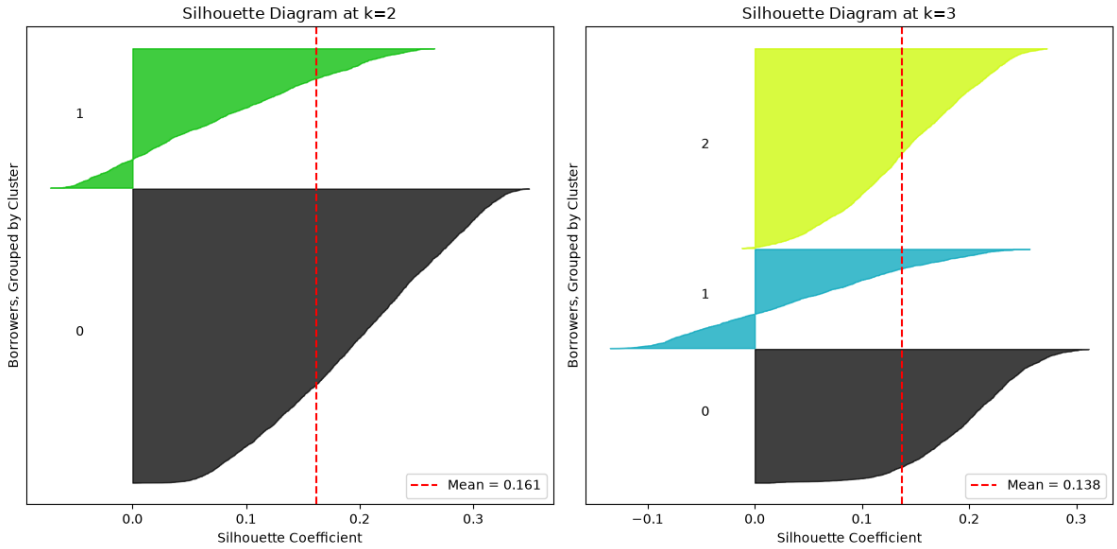


Per-cluster silhouette diagrams

[8]:
```python
diagram_k_values = [
    max(2, selected_k - 1),
    selected_k,
    selected_k + 1
]

diagram_k_values = sorted(set(diagram_k_values))

diagram_sample_size = 5000

diagram_rng = np.random.default_rng(42)
diagram_indices = diagram_rng.choice(
    X_scaled_full.shape[0],
    size=min(diagram_sample_size, X_scaled_full.shape[0]),
    replace=False
)
```

```
X_diagram = X_scaled_full[diagram_indices]

fig, axes = plt.subplots(
    1,
    len(diagram_k_values),
    figsize=(6 * len(diagram_k_values), 6)
)

if len(diagram_k_values) == 1:
    axes = [axes]

for axis, k in zip(axes, diagram_k_values):
    labels_sample = KMeans(
        n_clusters=k,
        n_init=10,
        random_state=42
    ).fit_predict(X_diagram)

    average_silhouette = silhouette_score(X_diagram, labels_sample)
    sample_silhouettes = silhouette_samples(X_diagram, labels_sample)

    lower_bound = 10

    for cluster in range(k):
        cluster_values = np.sort(
            sample_silhouettes[labels_sample == cluster]
        )
        cluster_size = cluster_values.shape[0]
        upper_bound = lower_bound + cluster_size

        colour = cm.nipy_spectral(float(cluster) / k)

        axis.fill_betweenx(
            np.arange(lower_bound, upper_bound),
            0,
            cluster_values,
            facecolor=colour,
            edgecolor=colour,
            alpha=0.75
        )

        axis.text(
            -0.05,
            lower_bound + 0.5 * cluster_size,
            str(cluster)
        )
```

```
        lower_bound = upper_bound + 10

    axis.axvline(
        average_silhouette,
        color="red",
        linestyle="--",
        label=f"Mean = {average_silhouette:.3f}"
    )
    axis.set_xlabel("Silhouette Coefficient")
    axis.set_ylabel("Borrowers, Grouped by Cluster")
    axis.set_title(f"Silhouette Diagram at k={k}")
    axis.set_yticks([])
    axis.legend(loc="lower right")

plt.tight_layout()
plt.show()

negative_share_results = []

for k in diagram_k_values:
    labels_sample = KMeans(
        n_clusters=k,
        n_init=10,
        random_state=42
    ).fit_predict(X_diagram)

    sample_silhouettes = silhouette_samples(X_diagram, labels_sample)

    negative_share_results.append({
        "Clusters": k,
        "Mean Silhouette": sample_silhouettes.mean(),
        "Share Below Zero": (sample_silhouettes < 0).mean(),
        "Weakest Cluster Mean": min(
            sample_silhouettes[labels_sample == cluster].mean()
            for cluster in range(k)
        )
    })

print("Silhouette Distribution Detail")
print(pd.DataFrame(negative_share_results).round(4).to_string(index=False))
```




```
Silhouette Distribution Detail
 Clusters  Mean Silhouette  Share Below Zero  Weakest Cluster Mean
        2          0.1615            0.0662                0.0827
        3          0.1378            0.0818                0.0483
```

Feature Space: Do the One-Hot Indicators Belong in the Distance?

[9]:
```
feature_space_results = []

feature_space_labels = {}

for label, matrix in [
    ("Numeric only", X_numeric_only),
    ("Numeric plus one-hot", X_scaled_full)
]:
    labels = KMeans(
        n_clusters=selected_k,
        n_init=10,
        random_state=42
    ).fit_predict(matrix)

    feature_space_labels[label] = labels

    debt_burden_by_cluster = pd.Series(
        df_model["loan_percent_income"].values
    ).groupby(labels).mean()

    default_rate_by_cluster = pd.Series(
        y_holdout.values
```

```
    ).groupby(labels).mean()

    feature_space_results.append({
        "Feature Space": label,
        "Dimensions": matrix.shape[1],
        "Silhouette": silhouette_score(
            matrix,
            labels,
            sample_size=5000,
            random_state=42
        ),
        "Davies-Bouldin": davies_bouldin_score(matrix, labels),
        "Debt Burden Spread": (
            debt_burden_by_cluster.max()
            - debt_burden_by_cluster.min()
        ),
        "Default Rate Spread": (
            default_rate_by_cluster.max()
            - default_rate_by_cluster.min()
        )
    })

feature_space_df = pd.DataFrame(feature_space_results)

print(f"Feature Space Comparison at k={selected_k}")
print(feature_space_df.round(4).to_string(index=False))

space_agreement = adjusted_rand_score(
    feature_space_labels["Numeric only"],
    feature_space_labels["Numeric plus one-hot"]
)

print(
    f"\nAdjusted Rand index between the two partitions: {space_agreement:.4f}"
)

if space_agreement > 0.7:
    print(
        " The two representations recover substantially the same "
        "segments, so the categorical indicators are not driving the "
        "partition and the choice between spaces is low-stakes."
    )
else:
    print(
        "The two representations produce materially different "
        "segments. The categorical indicators are exerting real influence on "
        "the distance metric, so the choice of feature space is itself a "
```

```
        "substantive modelling decision rather than a technicality."
    )

if (
    feature_space_df.loc[1, "Default Rate Spread"]
    >= feature_space_df.loc[0, "Default Rate Spread"]
):
    selected_matrix = X_scaled_full
    selected_space_label = "numeric plus one-hot"
else:
    selected_matrix = X_numeric_only
    selected_space_label = "numeric only"

print(
    "\nFeature space carried forward (chosen on risk separation, "
    f"not on internal geometry): {selected_space_label}"
)
```

```
Feature Space Comparison at k=2
       Feature Space  Dimensions  Silhouette  Davies-Bouldin  Debt Burden Spread
Default Rate Spread
        Numeric only           6      0.2215          1.8216              0.1591
0.2241
Numeric plus one-hot          25      0.1560          2.3205              0.1544
0.2410

Adjusted Rand index between the two partitions: 0.8602
 The two representations recover substantially the same segments, so the
categorical indicators are not driving the partition and the choice between
spaces is low-stakes.

Feature space carried forward (chosen on risk separation, not on internal
geometry): numeric plus one-hot
```

Distance Metrics and K-Means

Spherical K-Means projects every borrower onto the unit sphere by L2-normalising their feature vector, then runs ordinary K-Means. On unit-length vectors, squared Euclidean distance is a monotone function of cosine distance, so this genuinely clusters by cosine similarity while keeping the standard algorithm valid. Substantively it discards magnitude and clusters on *profile shape*: two borrowers with the same balance of income to debt burden to rate land together even at very different absolute scales.

K-Medoids replaces the mean with the member minimising total distance to its cluster, which is a valid update for any metric. This admits Manhattan distance, which Week 8 found more robust to the outliers in the income distribution. Because it requires a full pairwise distance matrix it is run on a stratified subsample.

[10]:
```python
def k_medoids(distance_matrix, n_clusters, random_state=42, max_iterations=60):
    rng = np.random.default_rng(random_state)
    n_points = distance_matrix.shape[0]

    medoid_indices = rng.choice(
        n_points,
        size=n_clusters,
        replace=False
    )

    for _ in range(max_iterations):
        assignments = np.argmin(
            distance_matrix[:, medoid_indices],
            axis=1
        )

        updated_medoids = medoid_indices.copy()

        for cluster in range(n_clusters):
            members = np.where(assignments == cluster)[0]

            if members.size == 0:
                continue

            within_cluster_totals = distance_matrix[
                np.ix_(members, members)
            ].sum(axis=1)

            updated_medoids[cluster] = members[
                np.argmin(within_cluster_totals)
            ]

        if np.array_equal(
            np.sort(updated_medoids),
            np.sort(medoid_indices)
        ):
            medoid_indices = updated_medoids
            break

        medoid_indices = updated_medoids

    assignments = np.argmin(
        distance_matrix[:, medoid_indices],
        axis=1
    )

    return medoid_indices, assignments
```

```
metric_sample_size = 4000

metric_rng = np.random.default_rng(42)
metric_indices = metric_rng.choice(
    selected_matrix.shape[0],
    size=min(metric_sample_size, selected_matrix.shape[0]),
    replace=False
)

X_metric = selected_matrix[metric_indices]
y_metric = y_holdout.values[metric_indices]

euclidean_labels = KMeans(
    n_clusters=selected_k,
    n_init=10,
    random_state=42
).fit_predict(X_metric)

X_normalised = normalize(X_metric)

spherical_labels = KMeans(
    n_clusters=selected_k,
    n_init=10,
    random_state=42
).fit_predict(X_normalised)

manhattan_distances = pairwise_distances(
    X_metric,
    metric="manhattan"
)

_, manhattan_labels = k_medoids(
    distance_matrix=manhattan_distances,
    n_clusters=selected_k,
    random_state=42
)

metric_comparison = []

for label, labels, matrix, metric_name in [
    ("K-Means (Euclidean)", euclidean_labels, X_metric, "euclidean"),
    ("Spherical K-Means (cosine)", spherical_labels, X_normalised, "cosine"),
    ("K-Medoids (Manhattan)", manhattan_labels, X_metric, "manhattan")
]:
    default_rate_by_cluster = pd.Series(y_metric).groupby(labels).mean()
```

```
    metric_comparison.append({
        "Method": label,
        "Silhouette (own metric)": silhouette_score(
            matrix,
            labels,
            metric=metric_name
        ),
        "Clusters Found": len(np.unique(labels)),
        "Smallest Cluster Share": (
            pd.Series(labels).value_counts(normalize=True).min()
        ),
        "Default Rate Spread": (
            default_rate_by_cluster.max()
            - default_rate_by_cluster.min()
        )
    })

metric_comparison_df = pd.DataFrame(metric_comparison)

print(f"Distance Geometry Comparison at k={selected_k}")
print(f"Subsample: {len(metric_indices):,} borrowers")
print(metric_comparison_df.round(4).to_string(index=False))

print("\nPartition Agreement (Adjusted Rand Index)")
print(
    "Euclidean vs spherical  :",
    f"{adjusted_rand_score(euclidean_labels, spherical_labels):.4f}"
)
print(
    "Euclidean vs Manhattan  :",
    f"{adjusted_rand_score(euclidean_labels, manhattan_labels):.4f}"
)
print(
    "Spherical vs Manhattan  :",
    f"{adjusted_rand_score(spherical_labels, manhattan_labels):.4f}"
)

print(
    "\n Silhouette values in the table are each computed under "
    "their own metric and are therefore not directly comparable across rows. "
    "The Rand indices and the default-rate spread are the comparable "
    "quantities: they show whether the three geometries are describing the "
    "same borrower population differently or finding genuinely different "
    "structure."
)
```

```
Distance Geometry Comparison at k=2
```

```
Subsample: 4,000 borrowers
                    Method  Silhouette (own metric)  Clusters Found  Smallest
Cluster Share  Default Rate Spread
       K-Means (Euclidean)                   0.1684               2
0.3135               0.2275
Spherical K-Means (cosine)                   0.2365               2
0.4868               0.2307
     K-Medoids (Manhattan)                   0.1147               2
0.4440               0.0267

Partition Agreement (Adjusted Rand Index)
Euclidean vs spherical  : 0.2259
Euclidean vs Manhattan  : 0.1257
Spherical vs Manhattan  : 0.1956
```

Silhouette values in the table are each computed under their own metric and are therefore not directly comparable across rows. The Rand indices and the default-rate spread are the comparable quantities: they show whether the three geometries are describing the same borrower population differently or finding genuinely different structure.

Cluster Profiles in Business Terms

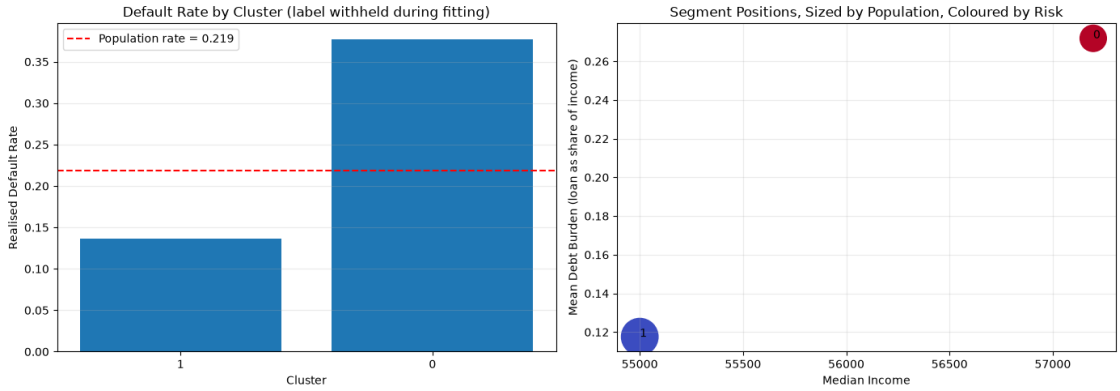
[11]:
```python
final_kmeans = KMeans(
    n_clusters=selected_k,
    n_init=25,
    random_state=42
)

final_labels = final_kmeans.fit_predict(selected_matrix)

profile_frame = df_model.copy()
profile_frame["cluster"] = final_labels

cluster_profile = profile_frame.groupby("cluster").agg(
    Borrowers=("loan_status", "size"),
    Median_Income=("person_income", "median"),
    Median_Loan=("loan_amnt", "median"),
    Mean_Debt_Burden=("loan_percent_income", "mean"),
    Mean_Interest_Rate=("loan_int_rate", "mean"),
    Median_Age=("person_age", "median"),
    Mean_Employment_Years=("person_emp_length", "mean"),
    Default_Rate=("loan_status", "mean")
).round(4)

cluster_profile["Population Share"] = (
    cluster_profile["Borrowers"]
    / cluster_profile["Borrowers"].sum()
```

```
).round(4)

print(f"Cluster Profiles at k={selected_k}")
print(cluster_profile.to_string())

overall_default_rate = y_holdout.mean()

cluster_profile_sorted = cluster_profile.sort_values("Default_Rate")

default_spread = (
    cluster_profile["Default_Rate"].max()
    - cluster_profile["Default_Rate"].min()
)

print(f"\nPopulation default rate: {overall_default_rate:.4f}")
print(f"Lowest-risk cluster default rate: {cluster_profile['Default_Rate'].
 ↪min():.4f}")
print(f"Highest-risk cluster default rate: {cluster_profile['Default_Rate'].
 ↪max():.4f}")
print(f"Spread across clusters: {default_spread:.4f}")
print(
    "Risk ratio, highest to lowest cluster:",
    f"{cluster_profile['Default_Rate'].max() /␣
 ↪max(cluster_profile['Default_Rate'].min(), 1e-9):.2f}x"
)

if default_spread > 0.10:
    print(
        "\n The segments differ substantially in realised default "
        "rate despite the label never entering the clustering. The geometry "
        "of borrower attributes carries genuine risk information, which is "
        "the premise the supervised models of Weeks 4 through 9 relied on."
    )
else:
    print(
        "\n Default rates are similar across segments. K-Means "
        "has partitioned this population along dimensions largely orthogonal "
        "to credit risk, which is an honest negative result and a real "
        "limitation of unsupervised segmentation on this dataset."
    )

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(
    cluster_profile_sorted.index.astype(str),
    cluster_profile_sorted["Default_Rate"]
)
```

```
axes[0].axhline(
    overall_default_rate,
    linestyle="--",
    color="red",
    label=f"Population rate = {overall_default_rate:.3f}"
)
axes[0].set_xlabel("Cluster")
axes[0].set_ylabel("Realised Default Rate")
axes[0].set_title("Default Rate by Cluster (label withheld during fitting)")
axes[0].legend()
axes[0].grid(axis="y", alpha=0.25)

axes[1].scatter(
    cluster_profile["Median_Income"],
    cluster_profile["Mean_Debt_Burden"],
    s=cluster_profile["Borrowers"] / 20,
    c=cluster_profile["Default_Rate"],
    cmap="coolwarm"
)
for cluster_id, row in cluster_profile.iterrows():
    axes[1].annotate(
        str(cluster_id),
        (row["Median_Income"], row["Mean_Debt_Burden"])
    )
axes[1].set_xlabel("Median Income")
axes[1].set_ylabel("Mean Debt Burden (loan as share of income)")
axes[1].set_title("Segment Positions, Sized by Population, Coloured by Risk")
axes[1].grid(alpha=0.25)

plt.tight_layout()
plt.show()
```

```
Cluster Profiles at k=2
         Borrowers  Median_Income  Median_Loan  Mean_Debt_Burden
Mean_Interest_Rate  Median_Age  Mean_Employment_Years  Default_Rate  Population
Share
cluster
0            11064        57196.0      15000.0            0.2719
12.5280        26.0                4.9371        0.3774            0.3414
1            21345        55000.0       6000.0            0.1176
10.2384        26.0                4.6704        0.1364            0.6586

Population default rate: 0.2187
Lowest-risk cluster default rate: 0.1364
Highest-risk cluster default rate: 0.3774
Spread across clusters: 0.2410
Risk ratio, highest to lowest cluster: 2.77x
```

The segments differ substantially in realised default rate despite the label never entering the clustering. The geometry of borrower attributes carries genuine risk information, which is the premise the supervised models of Weeks 4 through 9 relied on.

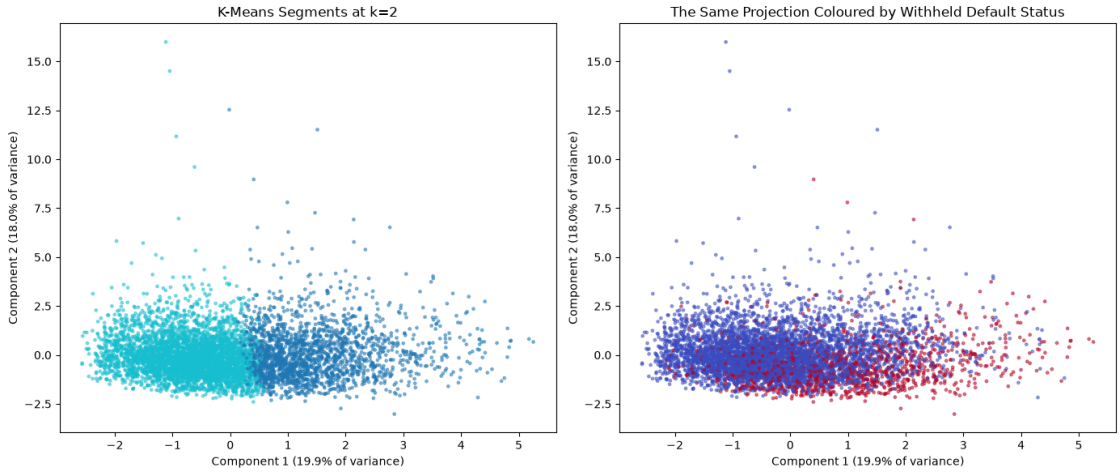


Visualising the Clusters in Two Dimensions

[12]:
```
pca = PCA(n_components=2, random_state=42)
X_projected = pca.fit_transform(selected_matrix)

explained = pca.explained_variance_ratio_

print(f"Variance explained by component 1: {explained[0]:.4f}")
print(f"Variance explained by component 2: {explained[1]:.4f}")
print(f"Total variance captured in this projection: {explained.sum():.4f}")

plot_rng = np.random.default_rng(42)
plot_indices = plot_rng.choice(
    X_projected.shape[0],
    size=min(6000, X_projected.shape[0]),
    replace=False
)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

scatter_clusters = axes[0].scatter(
    X_projected[plot_indices, 0],
    X_projected[plot_indices, 1],
    c=final_labels[plot_indices],
    cmap="tab10",
    s=6,
    alpha=0.5
)
axes[0].set_xlabel(f"Component 1 ({explained[0]:.1%} of variance)")
```

```
axes[0].set_ylabel(f"Component 2 ({explained[1]:.1%} of variance)")
axes[0].set_title(f"K-Means Segments at k={selected_k}")

scatter_defaults = axes[1].scatter(
    X_projected[plot_indices, 0],
    X_projected[plot_indices, 1],
    c=y_holdout.values[plot_indices],
    cmap="coolwarm",
    s=6,
    alpha=0.5
)
axes[1].set_xlabel(f"Component 1 ({explained[0]:.1%} of variance)")
axes[1].set_ylabel(f"Component 2 ({explained[1]:.1%} of variance)")
axes[1].set_title("The Same Projection Coloured by Withheld Default Status")

plt.tight_layout()
plt.show()

print(
    "\n Comparing the two panels shows whether the cluster "
    "boundaries K-Means drew align with where defaults actually concentrate. "
    "Visible correspondence supports the default-rate spread measured in "
    "Section 8; visible mismatch means the segments are capturing structure "
    "that is real but not risk-related."
)
```

```
Variance explained by component 1: 0.1995
Variance explained by component 2: 0.1802
Total variance captured in this projection: 0.3797
```




```
 Comparing the two panels shows whether the cluster boundaries K-Means drew
```

align with where defaults actually concentrate. Visible correspondence supports the default-rate spread measured in Section 8; visible mismatch means the segments are capturing structure that is real but not risk-related.

Stability Under Different Scaling and Feature Choices

Four perturbations are applied, each defensible and each one an analyst could plausibly have chosen instead:

- **RobustScaler** centres on the median and scales by the interquartile range, so the extreme incomes in this dataset pull the transformation far less than they do under StandardScaler.
- **MinMaxScaler** compresses every feature into a fixed zero-to-one range, which changes the relative weight of the one-hot indicators against the continuous attributes.
- **Dropping `loan_grade`** removes the lender-assigned artifact identified in Week 3, leaving only attributes intrinsic to the borrower.
- **A different random seed** changes centroid initialisation, testing whether the algorithm is even converging to the same solution on identical data.

[13]:
```
robust_preprocessor = ColumnTransformer(
    transformers=[
        ("num", RobustScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            categorical_features
        )
    ]
)

minmax_preprocessor = ColumnTransformer(
    transformers=[
        ("num", MinMaxScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            categorical_features
        )
    ]
)

reduced_categoricals = [
    column for column in categorical_features
    if column != "loan_grade"
]

no_grade_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        (
```

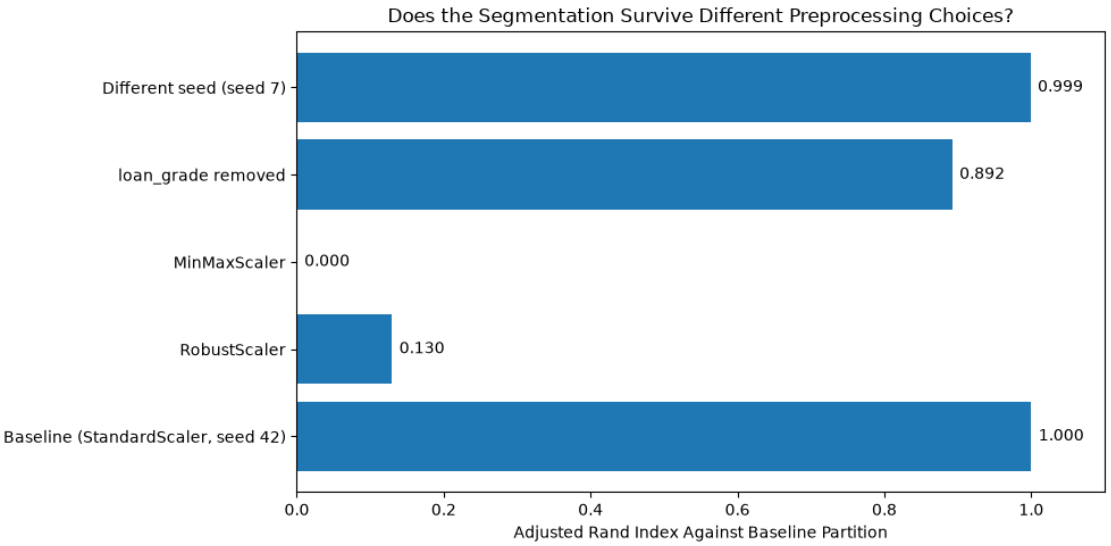
```
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            reduced_categoricals
        )
    ]
)

stability_variants = [
    (
        "Baseline (StandardScaler, seed 42)",
        selected_matrix,
        42
    ),
    (
        "RobustScaler",
        np.asarray(robust_preprocessor.fit_transform(X), dtype=np.float64),
        42
    ),
    (
        "MinMaxScaler",
        np.asarray(minmax_preprocessor.fit_transform(X), dtype=np.float64),
        42
    ),
    (
        "loan_grade removed",
        np.asarray(no_grade_preprocessor.fit_transform(X), dtype=np.float64),
        42
    ),
    (
        "Different seed (seed 7)",
        selected_matrix,
        7
    )
]

stability_results = []

baseline_labels = None

for label, matrix, seed in stability_variants:
    labels = KMeans(
        n_clusters=selected_k,
        n_init=10,
        random_state=seed
    ).fit_predict(matrix)

    if baseline_labels is None:
```

```
        baseline_labels = labels

    default_rate_by_cluster = pd.Series(
        y_holdout.values
    ).groupby(labels).mean()

    stability_results.append({
        "Variant": label,
        "Agreement With Baseline": adjusted_rand_score(
            baseline_labels,
            labels
        ),
        "Silhouette": silhouette_score(
            matrix,
            labels,
            sample_size=5000,
            random_state=42
        ),
        "Default Rate Spread": (
            default_rate_by_cluster.max()
            - default_rate_by_cluster.min()
        ),
        "Smallest Cluster Share": (
            pd.Series(labels).value_counts(normalize=True).min()
        )
    })

stability_df = pd.DataFrame(stability_results)

print(f"Cluster Stability at k={selected_k}")
print(stability_df.round(4).to_string(index=False))

perturbation_agreements = stability_df.loc[1:, "Agreement With Baseline"]
weakest_agreement = perturbation_agreements.min()
weakest_variant = stability_df.loc[
    perturbation_agreements.idxmin(),
    "Variant"
]

print(f"\nWeakest agreement: {weakest_variant} at {weakest_agreement:.4f}")

if weakest_agreement > 0.6:
    print(
        " The segmentation survives every perturbation tested. "
        "The clusters reflect structure in the borrower population rather "
        "than an accident of one preprocessing choice, which is the "
        "precondition for reporting them to a decision-maker."
```

```
    )
elif weakest_agreement > 0.3:
    print(
        "The partition is moderately stable. Its broad shape "
        "survives, but individual borrowers near cluster boundaries move "
        "between segments depending on preprocessing, so segment membership "
        "should not be treated as a fixed property of a borrower."
    )
else:
    print(
        " The partition is not stable under reasonable "
        "preprocessing changes. This is the most important finding in the "
        "notebook and it argues against operationalising these specific "
        "segments, whatever their silhouette score suggests."
    )

plt.figure(figsize=(10, 5))
plt.barh(
    stability_df["Variant"],
    stability_df["Agreement With Baseline"]
)
for index, value in enumerate(stability_df["Agreement With Baseline"]):
    plt.text(
        value + 0.01,
        index,
        f"{value:.3f}",
        va="center"
    )
plt.xlabel("Adjusted Rand Index Against Baseline Partition")
plt.title("Does the Segmentation Survive Different Preprocessing Choices?")
plt.xlim(0, 1.1)
plt.tight_layout()
plt.show()
```

```
Cluster Stability at k=2
                          Variant  Agreement With Baseline  Silhouette  Default
Rate Spread  Smallest Cluster Share
Baseline (StandardScaler, seed 42)                   1.0000      0.1560
0.2410                  0.3415
                      RobustScaler                   0.1295      0.1863
0.0798                  0.2571
                      MinMaxScaler                   0.0003      0.1794
0.1978                  0.4943
                loan_grade removed                   0.8923      0.1675
0.2348                  0.3345
           Different seed (seed 7)                   0.9989      0.1561
0.2410                  0.3415
```

```
Weakest agreement: MinMaxScaler at 0.0003
 The partition is not stable under reasonable preprocessing changes. This is the
most important finding in the notebook and it argues against operationalising
these specific segments, whatever their silhouette score suggests.
```



Does Clustering Improve the Supervised Model?

Two versions are tested against the Week 9 tuned gradient boosting benchmark:

1. **Cluster as a feature** - the segment label is appended to the feature matrix as one-hot indicators and a single model is fit as before.
2. **Per-cluster specialists** - a separate gradient boosting model is fit within each segment, and test borrowers are scored by whichever specialist owns their segment.

The leakage discipline matters here. K-Means is fit on the training features only, then used to assign segments to test borrowers, exactly as it would be in production where tomorrow's applicants were not available when the segmentation was built.

[14]:
```
X_supervised_train, X_supervised_test, y_supervised_train, y_supervised_test = (
    train_test_split(
        X,
        y_holdout,
        test_size=0.20,
        random_state=42,
        stratify=y_holdout
    )
)

supervised_encoder = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numeric_features),
```

```
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            categorical_features
        )
    ]
)

X_sup_train_enc = np.asarray(
    supervised_encoder.fit_transform(X_supervised_train),
    dtype=np.float32
)
X_sup_test_enc = np.asarray(
    supervised_encoder.transform(X_supervised_test),
    dtype=np.float32
)

clustering_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            categorical_features
        )
    ]
)

X_cluster_train = clustering_preprocessor.fit_transform(X_supervised_train)
X_cluster_test = clustering_preprocessor.transform(X_supervised_test)

segmenter = KMeans(
    n_clusters=selected_k,
    n_init=25,
    random_state=42
)

train_segments = segmenter.fit_predict(X_cluster_train)
test_segments = segmenter.predict(X_cluster_test)

print(f"Segments assigned using k={selected_k}")
print("\nTraining segment sizes and default rates")
print(
    pd.DataFrame({
        "Segment": train_segments,
        "Default": np.asarray(y_supervised_train)
    }).groupby("Segment").agg(
```

```
        Borrowers=("Default", "size"),
        Default_Rate=("Default", "mean")
    ).round(4).to_string()
)


def fit_gradient_boosting(X_fit, y_fit):
    model = GradientBoostingClassifier(
        subsample=0.85,
        n_estimators=300,
        min_samples_leaf=3,
        max_features=None,
        max_depth=4,
        learning_rate=0.10,
        random_state=42
    )

    weights = compute_sample_weight(
        class_weight="balanced",
        y=y_fit
    )

    model.fit(X_fit, y_fit, sample_weight=weights)

    return model


baseline_model = fit_gradient_boosting(
    X_sup_train_enc,
    y_supervised_train
)

baseline_probabilities = baseline_model.predict_proba(
    X_sup_test_enc
)[:, 1]

segment_indicator_train = np.zeros(
    (len(train_segments), selected_k),
    dtype=np.float32
)
segment_indicator_train[
    np.arange(len(train_segments)),
    train_segments
] = 1.0

segment_indicator_test = np.zeros(
    (len(test_segments), selected_k),
```

```
    dtype=np.float32
)
segment_indicator_test[
    np.arange(len(test_segments)),
    test_segments
] = 1.0

X_augmented_train = np.hstack(
    [X_sup_train_enc, segment_indicator_train]
)
X_augmented_test = np.hstack(
    [X_sup_test_enc, segment_indicator_test]
)

augmented_model = fit_gradient_boosting(
    X_augmented_train,
    y_supervised_train
)

augmented_probabilities = augmented_model.predict_proba(
    X_augmented_test
)[:, 1]

specialist_probabilities = np.zeros(len(y_supervised_test))

y_supervised_train_array = np.asarray(y_supervised_train)

minimum_segment_size = 500

for segment in range(selected_k):
    train_mask = train_segments == segment
    test_mask = test_segments == segment

    if test_mask.sum() == 0:
        continue

    segment_y = y_supervised_train_array[train_mask]

    if train_mask.sum() < minimum_segment_size or len(np.unique(segment_y)) < 2:
        specialist_probabilities[test_mask] = baseline_probabilities[test_mask]
        print(
            f"Segment {segment}: too small or single-class "
            f"({train_mask.sum()} training borrowers), "
            "falling back to the pooled model."
        )
        continue
```

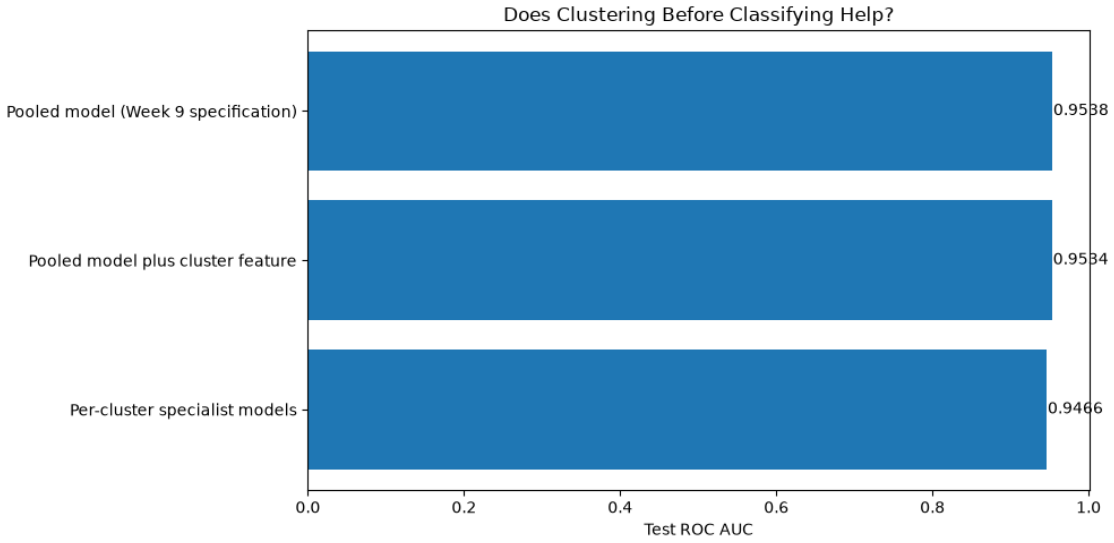
```
    specialist = fit_gradient_boosting(
        X_sup_train_enc[train_mask],
        segment_y
    )

    specialist_probabilities[test_mask] = specialist.predict_proba(
        X_sup_test_enc[test_mask]
    )[:, 1]

two_stage_comparison = pd.DataFrame([
    {
        "Approach": "Pooled model (Week 9 specification)",
        "Test ROC AUC": roc_auc_score(
            y_supervised_test,
            baseline_probabilities
        ),
        "Test Average Precision": average_precision_score(
            y_supervised_test,
            baseline_probabilities
        )
    },
    {
        "Approach": "Pooled model plus cluster feature",
        "Test ROC AUC": roc_auc_score(
            y_supervised_test,
            augmented_probabilities
        ),
        "Test Average Precision": average_precision_score(
            y_supervised_test,
            augmented_probabilities
        )
    },
    {
        "Approach": "Per-cluster specialist models",
        "Test ROC AUC": roc_auc_score(
            y_supervised_test,
            specialist_probabilities
        ),
        "Test Average Precision": average_precision_score(
            y_supervised_test,
            specialist_probabilities
        )
    }
])

print("\nTwo-Stage Architecture Comparison")
print(two_stage_comparison.round(4).to_string(index=False))
```

```
augmented_gain = (
    two_stage_comparison.loc[1, "Test ROC AUC"]
    - two_stage_comparison.loc[0, "Test ROC AUC"]
)

specialist_gain = (
    two_stage_comparison.loc[2, "Test ROC AUC"]
    - two_stage_comparison.loc[0, "Test ROC AUC"]
)

print(f"\nCluster feature changes AUC by {augmented_gain:+.4f}")
print(f"Per-cluster specialists change AUC by {specialist_gain:+.4f}")

best_gain = max(augmented_gain, specialist_gain)

if best_gain > 0.002:
    print(
        "\n Pre-segmenting borrowers improves risk ranking. The "
        "gain must still be weighed against the cost of explaining a "
        "two-stage pipeline to a regulator, since a lender must be able to "
        "justify a declined application."
    )
else:
    print(
        "\nPre-segmenting does not improve on the pooled model. "
        "This is informative rather than disappointing: gradient boosting "
        "already partitions the feature space internally through its splits, "
        "so an explicit K-Means partition supplies information the trees had "
        "already recovered. The simpler single-stage pipeline is preferred, "
        "and it is also the easier one to defend to a regulator."
    )

plt.figure(figsize=(10, 5))
auc_order = two_stage_comparison.sort_values("Test ROC AUC")
plt.barh(
    auc_order["Approach"],
    auc_order["Test ROC AUC"]
)
for index, value in enumerate(auc_order["Test ROC AUC"]):
    plt.text(
        value + 0.001,
        index,
        f"{value:.4f}",
        va="center"
    )
plt.xlabel("Test ROC AUC")
```

```
plt.title("Does Clustering Before Classifying Help?")
plt.tight_layout()
plt.show()
```

```
Segments assigned using k=2

Training segment sizes and default rates
         Borrowers  Default_Rate
Segment
0            17086        0.1349
1             8841        0.3806

Two-Stage Architecture Comparison
                           Approach  Test ROC AUC  Test Average Precision
Pooled model (Week 9 specification)        0.9538                  0.9113
  Pooled model plus cluster feature        0.9534                  0.9109
      Per-cluster specialist models        0.9466                  0.9039

Cluster feature changes AUC by -0.0003
Per-cluster specialists change AUC by -0.0072
```

Pre-segmenting does not improve on the pooled model. This is informative rather than disappointing: gradient boosting already partitions the feature space internally through its splits, so an explicit K-Means partition supplies information the trees had already recovered. The simpler single-stage pipeline is preferred, and it is also the easier one to defend to a regulator.



Week11_Gueorgui_Poklitar

August 1, 2026

******Week 11 - Clustering Part 2: DBSCAN and Hierarchical Agglomerative Clustering******

[1]:
```
%pip install -q pandas numpy matplotlib scipy scikit-learn kagglehub
```

```
[notice] A new release of pip is
available: 26.0.1 -> 26.2
[notice] To update, run:
pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
```

[2]:
```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy
import sklearn
import kagglehub

from scipy.cluster.hierarchy import (
    cophenet,
    dendrogram,
    fcluster,
    linkage
)
from scipy.spatial.distance import pdist
from sklearn.cluster import AgglomerativeClustering, DBSCAN, KMeans
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from sklearn.metrics import (
    adjusted_mutual_info_score,
    adjusted_rand_score,
    calinski_harabasz_score,
    davies_bouldin_score,
    silhouette_score
)
```

```
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import OneHotEncoder, StandardScaler

pd.set_option("display.max_columns", 100)
pd.set_option("display.width", 180)

print(f"scikit-learn version: {sklearn.__version__}")
print(f"scipy version: {scipy.__version__}")
print("All imports successful.")
```

```
scikit-learn version: 1.9.0
scipy version: 1.18.0
All imports successful.
```

Data Loading and Cleaning

[3]:
```
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")

df = df.drop_duplicates().copy()

df["person_emp_length"] = df["person_emp_length"].fillna(
    df["person_emp_length"].median()
)

df["loan_int_rate"] = df["loan_int_rate"].fillna(
    df.groupby("loan_grade")["loan_int_rate"].transform("median")
)

df = df[df["person_emp_length"] <= df["person_age"]]
df = df[df["person_age"] <= 100]
df = df[df["person_emp_length"] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate: {df['loan_status'].mean() * 100:.2f}%")
```

```
Raw shape: (32581, 12)
Clean shape: (32409, 12)
Default rate: 21.87%
```

Feature Preparation and the Sampling Decision

[4]:
```
numeric_features = [
    "person_income",
    "person_age",
    "person_emp_length",
    "loan_amnt",
```

```
    "loan_percent_income",
    "loan_int_rate"
]

categorical_features = [
    "loan_intent",
    "loan_grade",
    "person_home_ownership",
    "cb_person_default_on_file"
]

X = df[numeric_features + categorical_features].copy()
y_holdout = df["loan_status"].copy()

complete_mask = X.notna().all(axis=1)
X = X.loc[complete_mask]
y_holdout = y_holdout.loc[complete_mask]
df_model = df.loc[complete_mask].copy()

scaled_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(
                handle_unknown="ignore",
                sparse_output=False
            ),
            categorical_features
        )
    ]
)

X_scaled = np.asarray(
    scaled_preprocessor.fit_transform(X),
    dtype=np.float64
)

categorical_names = (
    scaled_preprocessor.named_transformers_["cat"]
    .get_feature_names_out(categorical_features)
    .tolist()
)
feature_names = numeric_features + categorical_names

sample_rng = np.random.default_rng(42)
```

```
dbscan_sample_size = 12000
hierarchical_sample_size = 4000
dendrogram_sample_size = 1200

dbscan_indices = sample_rng.choice(
    X_scaled.shape[0],
    size=min(dbscan_sample_size, X_scaled.shape[0]),
    replace=False
)

hierarchical_indices = sample_rng.choice(
    X_scaled.shape[0],
    size=min(hierarchical_sample_size, X_scaled.shape[0]),
    replace=False
)

dendrogram_indices = hierarchical_indices[:dendrogram_sample_size]

X_dbscan = X_scaled[dbscan_indices]
y_dbscan = y_holdout.values[dbscan_indices]
df_dbscan = df_model.iloc[dbscan_indices].copy()

X_hierarchical = X_scaled[hierarchical_indices]
y_hierarchical = y_holdout.values[hierarchical_indices]
df_hierarchical = df_model.iloc[hierarchical_indices].copy()

X_dendrogram = X_scaled[dendrogram_indices]

print(f"Full population available: {X_scaled.shape[0]:,} borrowers")
print(f"Feature space dimensions: {X_scaled.shape[1]}")
print(f"DBSCAN working sample: {X_dbscan.shape[0]:,}")
print(f"Hierarchical working sample: {X_hierarchical.shape[0]:,}")
print(f"Dendrogram display sample: {X_dendrogram.shape[0]:,}")
print(f"\nWithheld default rate, full population: {y_holdout.mean() * 100:.
  ↪2f}%")
print(f"Withheld default rate, DBSCAN sample: {y_dbscan.mean() * 100:.2f}%")
print(
    f"Withheld default rate, hierarchical sample: "
    f"{y_hierarchical.mean() * 100:.2f}%"
)
```

```
Full population available: 32,409 borrowers
Feature space dimensions: 25
DBSCAN working sample: 12,000
Hierarchical working sample: 4,000
Dendrogram display sample: 1,200

Withheld default rate, full population: 21.87%
```

```
Withheld default rate, DBSCAN sample: 22.18%
Withheld default rate, hierarchical sample: 22.25%
```

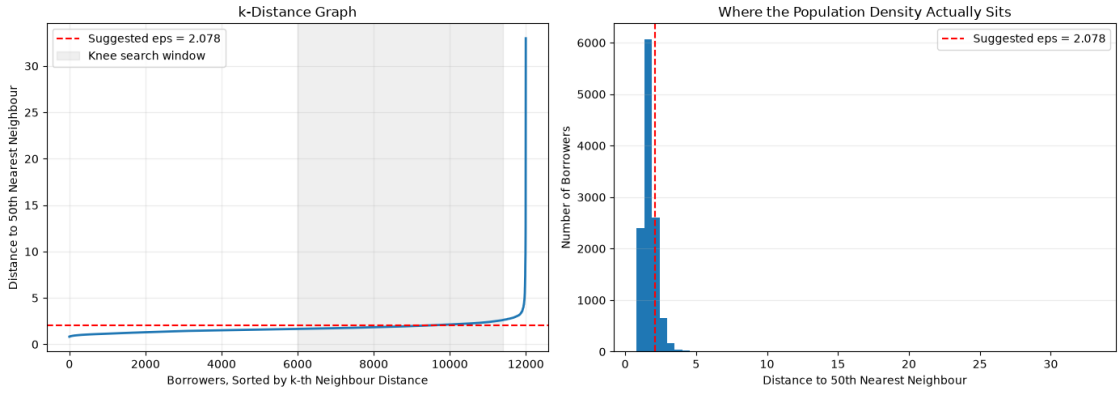
Choosing eps: The k-Distance Graph

[5]:
```python
min_samples_reference = 2 * X_dbscan.shape[1]

print(
    f"Rule-of-thumb min_samples for {X_dbscan.shape[1]} dimensions "
    f"(twice the dimension count): {min_samples_reference}"
)

neighbour_model = NearestNeighbors(n_neighbors=min_samples_reference)
neighbour_model.fit(X_dbscan)

neighbour_distances, _ = neighbour_model.kneighbors(X_dbscan)

kth_distances = np.sort(neighbour_distances[:, -1])
n_points = len(kth_distances)

percentile_points = [50, 75, 90, 95, 97, 99]

print("\nDistribution of the k-th Nearest Neighbour Distance")
for percentile in percentile_points:
    print(
        f"  {percentile}th percentile: "
        f"{np.percentile(kth_distances, percentile):.4f}"
    )


lower_bound = int(0.50 * n_points)
upper_bound = int(0.95 * n_points)

segment_x = np.arange(lower_bound, upper_bound, dtype=float)
segment_y = kth_distances[lower_bound:upper_bound]

x_norm = (segment_x - segment_x[0]) / (segment_x[-1] - segment_x[0])
y_norm = (segment_y - segment_y[0]) / (segment_y[-1] - segment_y[0])

chord_gap = x_norm - y_norm

knee_position = lower_bound + int(np.argmax(chord_gap))
knee_eps = float(kth_distances[knee_position])
knee_percentile = 100.0 * knee_position / n_points

print(f"\nEstimated knee at index {knee_position:,} of {n_points:,}")
print(
    f"That is the {knee_percentile:.1f}th percentile "
```

```
    "of the k-distance curve"
)
print(f"Suggested eps from the k-distance graph: {knee_eps:.4f}")

if knee_percentile > 97.0:
    print(
        "[Warning] The knee sits in the outlier tail. Treat this eps as an "
        "upper bound rather than a starting point, or DBSCAN will merge the "
        "whole population into a single cluster."
    )

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(kth_distances, linewidth=2)
axes[0].axhline(
    knee_eps,
    linestyle="--",
    color="red",
    label=f"Suggested eps = {knee_eps:.3f}"
)
axes[0].axvspan(
    lower_bound,
    upper_bound,
    color="grey",
    alpha=0.12,
    label="Knee search window"
)
axes[0].set_xlabel("Borrowers, Sorted by k-th Neighbour Distance")
axes[0].set_ylabel(f"Distance to {min_samples_reference}th Nearest Neighbour")
axes[0].set_title("k-Distance Graph")
axes[0].legend()
axes[0].grid(alpha=0.25)

axes[1].hist(kth_distances, bins=60)
axes[1].axvline(
    knee_eps,
    linestyle="--",
    color="red",
    label=f"Suggested eps = {knee_eps:.3f}"
)
axes[1].set_xlabel(f"Distance to {min_samples_reference}th Nearest Neighbour")
axes[1].set_ylabel("Number of Borrowers")
axes[1].set_title("Where the Population Density Actually Sits")
axes[1].legend()
axes[1].grid(axis="y", alpha=0.25)

plt.tight_layout()
```

```
plt.show()
```

```
Rule-of-thumb min_samples for 25 dimensions (twice the dimension count): 50

Distribution of the k-th Nearest Neighbour Distance
  50th percentile: 1.6573
  75th percentile: 1.9641
  90th percentile: 2.3184
  95th percentile: 2.6218
  97th percentile: 2.8309
  99th percentile: 3.4156

Estimated knee at index 9,718 of 12,000
That is the 81.0th percentile of the k-distance curve
Suggested eps from the k-distance graph: 2.0781
```



DBSCAN Parameter Sweep

The sweep below varies `eps` around the suggested value and `min_samples` across three settings, and reports what each combination produces. Four outputs are tracked, and reading them together is the whole skill:

- **Clusters found** is discovered rather than specified. One cluster means `eps` was large enough to connect the entire population into a single blob; dozens of tiny clusters means it was too small and the algorithm is fragmenting the dense core.
- **Noise share** is the fraction of borrowers assigned no cluster. A very low figure means DBSCAN is behaving like K-Means and absorbing everyone; a very high figure means it found little density anywhere.
- **Silhouette excluding noise** measures separation among the clustered points only. Including noise would corrupt the score, since noise is not a cluster and its members are not supposed to be near each other.
- **Largest cluster share** exposes the most common outcome on data like this: one enormous cluster plus a handful of specks, which is a degenerate result even when the silhouette looks respectable.

[6]:
```
eps_candidates = [
    round(knee_eps * multiplier, 4)
    for multiplier in [0.30, 0.45, 0.60, 0.80, 1.00, 1.25]
]

min_samples_candidates = [
    max(5, min_samples_reference // 4),
    max(10, min_samples_reference // 2),
    min_samples_reference
]

print(f"eps values tested: {eps_candidates}")
print(f"min_samples values tested: {min_samples_candidates}")

dbscan_sweep_results = []
dbscan_label_store = {}

for eps_value in eps_candidates:
    for min_samples_value in min_samples_candidates:
        model = DBSCAN(
            eps=eps_value,
            min_samples=min_samples_value,
            n_jobs=-1
        )

        labels = model.fit_predict(X_dbscan)

        noise_mask = labels == -1
        clustered_mask = ~noise_mask

        unique_clusters = np.unique(labels[clustered_mask])
        cluster_count = len(unique_clusters)

        if cluster_count >= 2 and clustered_mask.sum() > 100:
            silhouette = silhouette_score(
                X_dbscan[clustered_mask],
                labels[clustered_mask],
                sample_size=min(4000, int(clustered_mask.sum())),
                random_state=42
            )
        else:
            silhouette = np.nan

        if cluster_count >= 1:
            largest_share = (
                pd.Series(labels[clustered_mask]).value_counts().iloc[0]
                / len(labels)
```

```
            )
        else:
            largest_share = np.nan

        if noise_mask.sum() > 0:
            noise_default_rate = y_dbscan[noise_mask].mean()
        else:
            noise_default_rate = np.nan

        dbscan_sweep_results.append({
            "eps": eps_value,
            "min_samples": min_samples_value,
            "Clusters Found": cluster_count,
            "Noise Share": noise_mask.mean(),
            "Silhouette (no noise)": silhouette,
            "Largest Cluster Share": largest_share,
            "Noise Default Rate": noise_default_rate
        })

        dbscan_label_store[(eps_value, min_samples_value)] = labels

dbscan_sweep_df = pd.DataFrame(dbscan_sweep_results)

print("\nDBSCAN Parameter Sweep")
print(dbscan_sweep_df.round(4).to_string(index=False))

multi_cluster_options = dbscan_sweep_df[
    (dbscan_sweep_df["Clusters Found"] >= 2)
    & (dbscan_sweep_df["Noise Share"] < 0.60)
    & (dbscan_sweep_df["Largest Cluster Share"] < 0.98)
].copy()

core_and_noise_options = dbscan_sweep_df[
    (dbscan_sweep_df["Clusters Found"] == 1)
    & (dbscan_sweep_df["Noise Share"] > 0.02)
    & (dbscan_sweep_df["Noise Share"] < 0.50)
].copy()

if len(multi_cluster_options) > 0:
    chosen = multi_cluster_options.loc[
        multi_cluster_options["Silhouette (no noise)"].idxmax()
    ]
    selection_regime = "multiple dense clusters"
elif len(core_and_noise_options) > 0:
    chosen = core_and_noise_options.loc[
        core_and_noise_options["Noise Share"].idxmax()
    ]
```

```
    selection_regime = "single dense core plus noise"
else:
    chosen = dbscan_sweep_df.loc[
        dbscan_sweep_df["Clusters Found"].idxmax()
    ]
    selection_regime = "fully degenerate"

chosen_eps = float(chosen["eps"])
chosen_min_samples = int(chosen["min_samples"])

print(
    f"\nSelected configuration: eps={chosen_eps}, "
    f"min_samples={chosen_min_samples}"
)
print(f"  Regime: {selection_regime}")
print(
    f"  Clusters: {int(chosen['Clusters Found'])}, "
    f"noise share: {chosen['Noise Share']:.4f}"
)

if selection_regime == "multiple dense clusters":
    print(
        "\n DBSCAN located more than one region of genuine "
        "density, so the population is not a single undifferentiated mass. "
        "Section 9 tests whether these regions correspond to the segments "
        "K-Means and Ward found."
    )
elif selection_regime == "single dense core plus noise":
    print(
        "\n Across the entire parameter grid DBSCAN found one "
        "dense region rather than several, with the remainder labelled "
        "noise. This is not a tuning failure, it is a substantive finding "
        "about the shape of the portfolio: borrower attributes form one "
        "continuous mass with sparse atypical applicants at its edges, "
        "rather than several distinct populations. It also means the "
        "K-Means partition of Week 10 was imposing boundaries on a "
        "continuum rather than discovering natural separations, which is "
        "the single most important caveat on any segmentation reported from "
        "this dataset. The core-versus-noise split remains directly useful, "
        "and Section 5 tests it against the withheld outcome."
    )
else:
    print(
        "\n No configuration produced either multiple clusters "
        "or a usable core-and-noise split. Falling back to the setting "
        "finding the most clusters. The degeneracy is itself the result: "
        "on a dense continuous population in twenty-five dimensions, "
```

```
        "nineteen of them binary, density-based clustering has no sparse "
        "gaps to exploit."
    )

dbscan_labels = dbscan_label_store[(chosen_eps, chosen_min_samples)]

pivot_clusters = dbscan_sweep_df.pivot(
    index="eps",
    columns="min_samples",
    values="Clusters Found"
)

pivot_noise = dbscan_sweep_df.pivot(
    index="eps",
    columns="min_samples",
    values="Noise Share"
)

print("\nClusters Found by Parameter Combination")
print(pivot_clusters.to_string())

print("\nNoise Share by Parameter Combination")
print(pivot_noise.round(4).to_string())

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for min_samples_value in min_samples_candidates:
    subset = dbscan_sweep_df[
        dbscan_sweep_df["min_samples"] == min_samples_value
    ]
    axes[0].plot(
        subset["eps"],
        subset["Clusters Found"],
        marker="o",
        linewidth=2,
        label=f"min_samples = {min_samples_value}"
    )
    axes[1].plot(
        subset["eps"],
        subset["Noise Share"],
        marker="s",
        linewidth=2,
        label=f"min_samples = {min_samples_value}"
    )

axes[0].set_xlabel("eps")
axes[0].set_ylabel("Number of Clusters Discovered")
```

```
axes[0].set_title("DBSCAN Discovers the Cluster Count")
axes[0].legend()
axes[0].grid(alpha=0.25)

axes[1].set_xlabel("eps")
axes[1].set_ylabel("Share of Borrowers Labelled Noise")
axes[1].set_title("Noise Share Falls as the Radius Grows")
axes[1].legend()
axes[1].grid(alpha=0.25)

plt.tight_layout()
plt.show()
```

```
eps values tested: [0.6234, 0.9351, 1.2468, 1.6624, 2.0781, 2.5976]
min_samples values tested: [12, 25, 50]

DBSCAN Parameter Sweep
   eps  min_samples  Clusters Found  Noise Share  Silhouette (no noise)  Largest
Cluster Share  Noise Default Rate
0.6234           12              26       0.9203                 0.3276
0.0103              0.2354
0.6234           25               2       0.9957                    NaN
0.0022              0.2225
0.6234           50               0       1.0000                    NaN
NaN                 0.2218
0.9351           12              49       0.5616                 0.1624
0.0283              0.3052
0.9351           25              23       0.7488                 0.2456
0.0243              0.2690
0.9351           50               8       0.9228                 0.3348
0.0158              0.2355
1.2468           12              64       0.3310                 0.1217
0.0338              0.3328
1.2468           25              36       0.4784                 0.1655
0.0314              0.3292
1.2468           50              26       0.6299                 0.1932
0.0285              0.2975
1.6624           12               1       0.0997                    NaN
0.9003              0.4005
1.6624           25               1       0.1429                    NaN
0.8571              0.3843
1.6624           50               1       0.2108                    NaN
0.7892              0.3636
2.0781           12               1       0.0282                    NaN
0.9718              0.3894
2.0781           25               1       0.0366                    NaN
0.9634              0.3759
2.0781           50               1       0.0508                    NaN
```

```
0.9492              0.4039
2.5976          12              1       0.0062                  NaN
0.9938              0.2400
2.5976          25              1       0.0084                  NaN
0.9916              0.2574
2.5976          50              1       0.0107                  NaN
0.9892              0.2868

Selected configuration: eps=1.2468, min_samples=25
  Regime: multiple dense clusters
  Clusters: 36, noise share: 0.4784
```

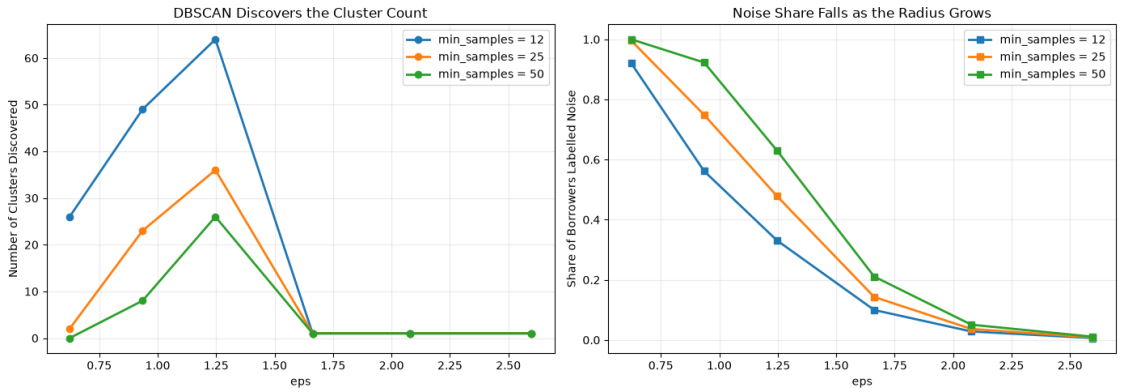
DBSCAN located more than one region of genuine density, so the population is not a single undifferentiated mass. Section 9 tests whether these regions correspond to the segments K-Means and Ward found.

Clusters Found by Parameter Combination

eps	min_samples 12	25	50
0.6234	26	2	0
0.9351	49	23	8
1.2468	64	36	26
1.6624	1	1	1
2.0781	1	1	1
2.5976	1	1	1

Noise Share by Parameter Combination

eps	min_samples 12	25	50
0.6234	0.9203	0.9957	1.0000
0.9351	0.5616	0.7488	0.9228
1.2468	0.3310	0.4784	0.6299
1.6624	0.0997	0.1429	0.2108
2.0781	0.0282	0.0366	0.0508
2.5976	0.0062	0.0084	0.0107



The Noise Points Are the Finding

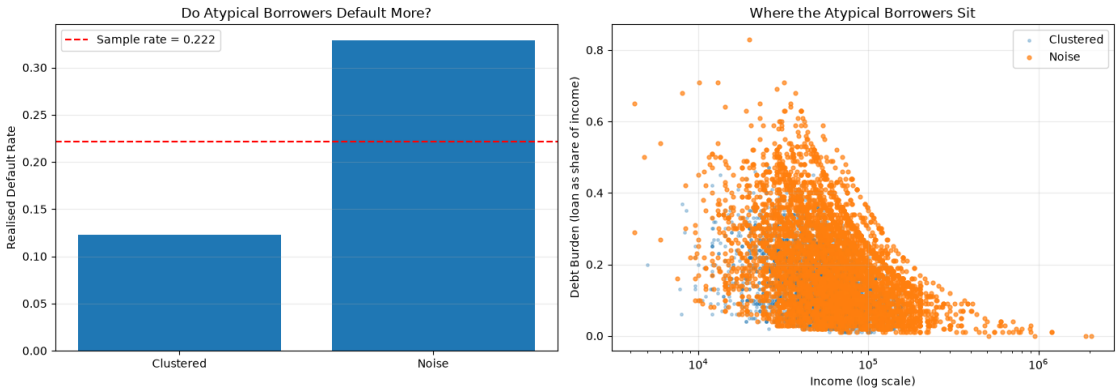
[7]:
```
noise_mask = dbscan_labels == -1
clustered_mask = ~noise_mask

print(f"Borrowers labelled noise: {noise_mask.sum():,} ({noise_mask.mean():.
 ↪2%})")
print(f"Borrowers assigned to a cluster: {clustered_mask.sum():,}")
print(f"Clusters discovered: {len(np.unique(dbscan_labels[clustered_mask]))}")

if noise_mask.sum() > 0 and clustered_mask.sum() > 0:
    noise_comparison = pd.DataFrame({
        "Group": ["Clustered (typical)", "Noise (atypical)"],
        "Borrowers": [clustered_mask.sum(), noise_mask.sum()],
        "Median Income": [
            df_dbscan.loc[clustered_mask, "person_income"].median(),
            df_dbscan.loc[noise_mask, "person_income"].median()
        ],
        "Median Loan": [
            df_dbscan.loc[clustered_mask, "loan_amnt"].median(),
            df_dbscan.loc[noise_mask, "loan_amnt"].median()
        ],
        "Mean Debt Burden": [
            df_dbscan.loc[clustered_mask, "loan_percent_income"].mean(),
            df_dbscan.loc[noise_mask, "loan_percent_income"].mean()
        ],
        "Mean Interest Rate": [
            df_dbscan.loc[clustered_mask, "loan_int_rate"].mean(),
            df_dbscan.loc[noise_mask, "loan_int_rate"].mean()
        ],
        "Median Age": [
            df_dbscan.loc[clustered_mask, "person_age"].median(),
            df_dbscan.loc[noise_mask, "person_age"].median()
        ],
        "Default Rate": [
            y_dbscan[clustered_mask].mean(),
            y_dbscan[noise_mask].mean()
        ]
    })

    print("\nTypical Versus Atypical Borrowers")
    print(noise_comparison.round(4).to_string(index=False))

    noise_lift = (
        y_dbscan[noise_mask].mean()
```

```
    / max(y_dbscan[clustered_mask].mean(), 1e-9)
)

print(f"\nDefault rate lift for noise borrowers: {noise_lift:.2f}x")

if noise_lift > 1.25:
    print(
        " Borrowers with no dense neighbourhood default at a "
        "materially higher rate. DBSCAN has produced an unsupervised "
        "screen for atypical applications, which a lender could use to "
        "route files to manual underwriting without any labelled "
        "training data."
    )
elif noise_lift < 0.80:
    print(
        " Noise borrowers default at a *lower* rate than the "
        "dense core. Atypical does not mean risky here, and a naive "
        "outlier-based decline rule would reject better-than-average "
        "business."
    )
else:
    print(
        " Noise borrowers default at roughly the population "
        "rate. Being unusual in feature space carries no risk "
        "information on this dataset, which is a useful correction to an "
        "intuitive but unfounded assumption."
    )

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(
    ["Clustered", "Noise"],
    [
        y_dbscan[clustered_mask].mean(),
        y_dbscan[noise_mask].mean()
    ]
)
axes[0].axhline(
    y_dbscan.mean(),
    linestyle="--",
    color="red",
    label=f"Sample rate = {y_dbscan.mean():.3f}"
)
axes[0].set_ylabel("Realised Default Rate")
axes[0].set_title("Do Atypical Borrowers Default More?")
axes[0].legend()
axes[0].grid(axis="y", alpha=0.25)
```

```
    axes[1].scatter(
        df_dbscan.loc[clustered_mask, "person_income"],
        df_dbscan.loc[clustered_mask, "loan_percent_income"],
        s=6,
        alpha=0.3,
        label="Clustered"
    )
    axes[1].scatter(
        df_dbscan.loc[noise_mask, "person_income"],
        df_dbscan.loc[noise_mask, "loan_percent_income"],
        s=12,
        alpha=0.7,
        label="Noise"
    )
    axes[1].set_xscale("log")
    axes[1].set_xlabel("Income (log scale)")
    axes[1].set_ylabel("Debt Burden (loan as share of income)")
    axes[1].set_title("Where the Atypical Borrowers Sit")
    axes[1].legend()
    axes[1].grid(alpha=0.25)

    plt.tight_layout()
    plt.show()
else:
    print(
        "\n This configuration produced no noise points or no "
        "clustered points, so the typical-versus-atypical comparison cannot "
        "be made. That degeneracy is itself informative about how DBSCAN "
        "behaves on a dense, continuous population in high dimensions."
    )
```

```
Borrowers labelled noise: 5,741 (47.84%)
Borrowers assigned to a cluster: 6,259
Clusters discovered: 36

Typical Versus Atypical Borrowers
              Group  Borrowers  Median Income  Median Loan  Mean Debt Burden
Mean Interest Rate  Median Age  Default Rate
Clustered (typical)       6259        52000.0       7000.0            0.1496
9.6125        25.0        0.1232
   Noise (atypical)       5741        60000.0      10000.0            0.1932
12.6185        28.0         0.3292

Default rate lift for noise borrowers: 2.67x
 Borrowers with no dense neighbourhood default at a materially higher rate.
DBSCAN has produced an unsupervised screen for atypical applications, which a
lender could use to route files to manual underwriting without any labelled
```

training data.

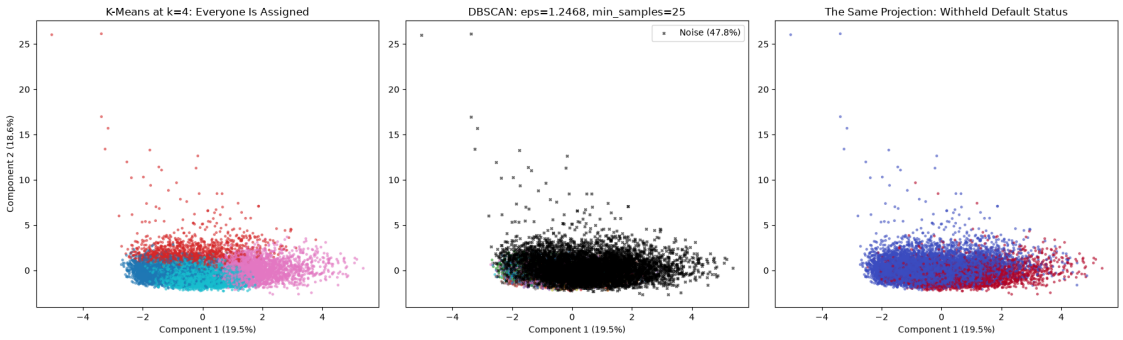


Visualising DBSCAN Against K-Means

[8]:
```
pca = PCA(n_components=2, random_state=42)
X_projected = pca.fit_transform(X_dbscan)

explained = pca.explained_variance_ratio_

print(f"Variance captured in this projection: {explained.sum():.4f}")

reference_kmeans_labels = KMeans(
    n_clusters=4,
    n_init=25,
    random_state=42
).fit_predict(X_dbscan)

fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))

axes[0].scatter(
    X_projected[:, 0],
    X_projected[:, 1],
    c=reference_kmeans_labels,
    cmap="tab10",
    s=5,
    alpha=0.5
)
axes[0].set_title("K-Means at k=4: Everyone Is Assigned")
axes[0].set_xlabel(f"Component 1 ({explained[0]:.1%})")
axes[0].set_ylabel(f"Component 2 ({explained[1]:.1%})")

axes[1].scatter(
    X_projected[clustered_mask, 0],
    X_projected[clustered_mask, 1],
```

```
    c=dbscan_labels[clustered_mask],
    cmap="tab10",
    s=5,
    alpha=0.5
)
if noise_mask.sum() > 0:
    axes[1].scatter(
        X_projected[noise_mask, 0],
        X_projected[noise_mask, 1],
        c="black",
        s=8,
        alpha=0.6,
        marker="x",
        label=f"Noise ({noise_mask.mean():.1%})"
    )
    axes[1].legend()
axes[1].set_title(
    f"DBSCAN: eps={chosen_eps}, min_samples={chosen_min_samples}"
)
axes[1].set_xlabel(f"Component 1 ({explained[0]:.1%})")

axes[2].scatter(
    X_projected[:, 0],
    X_projected[:, 1],
    c=y_dbscan,
    cmap="coolwarm",
    s=5,
    alpha=0.5
)
axes[2].set_title("The Same Projection: Withheld Default Status")
axes[2].set_xlabel(f"Component 1 ({explained[0]:.1%})")

plt.tight_layout()
plt.show()
```

```
Variance captured in this projection: 0.3815
```



Hierarchical Agglomerative Clustering

The critical choice is the linkage method, which defines the distance between two *groups* of points rather than two points. The definition matters more than intuition suggests:

- **Ward** merges whichever pair produces the smallest increase in total within-cluster variance. It is the closest analogue to the K-Means objective and tends to produce compact, similarly sized clusters. It requires Euclidean distance for the same reason K-Means does.
- **Complete** measures group distance as the distance between the two *furthest* members, so a merge is only cheap when every member of one group is close to every member of the other. This resists elongated clusters and produces compact ones.
- **Average** uses the mean distance across all cross-group pairs, a middle course between complete and single.
- **Single** uses the *closest* pair of members, so two groups merge if any one point in each is nearby. This can follow irregular shapes that the others cannot, but it is prone to chaining: a thin bridge of intermediate points causes two otherwise distinct groups to fuse, and one enormous straggling cluster absorbs almost everything.

The cophenetic correlation reported below measures how faithfully the tree's implied distances reproduce the original pairwise distances, giving a single number for how much each linkage method distorts the data.

[9]:
```
linkage_methods = ["ward", "complete", "average", "single"]

pairwise_condensed = pdist(X_hierarchical, metric="euclidean")

linkage_matrices = {}
cophenetic_scores = {}

for method in linkage_methods:
    linkage_matrix = linkage(
        X_hierarchical,
        method=method,
        metric="euclidean"
    )

    linkage_matrices[method] = linkage_matrix

    correlation, _ = cophenet(linkage_matrix, pairwise_condensed)
    cophenetic_scores[method] = correlation

print("Cophenetic Correlation by Linkage Method")
print("(how faithfully the tree preserves original distances)")
for method in linkage_methods:
    print(f"  {method:<10}: {cophenetic_scores[method]:.4f}")

best_cophenetic = max(cophenetic_scores, key=cophenetic_scores.get)
```

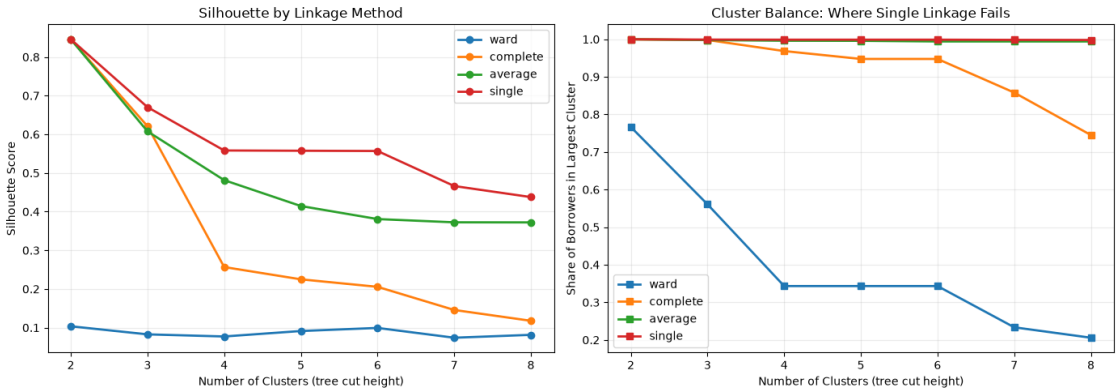
```
print(f"\nHighest cophenetic correlation: {best_cophenetic}")

hierarchical_results = []

cut_k_values = [2, 3, 4, 5, 6, 7, 8]

for method in linkage_methods:
    for k in cut_k_values:
        labels = fcluster(
            linkage_matrices[method],
            t=k,
            criterion="maxclust"
        )

        actual_clusters = len(np.unique(labels))

        if actual_clusters < 2:
            continue

        default_rate_by_cluster = pd.Series(
            y_hierarchical
        ).groupby(labels).mean()

        hierarchical_results.append({
            "Linkage": method,
            "Requested k": k,
            "Actual Clusters": actual_clusters,
            "Silhouette": silhouette_score(X_hierarchical, labels),
            "Davies-Bouldin": davies_bouldin_score(X_hierarchical, labels),
            "Largest Cluster Share": (
                pd.Series(labels).value_counts(normalize=True).iloc[0]
            ),
            "Default Rate Spread": (
                default_rate_by_cluster.max()
                - default_rate_by_cluster.min()
            )
        })

hierarchical_df = pd.DataFrame(hierarchical_results)

print("\nHierarchical Clustering Across Linkage Methods and Cut Heights")
print(hierarchical_df.round(4).to_string(index=False))

print("\nBalance Check: Largest Cluster Share by Linkage (averaged over k)")
balance_summary = hierarchical_df.groupby("Linkage").agg(
    Mean_Largest_Share=("Largest Cluster Share", "mean"),
    Mean_Silhouette=("Silhouette", "mean"),
```

```
    Mean_Default_Spread=("Default Rate Spread", "mean")
).round(4)
print(balance_summary.to_string())

single_share = balance_summary.loc["single", "Mean_Largest_Share"]
ward_share = balance_summary.loc["ward", "Mean_Largest_Share"]

print(
    f"\nSingle linkage puts {single_share:.1%} of borrowers in its largest "
    f"cluster on average; Ward puts {ward_share:.1%}."
)

if single_share > 0.90:
    print(
        " Single linkage has chained, exactly as the theory "
        "predicts on a dense continuous population. Almost every borrower "
        "ends up in one cluster with a few singletons split off. This is not "
        "a bug and it is worth reporting: it demonstrates that a linkage "
        "method can be entirely unsuitable for a dataset regardless of how "
        "it is tuned."
    )

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for method in linkage_methods:
    subset = hierarchical_df[hierarchical_df["Linkage"] == method]
    axes[0].plot(
        subset["Requested k"],
        subset["Silhouette"],
        marker="o",
        linewidth=2,
        label=method
    )
    axes[1].plot(
        subset["Requested k"],
        subset["Largest Cluster Share"],
        marker="s",
        linewidth=2,
        label=method
    )

axes[0].set_xlabel("Number of Clusters (tree cut height)")
axes[0].set_ylabel("Silhouette Score")
axes[0].set_title("Silhouette by Linkage Method")
axes[0].legend()
axes[0].grid(alpha=0.25)
```

```
axes[1].set_xlabel("Number of Clusters (tree cut height)")
axes[1].set_ylabel("Share of Borrowers in Largest Cluster")
axes[1].set_title("Cluster Balance: Where Single Linkage Fails")
axes[1].legend()
axes[1].grid(alpha=0.25)

plt.tight_layout()
plt.show()
```

```
Cophenetic Correlation by Linkage Method
(how faithfully the tree preserves original distances)
  ward      : 0.3366
  complete  : 0.6233
  average   : 0.7519
  single    : 0.7218

Highest cophenetic correlation: average

Hierarchical Clustering Across Linkage Methods and Cut Heights
 Linkage  Requested k  Actual Clusters  Silhouette  Davies-Bouldin  Largest
Cluster Share  Default Rate Spread
    ward            2                2      0.1035          2.5803
0.7665               0.2363
    ward            3                3      0.0826          2.4095
0.5608               0.2944
    ward            4                4      0.0771          2.4071
0.3432               0.3228
    ward            5                5      0.0913          2.1634
0.3432               0.3228
    ward            6                6      0.0992          1.9246
0.3432               0.3228
    ward            7                7      0.0739          1.9762
0.2335               0.3279
    ward            8                8      0.0814          2.0326
0.2058               0.3611
complete            2                2      0.8457          0.1090
0.9998               0.2226
complete            3                3      0.6209          0.4850
0.9980               0.2857
complete            4                4      0.2567          1.0374
0.9685               0.2857
complete            5                5      0.2249          1.1960
0.9475               0.2857
complete            6                6      0.2055          1.1234
0.9475               0.2857
complete            7                7      0.1457          1.3918
0.8580               0.3547
complete            8                8      0.1177          1.5302
```

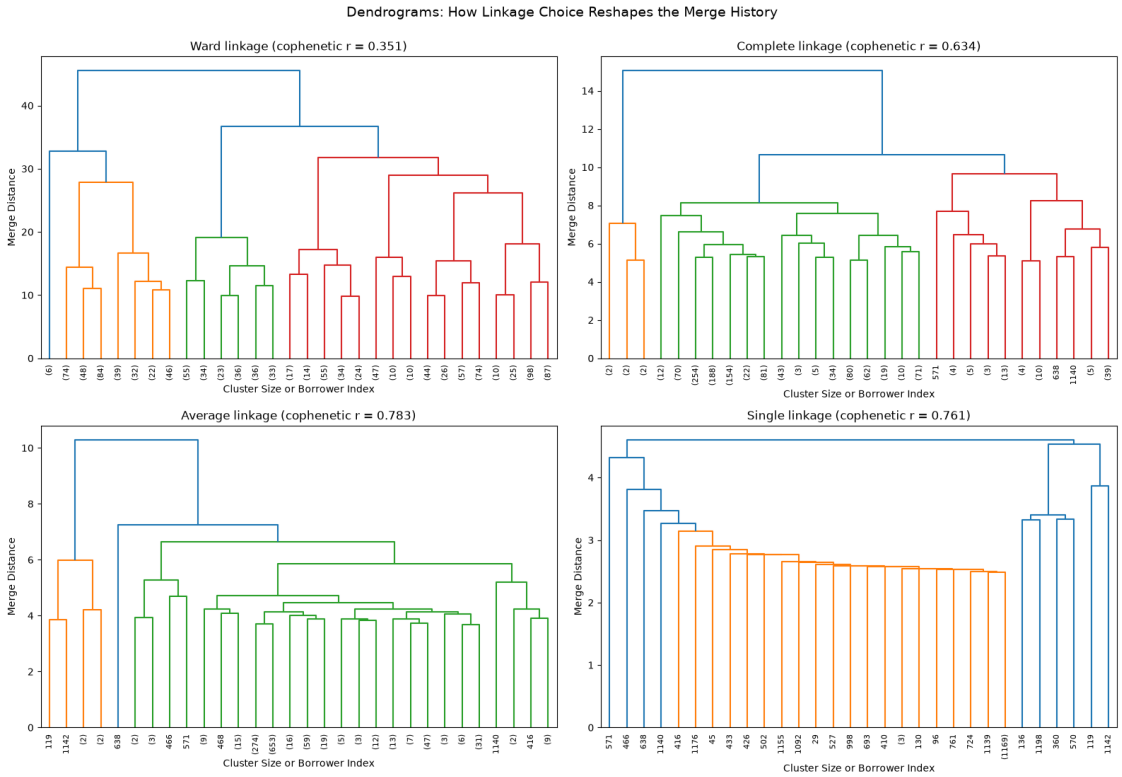
```
0.7450              0.5597
 average            2               2      0.8457          0.1090
0.9998              0.2226
 average            3               3      0.6076          0.5171
0.9978              0.3750
 average            4               4      0.4814          0.7066
0.9960              0.3750
 average            5               5      0.4145          0.6684
0.9958              1.0000
 average            6               6      0.3808          0.7079
0.9940              1.0000
 average            7               7      0.3724          0.7031
0.9940              1.0000
 average            8               8      0.3722          0.8059
0.9940              1.0000
  single            2               2      0.8457          0.1090
0.9998              0.2226
  single            3               3      0.6702          0.3458
0.9990              0.3333
  single            4               4      0.5582          0.3398
0.9988              0.3333
  single            5               5      0.5575          0.3966
0.9988              0.5000
  single            6               6      0.5569          0.2459
0.9988              1.0000
  single            7               7      0.4662          0.3290
0.9982              1.0000
  single            8               8      0.4376          0.3373
0.9980              1.0000

Balance Check: Largest Cluster Share by Linkage (averaged over k)
          Mean_Largest_Share  Mean_Silhouette  Mean_Default_Spread
Linkage
average               0.9959           0.4964               0.7104
complete              0.9235           0.3453               0.3257
single                0.9988           0.5846               0.6270
ward                  0.3995           0.0870               0.3126

Single linkage puts 99.9% of borrowers in its largest cluster on average; Ward
puts 40.0%.
 Single linkage has chained, exactly as the theory predicts on a dense
continuous population. Almost every borrower ends up in one cluster with a few
singletons split off. This is not a bug and it is worth reporting: it
demonstrates that a linkage method can be entirely unsuitable for a dataset
regardless of how it is tuned.
```



Dendrograms

[10]:
```
fig, axes = plt.subplots(2, 2, figsize=(16, 11))
axes = axes.flatten()

dendrogram_linkages = {}

for axis, method in zip(axes, linkage_methods):
    linkage_matrix = linkage(
        X_dendrogram,
        method=method,
        metric="euclidean"
    )

    dendrogram_linkages[method] = linkage_matrix

    dendrogram(
        linkage_matrix,
        ax=axis,
        truncate_mode="lastp",
        p=30,
        show_leaf_counts=True,
        no_labels=False,
        leaf_rotation=90,
        leaf_font_size=8
    )

    correlation, _ = cophenet(
        linkage_matrix,
        pdist(X_dendrogram, metric="euclidean")
    )

    axis.set_title(
```

```
        f"{method.capitalize()} linkage "
        f"(cophenetic r = {correlation:.3f})"
    )
    axis.set_xlabel("Cluster Size or Borrower Index")
    axis.set_ylabel("Merge Distance")

plt.suptitle(
    "Dendrograms: How Linkage Choice Reshapes the Merge History",
    y=1.00,
    fontsize=14
)
plt.tight_layout()
plt.show()

print("Largest Merge Distances by Linkage Method")
print("(a large final gap indicates well-separated top-level groups)")

merge_gap_summary = []

for method in linkage_methods:
    merge_heights = dendrogram_linkages[method][:, 2]
    top_merges = np.sort(merge_heights)[-6:]
    gaps = np.diff(top_merges)

    merge_gap_summary.append({
        "Linkage": method,
        "Final Merge Height": top_merges[-1],
        "Largest Gap Near Top": gaps.max() if len(gaps) > 0 else np.nan,
        "Gap as Share of Final Height": (
            gaps.max() / top_merges[-1] if len(gaps) > 0 else np.nan
        )
    })

print(pd.DataFrame(merge_gap_summary).round(4).to_string(index=False))
```



Largest Merge Distances by Linkage Method
(a large final gap indicates well-separated top-level groups)

Linkage	Final Merge Height	Largest Gap Near Top	Gap as Share of Final Height
ward	45.5362	8.8394	0.1941
complete	15.0577	4.4041	0.2925
average	10.2904	3.0473	0.2961
single	4.6040	0.4529	0.0984

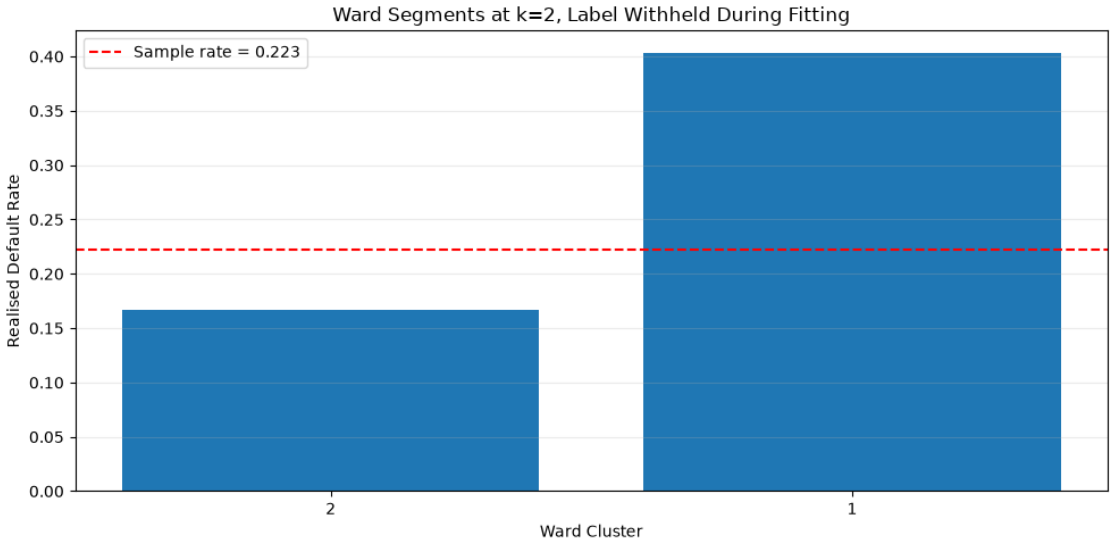
Cutting the Ward tree

[11]:
```
ward_linkage = linkage_matrices["ward"]

ward_silhouettes = hierarchical_df[
    hierarchical_df["Linkage"] == "ward"
]

best_ward_k = int(
    ward_silhouettes.loc[
        ward_silhouettes["Silhouette"].idxmax(),
        "Requested k"
    ]
)
```

```
print(f"Best Ward cut by silhouette: k={best_ward_k}")

ward_labels = fcluster(
    ward_linkage,
    t=best_ward_k,
    criterion="maxclust"
)

ward_profile_frame = df_hierarchical.copy()
ward_profile_frame["cluster"] = ward_labels

ward_profile = ward_profile_frame.groupby("cluster").agg(
    Borrowers=("loan_status", "size"),
    Median_Income=("person_income", "median"),
    Median_Loan=("loan_amnt", "median"),
    Mean_Debt_Burden=("loan_percent_income", "mean"),
    Mean_Interest_Rate=("loan_int_rate", "mean"),
    Median_Age=("person_age", "median"),
    Default_Rate=("loan_status", "mean")
).round(4)

ward_profile["Population Share"] = (
    ward_profile["Borrowers"] / ward_profile["Borrowers"].sum()
).round(4)

print(f"\nWard Cluster Profiles at k={best_ward_k}")
print(ward_profile.to_string())

ward_default_spread = (
    ward_profile["Default_Rate"].max()
    - ward_profile["Default_Rate"].min()
)

print(f"\nSample default rate: {y_hierarchical.mean():.4f}")
print(f"Default rate spread across Ward clusters: {ward_default_spread:.4f}")
print(
    "Risk ratio, highest to lowest:",
    f"{ward_profile['Default_Rate'].max() / max(ward_profile['Default_Rate'].
 ↪min(), 1e-9):.2f}x"
)

plt.figure(figsize=(10, 5))
ward_sorted = ward_profile.sort_values("Default_Rate")
plt.bar(
    ward_sorted.index.astype(str),
    ward_sorted["Default_Rate"]
)
```

```
plt.axhline(
    y_hierarchical.mean(),
    linestyle="--",
    color="red",
    label=f"Sample rate = {y_hierarchical.mean():.3f}"
)
plt.xlabel("Ward Cluster")
plt.ylabel("Realised Default Rate")
plt.title(f"Ward Segments at k={best_ward_k}, Label Withheld During Fitting")
plt.legend()
plt.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Best Ward cut by silhouette: k=2

Ward Cluster Profiles at k=2
         Borrowers  Median_Income  Median_Loan  Mean_Debt_Burden
Mean_Interest_Rate  Median_Age  Default_Rate  Population Share
cluster
1              934        55000.0      15000.0            0.2934
11.2242         25.0         0.4036           0.2335
2             3066        56000.0       6875.0            0.1357
10.9909         26.0         0.1673           0.7665

Sample default rate: 0.2225
Default rate spread across Ward clusters: 0.2363
Risk ratio, highest to lowest: 2.41x
```



Do the Three Algorithms Agree?

The three methods rest on incompatible premises. K-Means minimises variance around a fixed number of centroids, DBSCAN grows clusters from regions of density and discards the rest, and Ward merges to minimise variance increase while preserving a hierarchy. If all three converge on similar partitions, that is strong evidence of real structure, because three different objective functions found the same groups. If they disagree sharply, the honest conclusion is that this population has no single natural segmentation and any reported clustering is partly a statement about the algorithm rather than about the borrowers.

Agreement is measured two ways. Adjusted Rand index counts agreeing pairs of points, corrected for chance. Adjusted mutual information measures shared information between the two labellings and handles unbalanced cluster sizes more gracefully. Both read 1.0 for identical partitions and near 0.0 for unrelated ones. The comparison runs on the hierarchical sample, since that is the only sample all three methods can be fit on.

[12]:
```
kmeans_comparison_labels = KMeans(
    n_clusters=best_ward_k,
    n_init=25,
    random_state=42
).fit_predict(X_hierarchical)

dbscan_comparison = DBSCAN(
    eps=chosen_eps,
    min_samples=chosen_min_samples,
    n_jobs=-1
).fit_predict(X_hierarchical)

complete_labels = fcluster(
    linkage_matrices["complete"],
    t=best_ward_k,
    criterion="maxclust"
)

average_labels = fcluster(
    linkage_matrices["average"],
    t=best_ward_k,
    criterion="maxclust"
)

partitions = {
    "K-Means": kmeans_comparison_labels,
    "Ward HAC": ward_labels,
    "Complete HAC": complete_labels,
    "Average HAC": average_labels,
    "DBSCAN": dbscan_comparison
}
```

```
method_names = list(partitions.keys())

rand_matrix = pd.DataFrame(
    index=method_names,
    columns=method_names,
    dtype=float
)

mutual_info_matrix = pd.DataFrame(
    index=method_names,
    columns=method_names,
    dtype=float
)

for first in method_names:
    for second in method_names:
        rand_matrix.loc[first, second] = adjusted_rand_score(
            partitions[first],
            partitions[second]
        )
        mutual_info_matrix.loc[first, second] = adjusted_mutual_info_score(
            partitions[first],
            partitions[second]
        )

print("Adjusted Rand Index Between Methods")
print(rand_matrix.round(4).to_string())

print("\nAdjusted Mutual Information Between Methods")
print(mutual_info_matrix.round(4).to_string())

print("\nClusters Produced by Each Method on the Shared Sample")
for name, labels in partitions.items():
    noise_count = int((labels == -1).sum())
    cluster_count = len(np.unique(labels[labels != -1]))
    print(
        f"  {name:<14}: {cluster_count} clusters, "
        f"{noise_count:,} noise points"
    )

kmeans_ward_agreement = rand_matrix.loc["K-Means", "Ward HAC"]

print(
    f"\nK-Means versus Ward agreement: {kmeans_ward_agreement:.4f}"
)

if kmeans_ward_agreement > 0.6:
```

```
    print(
        " K-Means and Ward largely agree, which is expected given "
        "that both minimise within-cluster variance, and it confirms the "
        "Week 10 partition was not an artifact of the K-Means algorithm "
        "specifically."
    )
else:
    print(
        " Even the two variance-minimising methods disagree "
        "substantially. On a dense continuous population, cluster boundaries "
        "are largely a choice made by the algorithm rather than a feature of "
        "the data, and this is the most important caveat attached to any "
        "segmentation reported from this dataset."
    )

fig, axes = plt.subplots(1, 2, figsize=(15, 6))

for axis, matrix, title in [
    (axes[0], rand_matrix, "Adjusted Rand Index"),
    (axes[1], mutual_info_matrix, "Adjusted Mutual Information")
]:
    image = axis.imshow(
        matrix.values.astype(float),
        cmap="viridis",
        vmin=0,
        vmax=1
    )
    axis.set_xticks(range(len(method_names)))
    axis.set_yticks(range(len(method_names)))
    axis.set_xticklabels(method_names, rotation=45, ha="right")
    axis.set_yticklabels(method_names)
    axis.set_title(title)

    for row in range(len(method_names)):
        for column in range(len(method_names)):
            axis.text(
                column,
                row,
                f"{matrix.values[row, column]:.2f}",
                ha="center",
                va="center",
                color="white"
            )

    plt.colorbar(image, ax=axis, fraction=0.046)

plt.suptitle("Do Three Different Objectives Find the Same Segments?", y=1.02)
```

```
plt.tight_layout()
plt.show()
```

```
Adjusted Rand Index Between Methods
              K-Means  Ward HAC  Complete HAC  Average HAC  DBSCAN
K-Means        1.0000    0.3197       -0.0002      -0.0002 -0.0748
Ward HAC       0.3197    1.0000       -0.0003      -0.0003 -0.0777
Complete HAC  -0.0002   -0.0003        1.0000       1.0000 -0.0004
Average HAC   -0.0002   -0.0003        1.0000       1.0000 -0.0004
DBSCAN        -0.0748   -0.0777       -0.0004      -0.0004  1.0000

Adjusted Mutual Information Between Methods
              K-Means  Ward HAC  Complete HAC  Average HAC  DBSCAN
K-Means        1.0000    0.2364       -0.0002      -0.0002  0.0907
Ward HAC       0.2364    1.0000       -0.0003      -0.0003  0.0273
Complete HAC  -0.0002   -0.0003        1.0000       1.0000 -0.0004
Average HAC   -0.0002   -0.0003        1.0000       1.0000 -0.0004
DBSCAN         0.0907    0.0273       -0.0004      -0.0004  1.0000

Clusters Produced by Each Method on the Shared Sample
  K-Means       : 2 clusters, 0 noise points
  Ward HAC      : 2 clusters, 0 noise points
  Complete HAC  : 2 clusters, 0 noise points
  Average HAC   : 2 clusters, 0 noise points
  DBSCAN        : 15 clusters, 3,098 noise points

K-Means versus Ward agreement: 0.3197
 Even the two variance-minimising methods disagree substantially. On a dense
continuous population, cluster boundaries are largely a choice made by the
algorithm rather than a feature of the data, and this is the most important
caveat attached to any segmentation reported from this dataset.
```